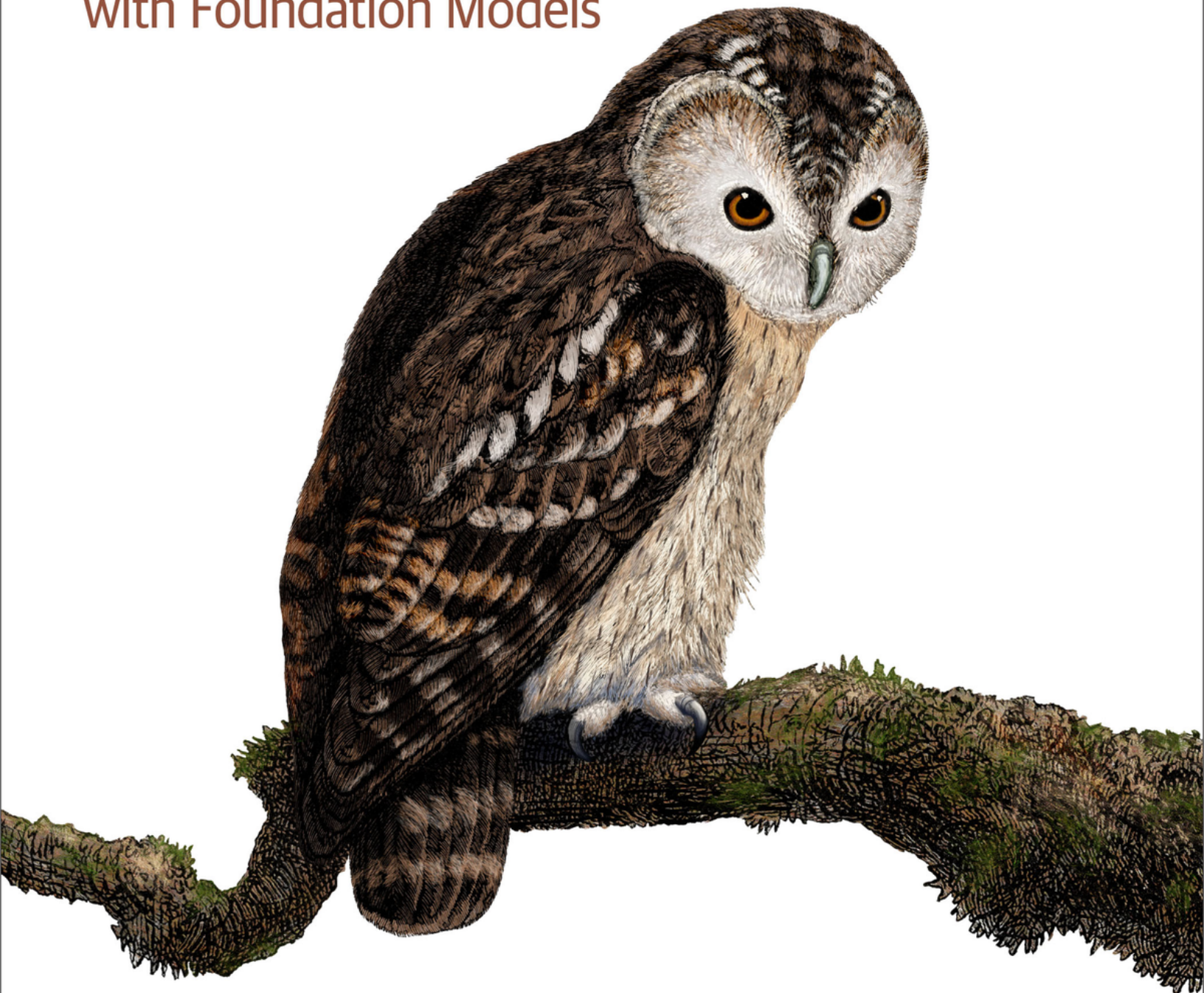


O'REILLY®

AI Engineering

Building Applications
with Foundation Models



Chip Huyen

You can think of RAG as a technique to construct context specific to each query, instead of using the same context for all queries. This helps with managing user data, as it allows you to include data specific to a user only in queries related to this user.

Context construction for foundation models is equivalent to feature engineering for classical ML models. They serve the same purpose: giving the model the necessary information to process an input.

In the early days of foundation models, RAG emerged as one of the most common patterns. Its main purpose was to overcome the models' context limitations. Many people think that a sufficiently long context will be the end of RAG. I don't think so. First, no matter how long a model's context length is, there will be applications that require context longer than that. After all, the amount of available data only grows over time. People generate and add new data but rarely delete data. Context length is expanding quickly, but not fast enough for the data needs of arbitrary applications.⁴

Second, a model that can process long context doesn't necessarily use that context well, as discussed in ["Context Length and Context Efficiency"](#). The longer the context, the more likely the model is to focus on the wrong part of the context. Every extra context token incurs extra cost and has the potential to add extra latency. RAG allows a model to use only the most

relevant information for each query, reducing the number of input tokens while potentially increasing the model's performance.

Efforts to expand context length are happening in parallel with efforts to make models use context more effectively. I wouldn't be surprised if a model provider incorporates a retrieval-like or attention-like mechanism to help a model pick out the most salient parts of a context to use.

NOTE

Anthropic suggested that for Claude models, if “your knowledge base is smaller than 200,000 tokens (about 500 pages of material), you can just include the entire knowledge base in the prompt that you give the model, with no need for RAG or similar methods” ([Anthropic, 2024](#)). It'd be amazing if other model developers provide similar guidance for RAG versus long context for their models.

RAG Architecture

A RAG system has two components: a retriever that retrieves information from external memory sources and a generator that generates a response based on the retrieved information. [Figure 6-2](#) shows a high-level architecture of a RAG system.

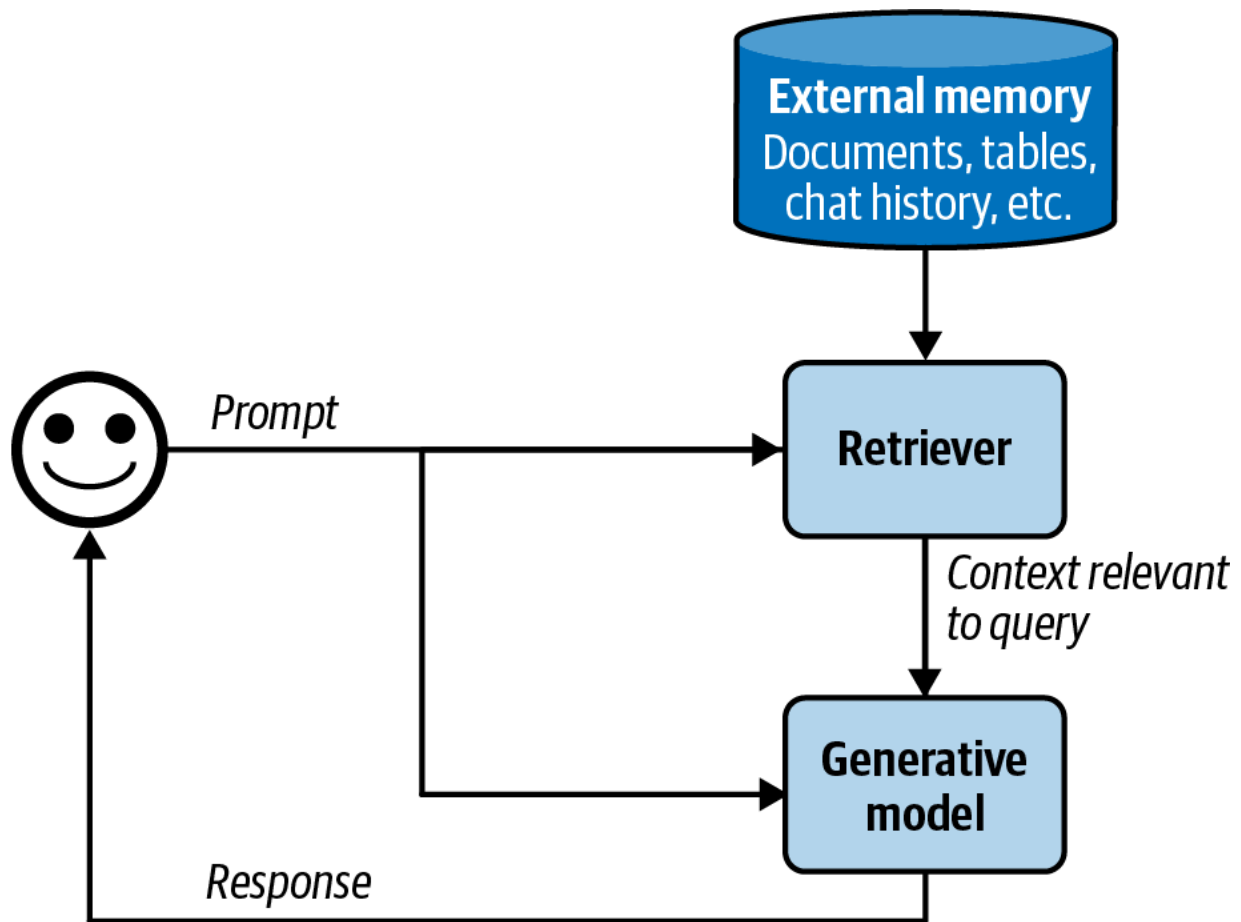


Figure 6-2. A basic RAG architecture.

In the original RAG paper, [Lewis et al.](#) trained the retriever and the generative model together. In today's RAG systems, these two components are often trained separately, and many teams build their RAG systems using off-the-shelf retrievers and models. However, finetuning the whole RAG system end-to-end can improve its performance significantly.

The success of a RAG system depends on the quality of its retriever. A retriever has two main functions: indexing and querying. Indexing involves processing data so that it can be quickly retrieved later. Sending a query to

retrieve data relevant to it is called querying. How to index data depends on how you want to retrieve it later on.

Now that we've covered the primary components, let's consider an example of how a RAG system works. For simplicity, let's assume that the external memory is a database of documents, such as a company's memos, contracts, and meeting notes. A document can be 10 tokens or 1 million tokens.

Naively retrieving whole documents can cause your context to be arbitrarily long. To avoid this, you can split each document into more manageable chunks. Chunking strategies will be discussed later in this chapter. For now, let's assume that all documents have been split into workable chunks. For each query, our goal is to retrieve the data chunks most relevant to this query. Minor post-processing is often needed to join the retrieved data chunks with the user prompt to generate the final prompt. This final prompt is then fed into the generative model.

NOTE

In this chapter, I use the term “document” to refer to both “document” and “chunk”, because technically, a chunk of a document is also a document. I do this to keep this book's terminologies consistent with classical NLP and information retrieval (IR) terminologies.

Retrieval Algorithms

Retrieval isn't unique to RAG. Information retrieval is a century-old idea.⁵ It's the backbone of search engines, recommender systems, log analytics,

etc. Many retrieval algorithms developed for traditional retrieval systems can also be used for RAG. For instance, information retrieval is a fertile research area with a large supporting industry that can hardly be sufficiently covered within a few pages. Accordingly, this section will cover only the broad strokes. See this book's [GitHub repository](#) for more in-depth resources on information retrieval.

NOTE

Retrieval is typically limited to one database or system, whereas search involves retrieval across various systems. This chapter uses retrieval and search interchangeably.

At its core, retrieval works by ranking documents based on their relevance to a given query. Retrieval algorithms differ based on how relevance scores are computed. I'll start with two common retrieval mechanisms: term-based retrieval and embedding-based retrieval.

SPARSE VERSUS DENSE RETRIEVAL

In the literature, you might encounter the division of retrieval algorithms into the following categories: sparse versus dense. This book, however, opted for term-based versus embedding-based categorization.

Sparse retrievers represent data using *sparse vectors*. A sparse vector is a vector where the majority of the values are 0. Term-based retrieval is considered sparse, as each term can be represented using a sparse *one-hot vector*, a vector that is 0 everywhere except one value of 1. The vector size is the length of the vocabulary. The value of 1 is in the index corresponding to the index of the term in the vocabulary.

If we have a simple dictionary, `{"food": 0, "banana": 1, "slug": 2}`, then the one-hot vectors of “food”, “banana”, and “slug” are `[1, 0, 0]`, `[0, 1, 0]`, and `[0, 0, 1]` respectively.

Dense retrievers represent data using *dense vectors*. A dense vector is a vector where the majority of the values aren't 0. Embedding-based retrieval is typically considered dense, as embeddings are generally dense vectors. However, there are also sparse embeddings. For example, SPLADE (Sparse Lexical and Expansion) is a retrieval algorithm that works using sparse embeddings ([Formal et al., 2021](#)). It leverages embeddings generated by BERT but uses regularization to push most embedding values to 0. The sparsity makes embedding operations more efficient.

The sparse versus dense division causes SPLADE to be grouped together with term-based algorithms, even though SPLADE's operations, strengths, and weaknesses are much more similar to those of dense embedding retrieval than those of term-based retrieval. Term-based versus embedding-based division avoids this miscategorization.

Term-based retrieval

Given a query, the most straightforward way to find relevant documents is with keywords. Some people call this approach *lexical retrieval*. For example, given the query “AI engineering”, the model will retrieve all the documents that contain “AI engineering”. However, this approach has two problems:

- Many documents might contain the given term, and your model might not have sufficient context space to include all of them as context. A heuristic is to include the documents that contain the term the greatest number of times. The assumption is that the more a term appears in a document, the more relevant this document is to this term. The number of times a term appears in a document is called *term frequency* (TF).
- A prompt can be long and contain many terms. Some are more important than others. For example, the prompt “Easy-to-follow recipes for Vietnamese food to cook at home” contains nine terms: *easy-to-follow*, *recipes*, *for*, *vietnamese*, *food*, *to*, *cook*, *at*, *home*. You want to focus on

more informative terms like *vietnamese* and *recipes*, not *for* and *at*. You need a way to identify important terms.

An intuition is that the more documents contain a term, the less informative this term is. “For” and “at” are likely to appear in most documents, hence, they are less informative. So a term’s importance is inversely proportional to the number of documents it appears in. This metric is called *inverse document frequency* (IDF). To compute IDF for a term, count all the documents that contain this term, then divide the total number of documents by this count. If there are 10 documents and 5 of them contain a given term, then the IDF of this term is $10 / 5 = 2$. The higher a term’s IDF, the more important it is.

TF-IDF is an algorithm that combines these two metrics: term frequency (TF) and inverse document frequency (IDF). Mathematically, the TF-IDF score of document D for the query Q is computed as follows:

- Let $t_1, t_2, \dots, t_q$ be the terms in the query Q .
- Given a term t , the term frequency of this term in the document D is $f(t, D)$.
- Let N be the total number of documents, and $C(t)$ be the number of documents that contain t . The IDF value of the term t can be written as $\text{IDF}(t) = \log \frac{N}{C(t)}$.
- Naively, the TF-IDF score of a document D with respect to Q is defined as $\text{Score}(D, Q) = \sum_{i=1}^q \text{IDF}(t_i) \times f(t_i, D)$.

Two common term-based retrieval solutions are Elasticsearch and BM25. [Elasticsearch](#) (Shay Banon, 2010), built on top of [Lucene](#), uses a data structure called an inverted index. It's a dictionary that maps from terms to documents that contain them. This dictionary allows for fast retrieval of documents given a term. The index might also store additional information such as the term frequency and the document count (how many documents contain this term), which are helpful for computing TF-IDF scores. [Table 6-1](#) illustrates an inverted index.

Table 6-1. A simplified example of an inverted index.

Term	Document count	(Document index, term frequency) for all documents containing the term
banana	2	(10, 3), (5, 2)
machine	4	(1, 5), (10, 1), (38, 9), (42, 5)
learning	3	(1, 5), (38, 7), (42, 5)
...	...	...

[Okapi BM25](#), the 25th generation of the Best Matching algorithm, was developed by Robertson et al. in the 1980s. Its scorer is a modification of TF-IDF. Compared to naive TF-IDF, BM25 normalizes term frequency

scores by document length. Longer documents are more likely to contain a given term and have higher term frequency values.⁶

BM25 and its variances (BM25+, BM25F) are still widely used in the industry and serve as formidable baselines to compare against modern, more sophisticated retrieval algorithms, such as embedding-based retrieval, discussed next.⁷

One process I glossed over is tokenization, the process of breaking a query into individual terms. The simplest method is to split the query into words, treating each word as a separate term. However, this can lead to multi-word terms being broken into individual words, losing their original meaning. For example, “hot dog” would be split into “hot” and “dog”. When this happens, neither retains the meaning of the original term. One way to mitigate this issue is to treat the most common n-grams as terms. If the bigram “hot dog” is common, it’ll be treated as a term.

Additionally, you might want to convert all characters to lowercase, remove punctuation, and eliminate stop words (like “the”, “and”, “is”, etc.). Term-based retrieval solutions often handle these automatically. Classical NLP packages, such as [NLTK](#) (Natural Language Toolkit), [spaCy](#), and [Stanford’s CoreNLP](#), also offer tokenization functionalities.

[Chapter 4](#) discusses measuring the lexical similarity between two texts based on their n-gram overlap. Can we retrieve documents based on the

extent of their n-gram overlap with the query? Yes, we can. This approach works best when the query and the documents are of similar lengths. If the documents are much longer than the query, the likelihood of them containing the query's n-grams increases, leading to many documents having similarly high overlap scores. This makes it difficult to distinguish truly relevant documents from less relevant ones.

Embedding-based retrieval

Term-based retrieval computes relevance at a lexical level rather than a semantic level. As mentioned in [Chapter 3](#), the appearance of a text doesn't necessarily capture its meaning. This can result in returning documents irrelevant to your intent. For example, querying “transformer architecture” might return documents about the electric device or the movie *Transformers*. On the other hand, *embedding-based retrievers* aim to rank documents based on how closely their meanings align with the query. This approach is also known as *semantic retrieval*.

With embedding-based retrieval, indexing has an extra function: converting the original data chunks into embeddings. The database where the generated embeddings are stored is called a *vector database*. Querying then consists of two steps, as shown in [Figure 6-3](#):

1. Embedding model: convert the query into an embedding using the same embedding model used during indexing.

2. Retriever: fetch k data chunks whose embeddings are closest to the query embedding, as determined by the retriever. The number of data chunks to fetch, k , depends on the use case, the generative model, and the query.

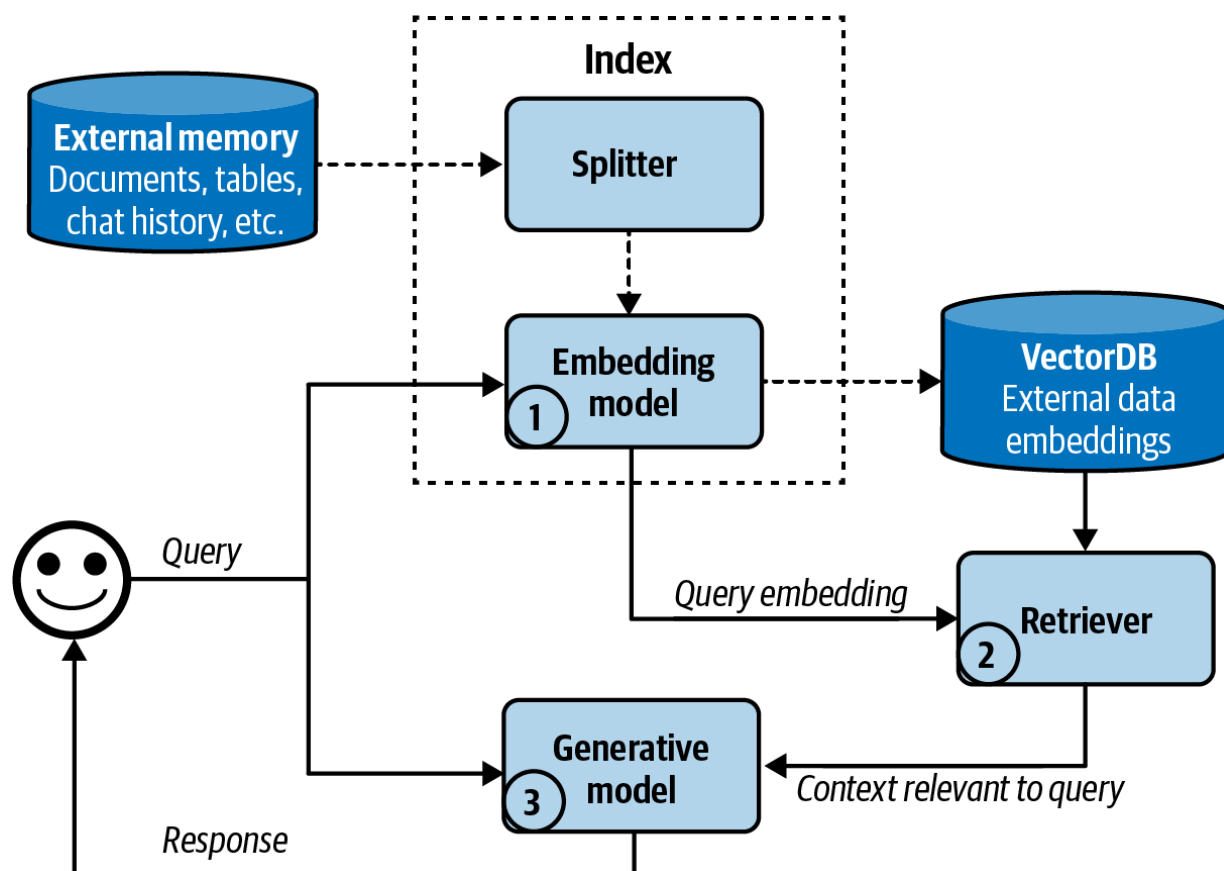


Figure 6-3. A high-level view of how an embedding-based, or semantic, retriever works.

The embedding-based retrieval workflow shown here is simplified. Real-world semantic retrieval systems might contain other components, such as a reranker to rerank all retrieved candidates, and caches to reduce latency.⁸

With embedding-based retrieval, we again encounter embeddings, which are discussed in [Chapter 3](#). As a reminder, an embedding is typically a

vector that aims to preserve the important properties of the original data. An embedding-based retriever doesn't work if the embedding model is bad.

Embedding-based retrieval also introduces a new component: vector databases. A vector database stores vectors. However, storing is the easy part of a vector database. The hard part is vector search. Given a query embedding, a vector database is responsible for finding vectors in the database close to the query and returning them. Vectors have to be indexed and stored in a way that makes vector search fast and efficient.

Like many other mechanisms that generative AI applications depend on, vector search isn't unique to generative AI. Vector search is common in any application that uses embeddings: search, recommendation, data organization, information retrieval, clustering, fraud detection, and more.

Vector search is typically framed as a nearest-neighbor search problem. For example, given a query, find the k nearest vectors. The naive solution is k -nearest neighbors (k -NN), which works as follows:

1. Compute the similarity scores between the query embedding and all vectors in the database, using metrics such as cosine similarity.
2. Rank all vectors by their similarity scores.
3. Return k vectors with the highest similarity scores.

This naive solution ensures that the results are precise, but it's computationally heavy and slow. It should be used only for small datasets.

For large datasets, vector search is typically done using an approximate nearest neighbor (ANN) algorithm. Due to the importance of vector search, many algorithms and libraries have been developed for it. Some popular vector search libraries are *FAISS* (Facebook AI Similarity Search) ([Johnson et al., 2017](#)), Google's *ScaNN* (Scalable Nearest Neighbors) ([Sun et al., 2020](#)), *Spotify's Annoy* (Bernhardsson, 2013), and *Hnswlib* ([Hierarchical Navigable Small World](#)) (Malkov and Yashunin, 2016).

Most application developers won't implement vector search themselves, so I'll give only a quick overview of different approaches. This overview might be helpful as you evaluate solutions.

In general, vector databases organize vectors into buckets, trees, or graphs. Vector search algorithms differ based on the heuristics they use to increase the likelihood that similar vectors are close to each other. Vectors can also be quantized (reduced precision) or made sparse. The idea is that quantized and sparse vectors are less computationally intensive to work with. For those wanting to learn more about vector search, Zilliz has an excellent [series](#) on it. Here are some significant vector search algorithms:

LSH (locality-sensitive hashing) ([Indyk and Motwani, 1999](#))

This is a powerful and versatile algorithm that works with more than just vectors. This involves hashing similar vectors into the same buckets to speed up similarity search, trading some accuracy for efficiency. It's implemented in *FAISS* and *Annoy*.

HNSW (Hierarchical Navigable Small World) ([Malkov and Yashunin, 2016](#))

HNSW constructs a multi-layer graph where nodes represent vectors, and edges connect similar vectors, allowing nearest-neighbor searches by traversing graph edges. Its implementation by the authors is open source, and it's also implemented in FAISS and Milvus.

Product Quantization ([Jégou et al., 2011](#))

This works by reducing each vector into a much simpler, lower-dimensional representation by decomposing each vector into multiple subvectors. The distances are then computed using the lower-dimensional representations, which are much faster to work with. Product quantization is a key component of FAISS and is supported by almost all popular vector search libraries.

IVF (inverted file index) ([Sivic and Zisserman, 2003](#))

IVF uses K-means clustering to organize similar vectors into the same cluster. Depending on the number of vectors in the database, it's typical to set the number of clusters so that, on average, there are 100 to 10,000 vectors in each cluster. During querying, IVF finds the cluster centroids closest to the query embedding, and the vectors in these clusters become candidate neighbors. Together with product quantization, IVF forms the backbone of FAISS.

Annoy (Approximate Nearest Neighbors Oh Yeah) ([Bernhardsson, 2013](#))

Annoy is a tree-based approach. It builds multiple binary trees, where each tree splits the vectors into clusters using random criteria, such as randomly drawing a line and splitting the vectors into two branches using this line. During a search, it traverses these trees to gather candidate neighbors. Spotify has open sourced its implementation.

There are other algorithms, such as [Microsoft's SPTAG](#) (Space Partition Tree And Graph), and [FLANN](#) (Fast Library for Approximate Nearest Neighbors).

Even though vector databases emerged as their own category with the rise of RAG, any database that can store vectors can be called a vector database. Many traditional databases have extended or will extend to support vector storage and vector search.

Comparing retrieval algorithms

Due to the long history of retrieval, its many mature solutions make both term-based and embedding-based retrieval relatively easy to start. Each approach has its pros and cons.

Term-based retrieval is generally much faster than embedding-based retrieval during both indexing and query. Term extraction is faster than

embedding generation, and mapping from a term to the documents that contain it can be less computationally expensive than a nearest-neighbor search.

Term-based retrieval also works well out of the box. Solutions like Elasticsearch and BM25 have successfully powered many search and retrieval applications. However, its simplicity also means that it has fewer components you can tweak to improve its performance.

Embedding-based retrieval, on the other hand, can be significantly improved over time to outperform term-based retrieval. You can finetune the embedding model and the retriever, either separately, together, or in conjunction with the generative model. However, converting data into embeddings can obscure keywords, such as specific error codes, e.g., EADDRNOTAVAIL (99), or product names, making them harder to search later on. This limitation can be addressed by combining embedding-based retrieval with term-based retrieval, as discussed later in this chapter.

The quality of a retriever can be evaluated based on the quality of the data it retrieves. Two metrics often used by RAG evaluation frameworks are *context precision* and *context recall*, or precision and recall for short (context precision is also called *context relevance*):

Context precision

Out of all the documents retrieved, what percentage is relevant to the query?

Context recall

Out of all the documents that are relevant to the query, what percentage is retrieved?

To compute these metrics, you curate an evaluation set with a list of test queries and a set of documents. For each test query, you annotate each test document to be relevant or not relevant. The annotation can be done either by humans or AI judges. You then compute the precision and recall score of the retriever on this evaluation set.

In production, some RAG frameworks only support context precision, not context recall. To compute context recall for a given query, you need to annotate the relevance of all documents in your database to that query. Context precision is simpler to compute. You only need to compare the retrieved documents to the query, which can be done by an AI judge.

If you care about the ranking of the retrieved documents, for example, more relevant documents should be ranked first, you can use metrics such as NDCG (normalized discounted cumulative gain), MAP (Mean Average Precision), and MRR (Mean Reciprocal Rank).

For semantic retrieval, you need to also evaluate the quality of your embeddings. As discussed in [Chapter 3](#), embeddings can be evaluated independently—they are considered good if more-similar documents have closer embeddings. Embeddings can also be evaluated by how well they work for specific tasks. The [MTEB](#) benchmark (Muennighoff et al., 2023) evaluates embeddings for a broad range of tasks including retrievals, classification, and clustering.

The quality of a retriever should also be evaluated in the context of the whole RAG system. Ultimately, a retriever is good if it helps the system generate high-quality answers. Evaluating outputs of generative models is discussed in [Chapters 3](#) and [4](#).

Whether the performance promise of a semantic retrieval system is worth pursuing depends on how much you prioritize cost and latency, particularly during the querying phase. Since much of RAG latency comes from output generation, especially for long outputs, *the added latency by query embedding generation and vector search might be minimal compared to the total RAG latency*. Even so, the added latency still can impact user experience.

Another concern is cost. Generating embeddings costs money. This is especially an issue if your data changes frequently and requires frequent embedding regeneration. Imagine having to generate embeddings for 100 million documents every day! Depending on what vector databases you use,

vector storage and vector search queries can be expensive, too. It’s not uncommon to see a company’s vector database spending be one-fifth or even half of their spending on model APIs.

[Table 6-2](#) shows a side-by-side comparison of term-based retrieval and embedding-based retrieval.

Table 6-2. Term-based retrieval and semantic retrieval by speed, performance, and cost.

	Term-based retrieval	Embedding-based retrieval
Querying speed	Much faster than embedding-based retrieval	Query embedding generation and vector search can be slow
Performance	Typically strong performance out of the box, but hard to improve Can retrieve wrong documents due to term ambiguity	Can outperform term-based retrieval with finetuning Allows for the use of more natural queries, as it focuses on semantics instead of terms
Cost	Much cheaper than embedding-based retrieval	Embedding, vector storage, and vector search solutions can be expensive

With retrieval systems, you can make certain trade-offs between indexing and querying. The more detailed the index is, the more accurate the retrieval process will be, but the indexing process will be slower and more memory-consuming. Imagine building an index of potential customers. Adding more details (e.g., name, company, email, phone, interests) makes it easier to find relevant people but takes longer to build and requires more storage.

In general, a detailed index like HNSW provides high accuracy and fast query times but requires significant time and memory to build. In contrast, a simpler index like LSH is quicker and less memory-intensive to create, but it results in slower and less accurate queries.

The [ANN-Benchmarks website](#) compares different ANN algorithms on multiple datasets using four main metrics, taking into account the trade-offs between indexing and querying. These include the following:

Recall

The fraction of the nearest neighbors found by the algorithm.

Query per second (QPS)

The number of queries the algorithm can handle per second. This is crucial for high-traffic applications.

Build time

The time required to build the index. This metric is especially important if you need to frequently update your index (e.g., because your data changes).

Index size

The size of the index created by the algorithm, which is crucial for assessing its scalability and storage requirements.

Additionally, BEIR (Benchmarking IR) ([Thakur et al., 2021](#)) is an evaluation harness for retrieval. It supports retrieval systems across 14 common retrieval benchmarks.

To summarize, the quality of a RAG system should be evaluated both component by component and end to end. To do this, you should do the following things:

1. Evaluate the retrieval quality.
2. Evaluate the final RAG outputs.
3. Evaluate the embeddings (for embedding-based retrieval).

Combining retrieval algorithms

Given the distinct advantages of different retrieval algorithms, a production retrieval system typically combines several approaches. Combining term-based retrieval and embedding-based retrieval is called *hybrid search*.

Different algorithms can be used in sequence. First, a cheap, less precise retriever, such as a term-based system, fetches candidates. Then, a more precise but more expensive mechanism, such as k-nearest neighbors, finds the best of these candidates. This second step is also called *reranking*.

For example, given the term “transformer”, you can fetch all documents that contain the word transformer, regardless of whether they are about the electric device, the neural architecture, or the movie. Then you use vector search to find among these documents those that are actually related to your transformer query. As another example, consider the query “Who’s responsible for the most sales to X?” First, you might fetch all documents associated with X using the keyword X. Then, you use vector search to retrieve the context associated with “Who’s responsible for the most sales?”

Different algorithms can also be used in parallel as an ensemble. Remember that a retriever works by ranking documents by their relevance scores to the query. You can use multiple retrievers to fetch candidates at the same time, then combine these different rankings together to generate a final ranking.

An algorithm for combining different rankings is called reciprocal rank fusion (RRF) (Cormack et al., 2009). It assigns each document a score based on its ranking by a retriever. Intuitively, if it ranks first, its score is $1/1 = 1$. If it ranks second, its score is $1/2 = 0.5$. The higher it ranks, the higher its score.

A document's final score is the sum of its scores with respect to all retrievers. If a document is ranked first by one retriever and second by another retriever, its score is $1 + 0.5 = 1.5$. This example is an oversimplification of RRF, but it shows the basics. The actual formula for a document D is more complicated, as follows:

$$\text{Score}(D) = \sum_{i=1}^n \frac{1}{k+r_i(D)}$$

- n is the number of ranked lists; each rank list is produced by a retriever.
- $r_i(D)$ is the rank of the document by the retriever i .
- k is a constant to avoid division by zero and to control the influence of lower-ranked documents. A typical value for k is 60.

Retrieval Optimization

Depending on the task, certain tactics can increase the chance of relevant documents being fetched. Four tactics discussed here are chunking strategy, reranking, query rewriting, and contextual retrieval.

Chunking strategy

How your data should be indexed depends on how you intend to retrieve it later. The last section covered different retrieval algorithms and their respective indexing strategies. There, the discussion was based on the assumption that documents have already been split into manageable chunks. In this section, I'll cover different chunking strategies. This is an important

consideration because the chunking strategy you use can significantly impact the performance of your retrieval system.

The simplest strategy is to chunk documents into chunks of equal length based on a certain unit. Common units are characters, words, sentences, and paragraphs. For example, you can split each document into chunks of 2,048 characters or 512 words. You can also split each document so that each chunk can contain a fixed number of sentences (such as 20 sentences) or paragraphs (such as each paragraph is its own chunk).

You can also split documents recursively using increasingly smaller units until each chunk fits within your maximum chunk size. For example, you can start by splitting a document into sections. If a section is too long, split it into paragraphs. If a paragraph is still too long, split it into sentences. This reduces the chance of related texts being arbitrarily broken off.

Specific documents might also support creative chunking strategies. For example, there are [splitters](#) developed especially for different programming languages. Q&A documents can be split by question or answer pair, where each pair makes up a chunk. Chinese texts might need to be split differently from English texts.

When a document is split into chunks without overlap, the chunks might be cut off in the middle of important context, leading to the loss of critical information. Consider the text “I left my wife a note”. If it’s split into “I left

my wife” and “a note”, neither of these two chunks conveys the key information of the original text. Overlapping ensures that important boundary information is included in at least one chunk. If you set the chunk size to be 2,048 characters, you can perhaps set the overlapping size to be 20 characters.

The chunk size shouldn't exceed the maximum context length of the generative model. For the embedding-based approach, the chunk size also shouldn't exceed the embedding model's context limit.

You can also chunk documents using tokens, determined by the generative model's tokenizer, as a unit. Let's say that you want to use Llama 3 as your generative model. You then first tokenize documents using Llama 3's tokenizer. You can then split documents into chunks using tokens as the boundaries. Chunking by tokens makes it easier to work with downstream models. However, the downside of this approach is that if you switch to another generative model with a different tokenizer, you'd need to reindex your data.

Regardless of which strategy you choose, chunk sizes matter. A smaller chunk size allows for more diverse information. Smaller chunks mean that you can fit more chunks into the model's context. If you halve the chunk size, you can fit twice as many chunks. More chunks can provide a model with a wider range of information, which can enable the model to produce a better answer.

Small chunk sizes, however, can cause the loss of important information. Imagine a document that contains important information about the topic X throughout the document, but X is only mentioned in the first half. If you split this document into two chunks, the second half of the document might not be retrieved, and the model won't be able to use its information.

Smaller chunk sizes can also increase computational overhead. This is especially an issue for embedding-based retrieval. Halving the chunk size means that you have twice as many chunks to index and twice as many embedding vectors to generate and store. Your vector search space will be twice as big, which can reduce the query speed.

There is no universal best chunk size or overlap size. You have to experiment to find what works best for you.

Reranking

The initial document rankings generated by the retriever can be further reranked to be more accurate. Reranking is especially useful when you need to reduce the number of retrieved documents, either to fit them into your model's context or to reduce the number of input tokens.

One common pattern for reranking is discussed in [“Combining retrieval algorithms”](#). A cheap but less precise retriever fetches candidates, then a more precise but more expensive mechanism reranks these candidates.

Documents can also be reranked based on time, giving higher weight to more recent data. This is useful for time-sensitive applications such as news aggregation, chat with your emails (e.g., a chatbot that can answer questions about your emails), or stock market analysis.

Context reranking differs from traditional search reranking in that the exact position of items is less critical. In search, the rank (e.g., first or fifth) is crucial. In context reranking, the order of documents still matters because it affects how well a model can process them. Models might better understand documents at the beginning and end of the context, as discussed in [“Context Length and Context Efficiency”](#). However, as long as a document is included, the impact of its order is less significant compared to search ranking.

Query rewriting

Query rewriting is also known as query reformulation, query normalization, and sometimes query expansion. Consider the following conversation:

User: When was the last time John Doe bought something from us?

AI: John last bought a Fruity Fedora hat from us two weeks ago, on January 3, 2030.

User: How about Emily Doe?

The last question, “How about Emily Doe?”, is ambiguous without context. If you use this query verbatim to retrieve documents, you’ll likely get irrelevant results. You need to rewrite this query to reflect what the user is actually asking. The new query should make sense on its own. In this case, the query should be rewritten to “When was the last time Emily Doe bought something from us?”

While I put query rewriting in “RAG”, query rewriting isn’t unique to RAG. In traditional search engines, query rewriting is often done using heuristics. In AI applications, query rewriting can also be done using other AI models, using a prompt similar to “Given the following conversation, rewrite the last user input to reflect what the user is actually asking”.

Figure 6-4 shows how ChatGPT rewrote the query using this prompt.

Given the following conversation, rewrite the last user input to reflect what the user is actually asking.

User: When was the last time John Doe bought something from us?

AI: John last bought a Fruity Fedora hat from us two weeks ago, on January 3, 2030.

User: How about Emily Doe?

When was the last time Emily Doe bought something from us?

Figure 6-4. You can use other generative models to rewrite queries.

Query rewriting can get complicated, especially if you need to do identity resolution or incorporate other knowledge. For example, if the user asks “How about his wife?” you will first need to query your database to find out

who his wife is. If you don't have this information, the rewriting model should acknowledge that this query isn't solvable instead of hallucinating a name, leading to a wrong answer.

Contextual retrieval

The idea behind contextual retrieval is to augment each chunk with relevant context to make it easier to retrieve the relevant chunks. A simple technique is to augment a chunk with metadata like tags and keywords. For ecommerce, a product can be augmented by its description and reviews. Images and videos can be queried by their titles or captions.

The metadata may also include entities automatically extracted from the chunk. If your document contains specific terms like the error code EADDRNOTAVAIL (99), adding them to the document's metadata allows the system to retrieve it by that keyword, even after the document has been converted into embeddings.

You can also augment each chunk with the questions it can answer. For customer support, you can augment each article with related questions. For example, the article on how to reset your password can be augmented with queries like "How to reset password?", "I forgot my password", "I can't log in", or even "Help, I can't find my account".⁹

If a document is split into multiple chunks, some chunks might lack the necessary context to help the retriever understand what the chunk is about.

To avoid this, you can augment each chunk with the context from the original document, such as the original document's title and summary. Anthropic used AI models to generate a short context, usually 50-100 tokens, that explains the chunk and its relationship to the original document. Here's the prompt Anthropic used for this purpose ([Anthropic, 2024](#)):

```
<document>
```

```
{{WHOLE_DOCUMENT}}
```

```
</document>
```

Here is the chunk we want to situate within the whole document:

```
<chunk>
```

```
{{CHUNK_CONTENT}}
```

```
</chunk>
```

Please give a short succinct context to situate this chunk within the overall document for the purposes of improving search retrieval of the chunk. Answer only with the succinct context and nothing else.

The generated context for each chunk is prepended to each chunk, and the augmented chunk is then indexed by the retrieval algorithm. [Figure 6-5](#) visualizes the process that Anthropic follows.

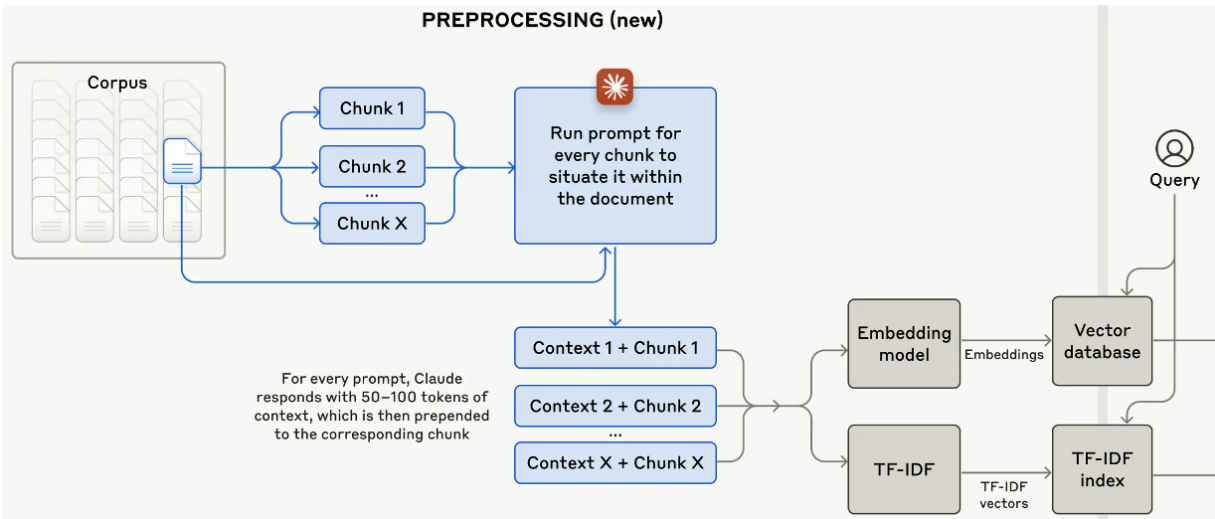


Figure 6-5. Anthropic augments each chunk with a short context that situates this chunk within the original document, making it easier for the retriever to find the relevant chunks given a query. Image from “Introducing Contextual Retrieval” (Anthropic, 2024).

EVALUATING RETRIEVAL SOLUTIONS

Here are some key factors to keep in mind when evaluating a retrieval solution:

- What retrieval mechanisms does it support? Does it support hybrid search?
- If it's a vector database, what embedding models and vector search algorithms does it support?
- How scalable is it, both in terms of data storage and query traffic? Does it work for your traffic patterns?
- How long does it take to index your data? How much data can you process (such as add/delete) in bulk at once?
- What's its query latency for different retrieval algorithms?
- If it's a managed solution, what's its pricing structure? Is it based on the document/vector volume or on the query volume?

This list doesn't include the functionalities typically associated with enterprise solutions such as access control, compliance, data plane and control plane separation, etc.

RAG Beyond Texts

The last section discussed text-based RAG systems where the external data sources are text documents. However, external data sources can also be

multimodal and tabular data.

Multimodal RAG

If your generator is multimodal, its contexts might be augmented not only with text documents but also with images, videos, audio, etc., from external sources. I'll use images in the examples to keep the writing concise, but you can replace images with any other modality. Given a query, the retriever fetches both texts and images relevant to it. For example, given “What’s the color of the house in the Pixar movie Up?” the retriever can fetch a picture of the house in *Up* to help the model answer, as shown in [Figure 6-6](#).

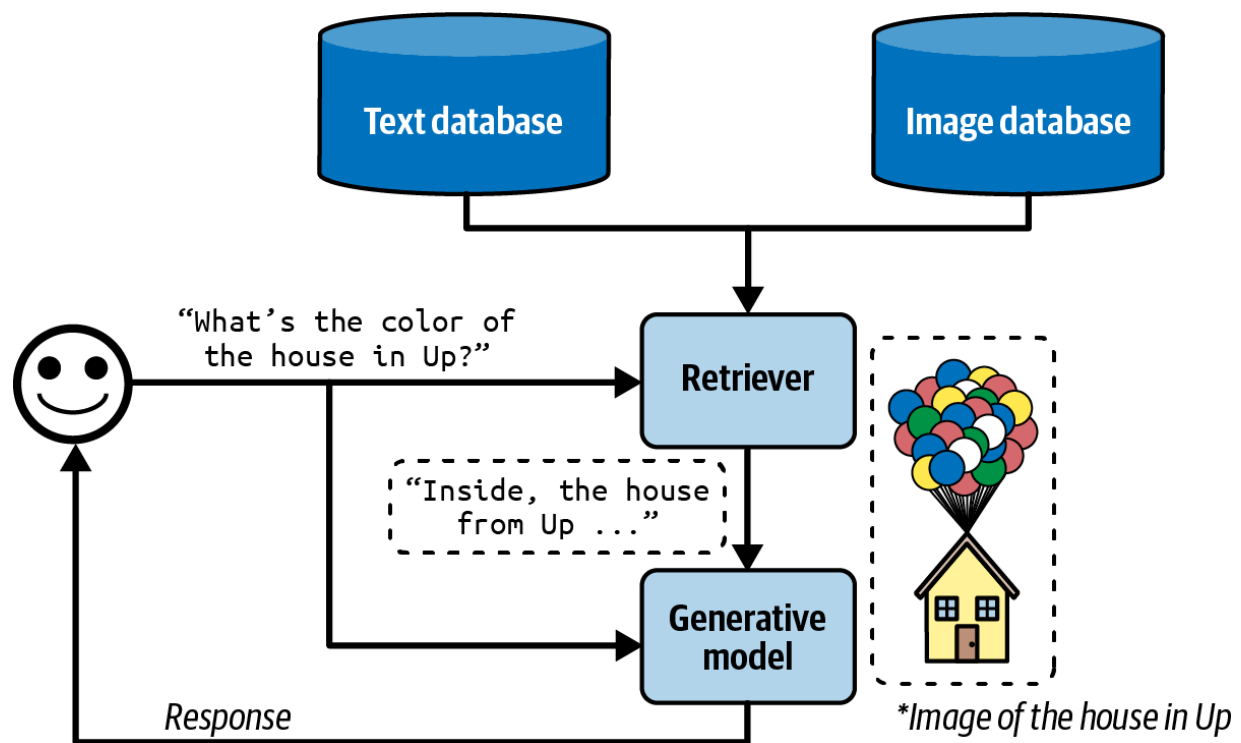


Figure 6-6. Multimodal RAG can augment a query with both text and images. (*The real image from *Up* is not used, for copyright reasons.)

If the images have metadata—such as titles, tags, and captions—they can be retrieved using the metadata. For example, an image is retrieved if its caption is considered relevant to the query.

If you want to retrieve images based on their content, you'll need to have a way to compare images to queries. If queries are texts, you'll need a multimodal embedding model that can generate embeddings for both images and texts. Let's say you use CLIP ([Radford et al., 2021](#)) as the multimodal embedding model. The retriever works as follows:

1. Generate CLIP embeddings for all your data, both texts and images, and store them in a vector database.
2. Given a query, generate its CLIP embedding.
3. Query in the vector database for all images and texts whose embeddings are close to the query embedding.

RAG with tabular data

Most applications work not only with unstructured data like texts and images but also with tabular data. Many queries might need information from data tables to answer. The workflow for augmenting a context using tabular data is significantly different from the classic RAG workflow.

Imagine you work for an ecommerce site called Kitty Vogue that specializes in cat fashion. This store has an order table named Sales, as shown in

[Table 6-3](#).

Table 6-3. An example of an order table, Sales, for the imaginary ecommerce site Kitty Vogue.

Order ID	Timestamp	Product ID	Product	Unit
1	...	2044	Meow Mix Seasoning	10.99
2	...	3492	Purr & Shake	25
3	...	2045	Fruity Fedora	18
...	...	...	...	...

To generate a response to the question “How many units of Fruity Fedora were sold in the last 7 days?”, your system needs to query this table for all orders involving Fruity Fedora and sum the number of units across all orders. Assume that this table can be queried using SQL. The SQL query might look like this:

```
SELECT SUM(units) AS total_units_sold
FROM Sales
WHERE product_name = 'Fruity Fedora'
AND timestamp >= DATE_SUB(CURDATE(), INTERVAL 7 DAY)
```

The workflow is as follows, visualized in [Figure 6-7](#). To run this workflow, your system must have the ability to generate and execute the SQL query:

1. Text-to-SQL: based on the user query and the provided table schemas, determine what SQL query is needed. Text-to-SQL is an example of semantic parsing, as discussed in [Chapter 2](#).
2. SQL execution: execute the SQL query.
3. Generation: generate a response based on the SQL result and the original user query.

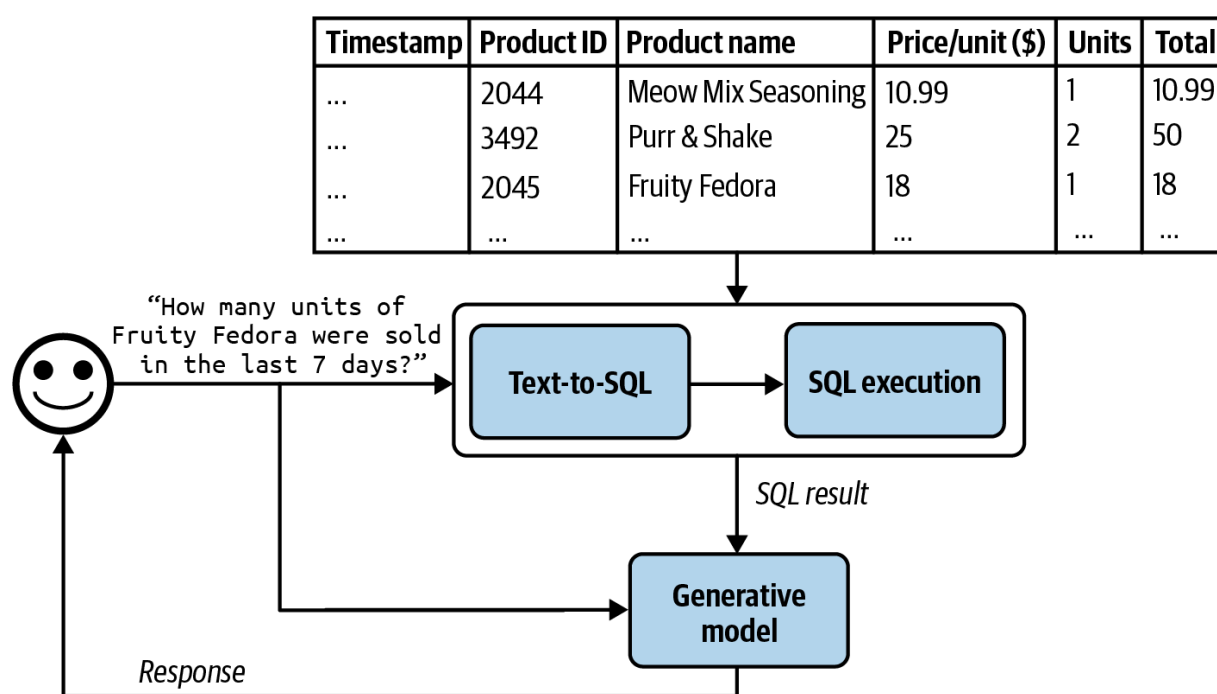


Figure 6-7. A RAG system that augments context with tabular data.

For the text-to-SQL step, if there are many available tables whose schemas can't all fit into the model context, you might need an intermediate step to predict what tables to use for each query. Text-to-SQL can be done by the same generator that generates the final response or a specialized text-to-SQL model.

In this section, we’ve discussed how tools such as retrievers and SQL executors can enable models to handle more queries and generate higher-quality responses. Would giving a model access to more tools improve its capabilities even more? Tool use is a core characteristic of the agentic pattern, which we’ll discuss in the next section.

Agents

Intelligent agents are considered by many to be the ultimate goal of AI. The classic book by Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach* (Prentice Hall, 1995) defines the field of *artificial intelligence research* as “the study and design of rational agents.”

The unprecedented capabilities of foundation models have opened the door to agentic applications that were previously unimaginable. These new capabilities make it finally possible to develop autonomous, intelligent agents to act as our assistants, coworkers, and coaches. They can help us create a website, gather data, plan a trip, do market research, manage a customer account, automate data entry, prepare us for interviews, interview our candidates, negotiate a deal, etc. The possibilities seem endless, and the potential economic value of these agents is enormous.

WARNING

AI-powered agents are an emerging field, with no established theoretical frameworks for defining, developing, and evaluating them. This section is a best-effort attempt to build a framework from the existing literature, but it will evolve as the field does. Compared to the rest of the book, this section is more experimental.

This section will start with an overview of agents, and then continue with two aspects that determine the capabilities of an agent: tools and planning. Agents, with their new modes of operations, have new modes of failures. This section will end with a discussion on how to evaluate agents to catch these failures.

Even though agents are novel, they are built upon concepts that have already appeared in this book, including self-critique, chain-of-thought, and structured outputs.

Agent Overview

The term *agent* has been used in many different engineering contexts, including but not limited to a software agent, intelligent agent, user agent, conversational agent, and reinforcement learning agent. So, what exactly is an agent?

An agent is anything that can perceive its environment and act upon that environment.¹⁰ This means that an agent is characterized by the

environment it operates in and *the set of actions* it can perform.

The *environment* an agent can operate in is defined by its use case. If an agent is developed to play a game (e.g., *Minecraft*, *Go*, *Dota*), that game is its environment. If you want an agent to scrape documents from the internet, the environment is the internet. If your agent is a cooking robot, the kitchen is its environment. A self-driving car agent's environment is the road system and its adjacent areas.

The *set of actions* an AI agent can perform is augmented by the *tools* it has access to. Many generative AI-powered applications you interact with daily are agents with access to tools, albeit simple ones. ChatGPT is an agent. It can search the web, execute Python code, and generate images. RAG systems are agents, and text retrievers, image retrievers, and SQL executors are their tools.

There's a strong dependency between an agent's environment and its set of tools. The environment determines what tools an agent can potentially use. For example, if the environment is a chess game, the only possible actions for an agent are the valid chess moves. However, an agent's tool inventory restricts the environment it can operate in. For example, if a robot's only action is swimming, it'll be confined to a water environment.

[Figure 6-8](#) shows a visualization of SWE-agent ([Yang et al., 2024](#)), an agent built on top of GPT-4. Its environment is the computer with the terminal

and the file system. Its set of actions include navigate repo, search files, view files, and edit lines.

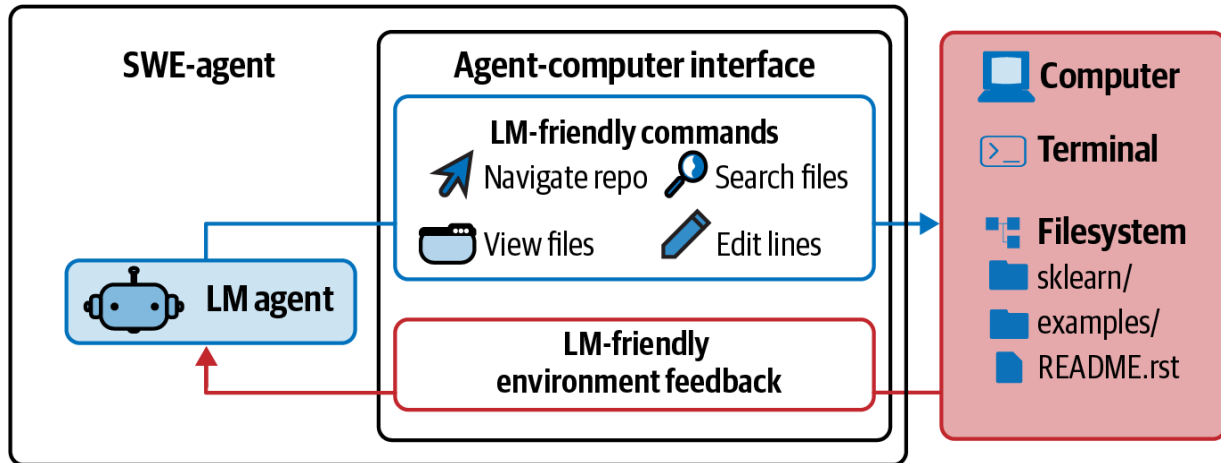


Figure 6-8. SWE-agent (Yang et al., 2024) is a coding agent whose environment is the computer and whose actions include navigation, search, and editing. Adapted from an original image licensed under CC BY 4.0.

An AI agent is meant to accomplish tasks typically provided by the users in the inputs. In an AI agent, AI is the brain that processes the information it receives, including the task and feedback from the environment, plans a sequence of actions to achieve this task, and determines whether the task has been accomplished.

Let's get back to the RAG system with tabular data in the Kitty Vogue example. This is a simple agent with three actions: response generation, SQL query generation, and SQL query execution. Given the query "Project the sales revenue for Fruity Fedora over the next three months", the agent might perform the following sequence of actions:

1. Reason about how to accomplish this task. It might decide that to predict future sales, it first needs the sales numbers from the last five years. Note that the agent's reasoning is shown as its intermediate response.
2. Invoke SQL query generation to generate the query to get sales numbers from the last five years.
3. Invoke SQL query execution to execute this query.
4. Reason about the tool outputs and how they help with sales prediction. It might decide that these numbers are insufficient to make a reliable projection, perhaps because of missing values. It then decides that it also needs information about past marketing campaigns.
5. Invoke SQL query generation to generate the queries for past marketing campaigns.
6. Invoke SQL query execution.
7. Reason that this new information is sufficient to help predict future sales. It then generates a projection.
8. Reason that the task has been successfully completed.

Compared to non-agent use cases, agents typically require more powerful models for two reasons:

- Compound mistakes: an agent often needs to perform multiple steps to accomplish a task, and the overall accuracy decreases as the number of steps increases. If the model's accuracy is 95% per step, over 10 steps, the accuracy will drop to 60%, and over 100 steps, the accuracy will be only 0.6%.

- Higher stakes: with access to tools, agents are capable of performing more impactful tasks, but any failure could have more severe consequences.

A task that requires many steps can take time and money to run.¹¹ However, if agents can be autonomous, they can save a lot of human time, making their costs worthwhile.

Given an environment, the success of an agent in an environment depends on the tool inventory it has access to and the strength of its AI planner. Let's start by looking into different kinds of tools a model can use.

Tools

A system doesn't need access to external tools to be an agent. However, without external tools, the agent's capabilities would be limited. By itself, a model can typically perform one action—for example, an LLM can generate text, and an image generator can generate images. External tools make an agent vastly more capable.

Tools help an agent to both perceive the environment and act upon it.

Actions that allow an agent to perceive the environment are *read-only actions*, whereas actions that allow an agent to act upon the environment are *write actions*.

This section gives an overview of external tools. How tools can be used will be discussed in [“Planning”](#).

The set of tools an agent has access to is its tool inventory. Since an agent’s tool inventory determines what an agent can do, it’s important to think through what and how many tools to give an agent. More tools give an agent more capabilities. However, the more tools there are, the more challenging it is to understand and utilize them well. Experimentation is necessary to find the right set of tools, as discussed in [“Tool selection”](#).

Depending on the agent’s environment, there are many possible tools. Here are three categories of tools that you might want to consider: knowledge augmentation (i.e., context construction), capability extension, and tools that let your agent act upon its environment.

Knowledge augmentation

I hope that this book, so far, has convinced you of the importance of having the relevant context for a model’s response quality. An important category of tools includes those that help augment your agent’s knowledge of your agent. Some of them have already been discussed: text retriever, image retriever, and SQL executor. Other potential tools include internal people search, an inventory API that returns the status of different products, Slack retrieval, an email reader, etc.

Many such tools augment a model with your organization's private processes and information. However, tools can also give models access to public information, especially from the internet.

Web browsing was among the earliest and most anticipated capabilities to be incorporated into chatbots like ChatGPT. Web browsing prevents a model from going stale. A model goes stale when the data it was trained on becomes outdated. If the model's training data was cut off last week, it won't be able to answer questions that require information from this week unless this information is provided in the context. Without web browsing, a model won't be able to tell you about the weather, news, upcoming events, stock prices, flight status, etc.

I use web browsing as an umbrella term to cover all tools that access the internet, including web browsers and specific APIs such as search APIs, news APIs, GitHub APIs, or social media APIs such as those of X, LinkedIn, and Reddit.

While web browsing allows your agent to reference up-to-date information to generate better responses and reduce hallucinations, it can also open up your agent to the cesspools of the internet. Select your Internet APIs with care.

Capability extension

The second category of tools to consider are those that address the inherent limitations of AI models. They are easy ways to give your model a performance boost. For example, AI models are notorious for being bad at math. If you ask a model what is 199,999 divided by 292, the model will likely fail. However, this calculation is trivial if the model has access to a calculator. Instead of trying to train the model to be good at arithmetic, it's a lot more resource-efficient to just give the model access to a tool.

Other simple tools that can significantly boost a model's capability include a calendar, timezone converter, unit converter (e.g., from lbs to kg), and translator that can translate to and from the languages that the model isn't good at.

More complex but powerful tools are code interpreters. Instead of training a model to understand code, you can give it access to a code interpreter so that it can execute a piece of code, return the results, or analyze the code's failures. This capability lets your agents act as coding assistants, data analysts, and even research assistants that can write code to run experiments and report results. However, automated code execution comes with the risk of code injection attacks, as discussed in ["Defensive Prompt Engineering"](#). Proper security measurements are crucial to keep you and your users safe.

External tools can make a text-only or image-only model multimodal. For example, a model that can generate only texts can leverage a text-to-image model as a tool, allowing it to generate both texts and images. Given a text request, the agent's AI planner decides whether to invoke text generation, image generation, or both. This is how ChatGPT can generate both text and images—it uses DALL-E as its image generator. Agents can also use a code interpreter to generate charts and graphs, a LaTeX compiler to render math equations, or a browser to render web pages from HTML code.

Similarly, a model that can process only text inputs can use an image captioning tool to process images and a transcription tool to process audio. It can use an OCR (optical character recognition) tool to read PDFs.

Tool use can significantly boost a model's performance compared to just prompting or even finetuning. Chameleon ([Lu et al., 2023](#)) shows that a GPT-4-powered agent, augmented with a set of 13 tools, can outperform GPT-4 alone on several benchmarks. Examples of tools this agent used are knowledge retrieval, a query generator, an image captioner, a text detector, and Bing search.

On ScienceQA, a science question answering benchmark, Chameleon improves the best published few-shot result by 11.37%. On TabMWP (Tabular Math Word Problems) (Lu et al., 2022), a benchmark involving tabular math questions, Chameleon improves the accuracy by 17%.

Write actions

So far, we've discussed read-only actions that allow a model to read from its data sources. But tools can also perform write actions, making changes to the data sources. A SQL executor can retrieve a data table (read) but can also change or delete the table (write). An email API can read an email but can also respond to it. A banking API can retrieve your current balance but can also initiate a bank transfer.

Write actions enable a system to do more. They can enable you to automate the whole customer outreach workflow: researching potential customers, finding their contacts, drafting emails, sending first emails, reading responses, following up, extracting orders, updating your databases with new orders, etc.

However, the prospect of giving AI the ability to automatically alter our lives is frightening. Just as you shouldn't give an intern the authority to delete your production database, you shouldn't allow an unreliable AI to initiate bank transfers. Trust in the system's capabilities and its security measures is crucial. You need to ensure that the system is protected from bad actors who might try to manipulate it into performing harmful actions.

When I talk about autonomous AI agents to a group of people, there is often someone who brings up self-driving cars. "What if someone hacks into the car to kidnap you?" While the self-driving car example seems visceral

because of its physicality, an AI system can cause harm without a presence in the physical world. It can manipulate the stock market, steal copyrights, violate privacy, reinforce biases, spread misinformation and propaganda, and more, as discussed in [“Defensive Prompt Engineering”](#).

These are all valid concerns, and any organization that wants to leverage AI needs to take safety and security seriously. However, this doesn’t mean that AI systems should never be given the ability to act in the real world. If we can get people to trust a machine to take us into space, I hope that one day, security measures will be sufficient for us to trust autonomous AI systems. Besides, humans can fail, too. Personally, I would trust a self-driving car more than the average stranger to drive me around.

Just as the right tools can help humans be vastly more productive—can you imagine doing business without Excel or building a skyscraper without cranes?—tools enable models to accomplish many more tasks. Many model providers already support tool use with their models, a feature often called function calling. Going forward, I would expect function calling with a wide set of tools to be common with most models.

Planning

At the heart of a foundation model agent is the model responsible for solving a task. A task is defined by its goal and constraints. For example, one task is to schedule a two-week trip from San Francisco to India with a

budget of \$5,000. The goal is the two-week trip. The constraint is the budget.

Complex tasks require planning. The output of the planning process is a plan, which is a roadmap outlining the steps needed to accomplish a task. Effective planning typically requires the model to understand the task, consider different options to achieve this task, and choose the most promising one.

If you've ever been in any planning meeting, you know that planning is hard. As an important computational problem, planning is well studied and would require several volumes to cover. I'll only be able to cover the surface here.

Planning overview

Given a task, there are many possible ways to decompose it, but not all of them will lead to a successful outcome. Among the correct solutions, some are more efficient than others. Consider the query, "How many companies without revenue have raised at least \$1 billion?" There are many possible ways to solve this, but as an illustration, consider the two options:

1. Find all companies without revenue, then filter them by the amount raised.
2. Find all companies that have raised at least \$1 billion, then filter them by revenue.

The second option is more efficient. There are vastly more companies without revenue than companies that have raised \$1 billion. Given only these two options, an intelligent agent should choose option 2.

You can couple planning with execution in the same prompt. For example, you give the model a prompt, ask it to think step by step (such as with a chain-of-thought prompt), and then execute those steps all in one prompt. But what if the model comes up with a 1,000-step plan that doesn't even accomplish the goal? Without oversight, an agent can run those steps for hours, wasting time and money on API calls, before you realize that it's not going anywhere.

To avoid fruitless execution, *planning* should be decoupled from *execution*. You ask the agent to first generate a plan, and only after this plan is *validated* is it executed. The plan can be validated using heuristics. For example, one simple heuristic is to eliminate plans with invalid actions. If the generated plan requires a Google search and the agent doesn't have access to Google Search, this plan is invalid. Another simple heuristic might be eliminating all plans with more than X steps. A plan can also be validated using AI judges. You can ask a model to evaluate whether the plan seems reasonable or how to improve it.

If the generated plan is evaluated to be bad, you can ask the planner to generate another plan. If the generated plan is good, execute it. If the plan consists of external tools, function calling will be invoked. Outputs from

executing this plan will then again need to be evaluated. Note that the generated plan doesn't have to be an end-to-end plan for the whole task. It can be a small plan for a subtask. The whole process looks like [Figure 6-9](#).

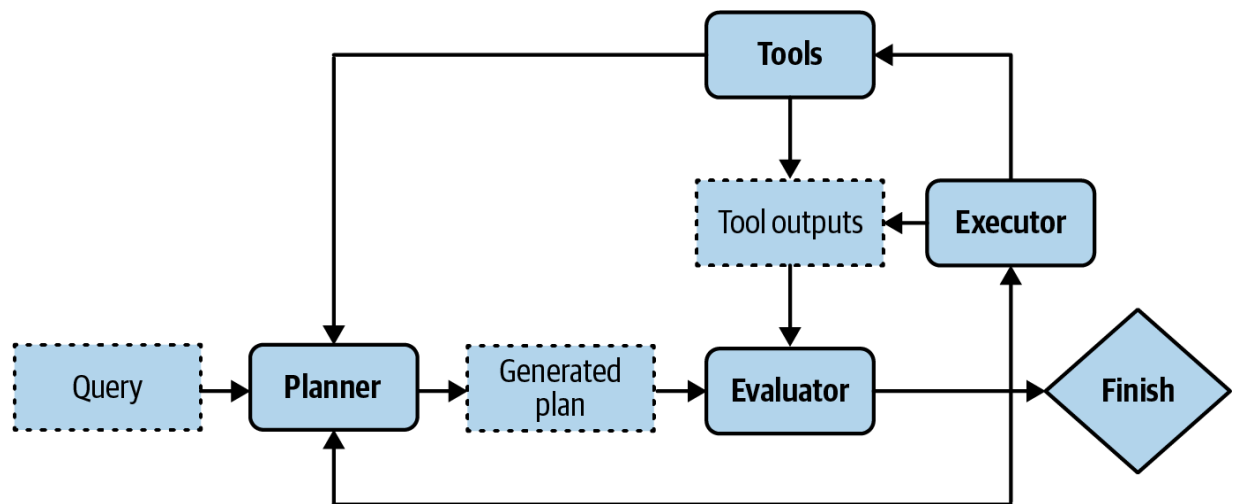


Figure 6-9. Decoupling planning and execution so that only validated plans are executed.

Your system now has three components: one to generate plans, one to validate plans, and another to execute plans. If you consider each component an agent, this is a multi-agent system.¹²

To speed up the process, instead of generating plans sequentially, you can generate several plans in parallel and ask the evaluator to pick the most promising one. This is another latency/cost trade-off, as generating multiple plans simultaneously will incur extra costs.

Planning requires understanding the intention behind a task: what's the user trying to do with this query? An intent classifier is often used to help agents plan. As shown in [“Break Complex Tasks into Simpler Subtasks”](#), intent

classification can be done using another prompt or a classification model trained for this task. The intent classification mechanism can be considered another agent in your multi-agent system.

Knowing the intent can help the agent pick the right tools. For example, for customer support, if the query is about billing, the agent might need access to a tool to retrieve a user's recent payments. But if the query is about how to reset a password, the agent might need to access documentation retrieval.

TIP

Some queries might be out of the scope of the agent. The intent classifier should be able to classify requests as IRRELEVANT so that the agent can politely reject those instead of wasting FLOPs coming up with impossible solutions.

So far, we've assumed that the agent automates all three stages: generating plans, validating plans, and executing plans. In reality, humans can be involved at any of those stages to aid with the process and mitigate risks. A human expert can provide a plan, validate a plan, or execute parts of a plan. For example, for complex tasks for which an agent has trouble generating the whole plan, a human expert can provide a high-level plan that the agent can expand upon. If a plan involves risky operations, such as updating a database or merging a code change, the system can ask for explicit human approval before executing or let humans execute these operations. To make

this possible, you need to clearly define the level of automation an agent can have for each action.

To summarize, solving a task typically involves the following processes. Note that reflection isn't mandatory for an agent, but it'll significantly boost the agent's performance:

1. *Plan generation*: come up with a plan for accomplishing this task. A plan is a sequence of manageable actions, so this process is also called task decomposition.
2. *Reflection and error correction*: evaluate the generated plan. If it's a bad plan, generate a new one.
3. *Execution*: take the actions outlined in the generated plan. This often involves calling specific functions.
4. *Reflection and error correction*: upon receiving the action outcomes, evaluate these outcomes and determine whether the goal has been accomplished. Identify and correct mistakes. If the goal is not completed, generate a new plan.

You've already seen some techniques for plan generation and reflection in this book. When you ask a model to "think step by step", you're asking it to decompose a task. When you ask a model to "verify if your answer is correct", you're asking it to reflect.

Foundation models as planners

An open question is how well foundation models can plan. Many researchers believe that foundation models, at least those built on top of autoregressive language models, cannot. Meta's Chief AI Scientist Yann LeCun states unequivocally that [autoregressive LLMs can't plan](#) (2023). In the article "Can LLMs Really Reason and Plan?" [Kambhampati \(2023\)](#) argues that LLMs are great at extracting knowledge but not planning. Kambhampati suggests that the papers claiming planning abilities of LLMs confuse general planning knowledge extracted from the LLMs with executable plans. "The plans that come out of LLMs may look reasonable to the lay user, and yet lead to execution time interactions and errors."

However, while there is a lot of anecdotal evidence that LLMs are poor planners, it's unclear whether it's because we don't know how to use LLMs the right way or because LLMs, fundamentally, can't plan.

Planning, at its core, is a search problem. You search among different paths to the goal, predict the outcome (reward) of each path, and pick the path with the most promising outcome. Often, you might determine that no path exists that can take you to the goal.

Search often requires *backtracking*. For example, imagine you're at a step where there are two possible actions: A and B. After taking action A, you

enter a state that's not promising, so you need to backtrack to the previous state to take action B.

Some people argue that an autoregressive model can only generate forward actions. It can't backtrack to generate alternate actions. Because of this, they conclude that autoregressive models can't plan. However, this isn't necessarily true. After executing a path with action A, if the model determines that this path doesn't make sense, it can revise the path using action B instead, effectively backtracking. The model can also always start over and choose another path.

It's also possible that LLMs are poor planners because they aren't given the toolings needed to plan. To plan, it's necessary to know not only the available actions but also *the potential outcome of each action*. As a simple example, let's say you want to walk up a mountain. Your potential actions are turn right, turn left, turn around, or go straight ahead. However, if turning right will cause you to fall off the cliff, you might not want to consider this action. In technical terms, an action takes you from one state to another, and it's necessary to know the outcome state to determine whether to take an action.

This means it's not sufficient to prompt a model to generate only a sequence of actions like what the popular chain-of-thought prompting technique does. The paper "Reasoning with Language Model is Planning with World Model" ([Hao et al., 2023](#)) argues that an LLM, by containing so much

information about the world, is capable of predicting the outcome of each action. This LLM can incorporate this outcome prediction to generate coherent plans.

Even if AI can't plan, it can still be a part of a planner. It might be possible to augment an LLM with a search tool and state tracking system to help it plan.

FOUNDATION MODEL (FM) VERSUS REINFORCEMENT LEARNING (RL) PLANNERS

The *agent* is a core concept in RL, which is defined in [Wikipedia](#) as a field “concerned with how an intelligent agent ought to take actions in a dynamic environment in order to maximize the cumulative reward.”

RL agents and FM agents are similar in many ways. They are both characterized by their environments and possible actions. The main difference is in how their planners work. In an RL agent, the planner is trained by an RL algorithm. Training this RL planner can require a lot of time and resources. In an FM agent, the model is the planner. This model can be prompted or finetuned to improve its planning capabilities, and generally requires less time and fewer resources.

However, there's nothing to prevent an FM agent from incorporating RL algorithms to improve its performance. I suspect that in the long run, FM agents and RL agents will merge.

Plan generation

The simplest way to turn a model into a plan generator is with prompt engineering. Imagine that you want to create an agent to help customers learn about products at Kitty Vogue. You give this agent access to three external tools: retrieve products by price, retrieve top products, and retrieve product information. Here's an example of a prompt for plan generation. This prompt is for illustration purposes only. Production prompts are likely more complex:

SYSTEM PROMPT

Propose a plan to solve the task. You have access to 5 actions:

get_today_date()

fetch_top_products(start_date, end_date, num_products)

fetch_product_info(product_name)

generate_query(task_history, tool_output)

generate_response(query)

The plan must be a sequence of valid actions.

Examples

Task: "Tell me about Fruity Fedora"

Plan: [fetch_product_info, generate_query, generate_response]

Task: "What was the best selling product last week?"

Plan: [fetch_top_products, generate_query, generate_response]

Task: {USER INPUT}

Plan:

There are two things to note about this example:

- The plan format used here—a list of functions whose parameters are inferred by the agent—is just one of many ways to structure the agent control flow.
- The `generate_query` function takes in the task’s current history and the most recent tool outputs to generate a query to be fed into the response generator. The tool output at each step is added to the task’s history.

Given the user input “What’s the price of the best-selling product last week”, a generated plan might look like this:

```
1. get_time()  
2. fetch_top_products()
```

```
3. fetch_product_info()  
4. generate_query()  
5. generate_response()
```

You might wonder, “What about the parameters needed for each function?”

The exact parameters are hard to predict in advance since they are often extracted from the previous tool outputs. If the first step, `get_time()`, outputs “2030-09-13”, then the agent can reason that the parameters for the next step should be called with the following parameters:

```
retrieve_top_products(  
    start_date="2030-09-07",  
    end_date="2030-09-13",  
    num_products=1  
)
```

Often, there’s insufficient information to determine the exact parameter values for a function. For example, if a user asks, “What’s the average price of best-selling products?”, the answers to the following questions are unclear:

- How many best-selling products does the user want to look at?
- Does the user want the best-selling products last week, last month, or of all time?

This means that models frequently have to guess, and guesses can be wrong.

Because both the action sequence and the associated parameters are generated by AI models, they can be hallucinated. Hallucinations can cause the model to call an invalid function or call a valid function but with wrong parameters. Techniques for improving a model's performance in general can be used to improve a model's planning capabilities.

Here are a few approaches to make an agent better at planning:

- Write a better system prompt with more examples.
- Give better descriptions of the tools and their parameters so that the model understands them better.
- Rewrite the functions themselves to make them simpler, such as refactoring a complex function into two simpler functions.
- Use a stronger model. In general, stronger models are better at planning.
- Finetune a model for plan generation.

Function calling

Many model providers offer tool use for their models, effectively turning their models into agents. A tool is a function. Invoking a tool is, therefore, often called *function calling*. Different model APIs work differently, but in general, function calling works as follows:

1. *Create a tool inventory.*

Declare all the tools that you might want a model to use. Each tool is described by its execution entry point (e.g., its function name), its parameters, and its documentation (e.g., what the function does and what parameters it needs).

2. *Specify what tools the agent can use.*

Because different queries might need different tools, many APIs let you specify a list of declared tools to be used per query. Some let you control tool use further by the following settings:

required

The model must use at least one tool.

none

The model shouldn't use any tool.

auto

The model decides which tools to use.

Function calling is illustrated in [Figure 6-10](#). This is written in pseudocode to make it representative of multiple APIs. To use a specific API, please refer to its documentation.

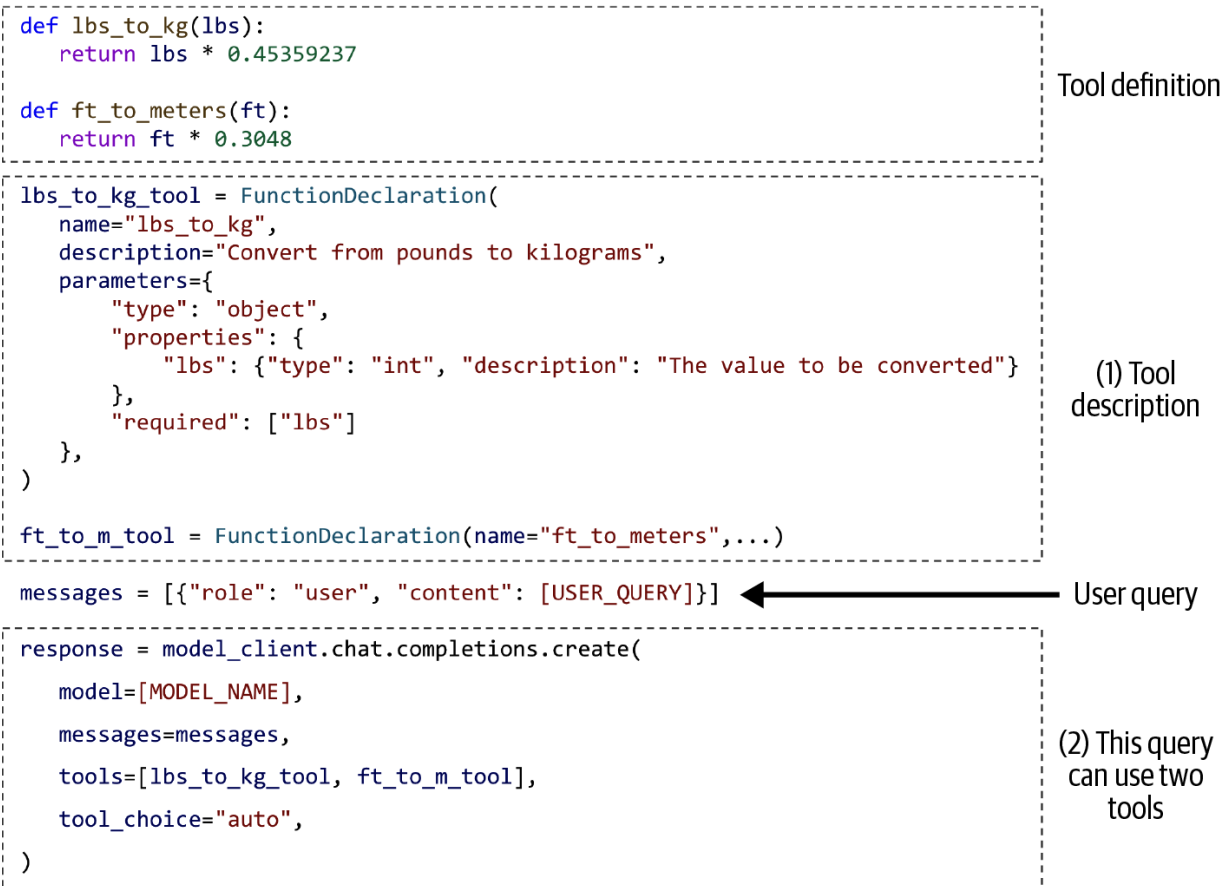


Figure 6-10. An example of a model using two simple tools.

Given a query, an agent defined as in [Figure 6-10](#) will automatically generate what tools to use and their parameters. Some function calling APIs will make sure that only valid functions are generated, though they won't be able to guarantee the correct parameter values.

For example, given the user query “How many kilograms are 40 pounds?”, the agent might decide that it needs the tool `lbs_to_kg_tool` with one parameter value of 40. The agent's response might look like this:

```
response = ModelResponse(
    finish_reason='tool_calls',
    message=chat.Message(
        content=None,
        role='assistant',
        tool_calls=[
            ToolCall(
                function=Function(
                    arguments='{"lbs":40}',
                    name='lbs_to_kg'),
                type='function')
        ])
    )
```

From this response, you can evoke the function `lbs_to_kg(lbs=40)` and use its output to generate a response to the users.

TIP

When working with agents, always ask the system to report what parameter values it uses for each function call. Inspect these values to make sure they are correct.

Planning granularity

A plan is a roadmap outlining the steps needed to accomplish a task. A roadmap can be of different levels of granularity. To plan for a year, a

quarter-by-quarter plan is higher-level than a month-by-month plan, which is, in turn, higher-level than a week-to-week plan.

There's a planning/execution trade-off. A detailed plan is harder to generate but easier to execute. A higher-level plan is easier to generate but harder to execute. An approach to circumvent this trade-off is to plan hierarchically. First, use a planner to generate a high-level plan, such as a quarter-to-quarter plan. Then, for each quarter, use the same or a different planner to generate a month-to-month plan.

So far, all examples of generated plans use the exact function names, which is very granular. A problem with this approach is that an agent's tool inventory can change over time. For example, the function to get the current date `get_time()` can be renamed to `get_current_time()`. When a tool changes, you'll need to update your prompt and all your examples. Using the exact function names also makes it harder to reuse a planner across different use cases with different tool APIs.

If you've previously finetuned a model to generate plans based on the old tool inventory, you'll need to finetune the model again on the new tool inventory.

To avoid this problem, plans can also be generated using a more natural language, which is higher-level than domain-specific function names. For

example, given the query “What’s the price of the best-selling product last week”, an agent can be instructed to output a plan that looks like this:

1. get current date
2. retrieve the best-selling product last week
3. retrieve product information
4. generate query
5. generate response

Using more natural language helps your plan generator become robust to changes in tool APIs. If your model was trained mostly on natural language, it’ll likely be better at understanding and generating plans in natural language and less likely to hallucinate.

The downside of this approach is that you need a translator to translate each natural language action into executable commands.¹³ However, translating is a much simpler task than planning and can be done by weaker models with a lower risk of hallucination.

Complex plans

The plan examples so far have been sequential: the next action in the plan is *always* executed after the previous action is done. The order in which actions can be executed is called a *control flow*. The sequential form is just one type of control flow. Other types of control flows include the parallel, if

statement, and for loop. The following list provides an overview of each control flow, including sequential for comparison:

Sequential

Executing task B after task A is complete, likely because task B depends on task A. For example, the SQL query can be executed only after it's been translated from the natural language input.

Parallel

Executing tasks A and B at the same time. For example, given the query “Find me best-selling products under \$100”, an agent might first retrieve the top 100 best-selling products and, for each of these products, retrieve its price.

If statement

Executing task B or task C depending on the output from the previous step. For example, the agent first checks NVIDIA's earnings report. Based on this report, it can then decide to sell or buy NVIDIA stocks.

For loop

Repeat executing task A until a specific condition is met. For example, keep on generating random numbers until a prime number.

These different control flows are visualized in [Figure 6-11](#).

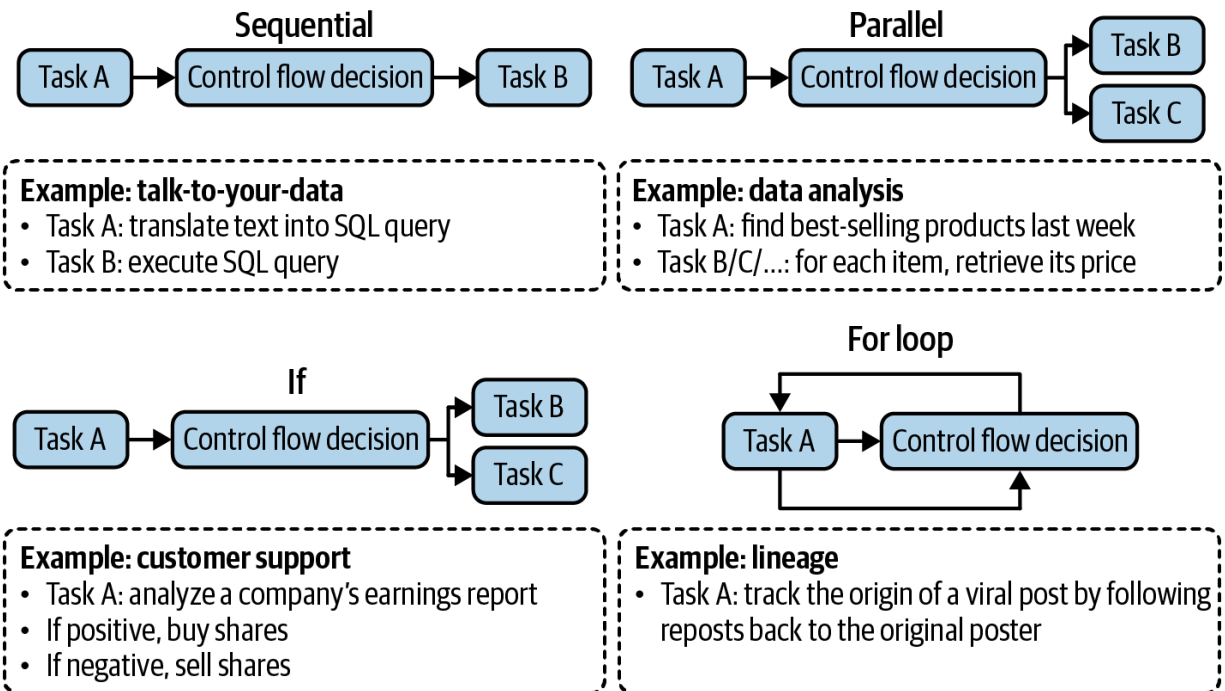


Figure 6-11. Examples of different orders in which a plan can be executed.

In traditional software engineering, conditions for control flows are exact. With AI-powered agents, AI models determine control flows. Plans with non-sequential control flows are more difficult to both generate and translate into executable commands.

When evaluating an agent framework, check what control flows it supports. For example, if the system needs to browse ten websites, can it do so simultaneously? Parallel execution can significantly reduce the latency perceived by users.

Reflection and error correction

Even the best plans need to be constantly evaluated and adjusted to maximize their chance of success. While reflection isn't strictly necessary for an agent to operate, it's necessary for an agent to succeed.

Reflection can be useful in many places during a task process:

- After receiving a user query to evaluate if the request is feasible.
- After the initial plan generation to evaluate whether the plan makes sense.
- After each execution step to evaluate if it's on the right track.
- After the whole plan has been executed to determine if the task has been accomplished.

Reflection and error correction are two different mechanisms that go hand in hand. Reflection generates insights that help uncover errors to be corrected.

Reflection can be done with the same agent using self-critique prompts. It can also be done with a separate component, such as a specialized scorer: a model that outputs a concrete score for each outcome.

First proposed by ReAct ([Yao et al., 2022](#)), interleaving reasoning and action has become a common pattern for agents. Yao et al. used the term “reasoning” to encompass both planning and reflection. At each step, the

agent is asked to explain its thinking (planning), take actions, then analyze observations (reflection), until the task is considered finished by the agent. The agent is typically prompted, using examples, to generate outputs in the following format:

```
Thought 1: ...  
Act 1: ...  
Observation 1: ...  
  
... [continue until reflection determines that the  
  
Thought N: ...  
Act N: Finish [Response to query]
```

[Figure 6-12](#) shows an example of an agent following the ReAct framework responding to a question from HotpotQA ([Yang et al., 2018](#)), a benchmark for multi-hop question answering.

You can implement reflection in a multi-agent setting: one agent plans and takes actions, and another agent evaluates the outcome after each step or after a number of steps.^{[14](#)}

If the agent's response failed to accomplish the task, you can prompt the agent to reflect on why it failed and how to improve. Based on this suggestion, the agent generates a new plan. This allows agents to learn from

their mistakes. For example, given a coding generation task, an evaluator might evaluate that the generated code fails $\frac{1}{3}$ of test cases. The agent then reflects the reason it failed is because it didn't take into account arrays where all numbers are negative. The actor then generates new code, taking into account all-negative arrays.

(1) Hotspot QA

Question: Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.

Act 1: `Search[Apple Remote]`

Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple ... originally designed to control the Front Row media center program ...

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.

Act 2: `Search[Front Row]`

Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports', 'Front Row (software)', ...]

Thought 3: Front Row is not found. I need to search Front Row (software) .

Act 3: `Search[Front Row (software)]`

Obs 3: Front Row is a discontinued media center software ...

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.

Act 4: `Finish[keyboard function keys]`



Figure 6-12. A ReAct agent in action. Image from the ReAct paper (Yao et al., 2022). The image is licensed under CC BY 4.0.

This is the approach that Reflexion ([Shinn et al., 2023](#)) took. In this framework, reflection is separated into two modules: an evaluator that evaluates the outcome and a self-reflection module that analyzes what went

wrong. [Figure 6-13](#) shows examples of Reflexion agents in action. The authors used the term “trajectory” to refer to a plan. At each step, after evaluation and self-reflection, the agent proposes a new trajectory.

Compared to plan generation, reflection is relatively easy to implement and can bring surprisingly good performance improvement. The downside of this approach is latency and cost. Thoughts, observations, and sometimes actions can take a lot of tokens to generate, which increases cost and user-perceived latency, especially for tasks with many intermediate steps. To nudge their agents to follow the format, both ReAct and Reflexion authors used plenty of examples in their prompts. This increases the cost of computing input tokens and reduces the context space available for other information.

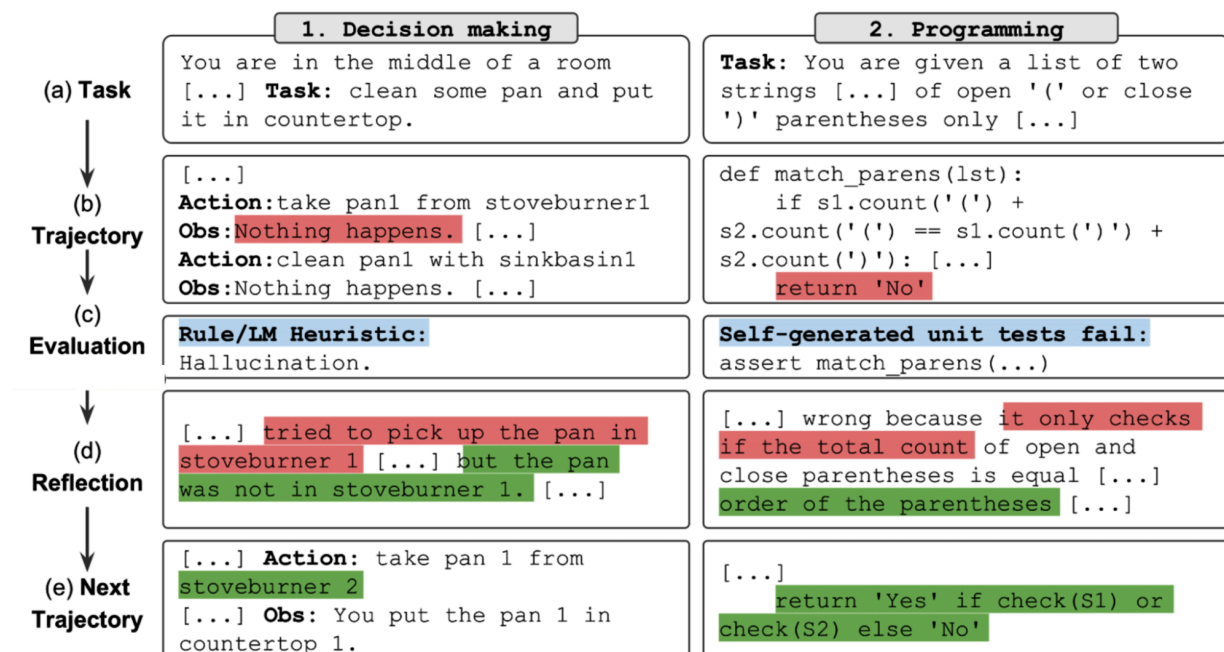


Figure 6-13. Examples of how Reflexion agents work. Images from the [Reflexion GitHub repo](#).

Tool selection

Because tools often play a crucial role in a task's success, tool selection requires careful consideration. The tools to give your agent depend on the environment and the task, but they also depend on the AI model that powers the agent.

There's no foolproof guide on how to select the best set of tools. Agent literature consists of a wide range of tool inventories. For example, Toolformer ([Schick et al., 2023](#)) finetuned GPT-J to learn five tools. Chameleon ([Lu et al., 2023](#)) uses 13 tools. On the other hand, Gorilla ([Patil et al., 2023](#)) attempted to prompt agents to select the right API call among 1,645 APIs.

More tools give the agent more capabilities. However, the more tools there are, the harder it is to efficiently use them. It's similar to how it's harder for humans to master a large set of tools. Adding tools also means increasing tool descriptions, which might not fit into a model's context.

Like many other decisions while building AI applications, tool selection requires experimentation and analysis. Here are a few things you can do to help you decide:

- Compare how an agent performs with different sets of tools.
- Do an ablation study to see how much the agent's performance drops if a tool is removed from its inventory. If a tool can be removed without a

performance drop, remove it.

- Look for tools that the agent frequently makes mistakes on. If a tool proves too hard for the agent to use—for example, extensive prompting and even finetuning can’t get the model to learn to use it—change the tool.
- Plot the distribution of tool calls to see what tools are most used and what tools are least used. [Figure 6-14](#) shows the differences in tool use patterns of GPT-4 and ChatGPT in Chameleon (Lu et al., 2023).

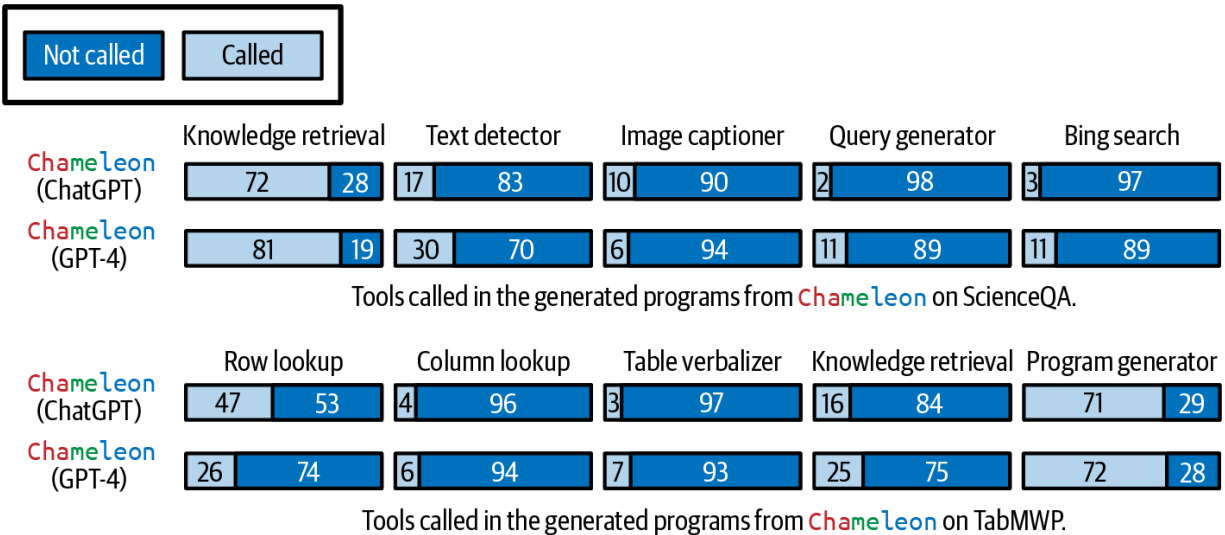


Figure 6-14. Different models and tasks express different tool use patterns. Image from Lu et al. (2023). Adapted from an original image licensed under CC BY 4.0.

Experiments by Lu et al. (2023) also demonstrate two points:

1. Different tasks require different tools. ScienceQA, the science question answering task, relies much more on knowledge retrieval tools than TabMWP, a tabular math problem-solving task.

2. Different models have different tool preferences. For example, GPT-4 seems to select a wider set of tools than ChatGPT. ChatGPT seems to favor image captioning, while GPT-4 seems to favor knowledge retrieval.

TIP

When evaluating an agent framework, evaluate what planners and tools it supports. Different frameworks might focus on different categories of tools. For example, AutoGPT focuses on social media APIs (Reddit, X, and Wikipedia), whereas Composio focuses on enterprise APIs (Google Apps, GitHub, and Slack).

As your needs will likely change over time, evaluate how easy it is to extend your agent to incorporate new tools.

As humans, we become more productive not just by using the tools we're given, but also by creating progressively more powerful tools from simpler ones. Can AI create new tools from its initial tools?

Chameleon (Lu et al., 2023) proposes the study of tool transition: after tool *X*, how likely is the agent to call tool *Y*? [Figure 6-15](#) shows an example of tool transition. If two tools are frequently used together, they can be combined into a bigger tool. If an agent is aware of this information, the agent itself can combine initial tools to continually build more complex tools.

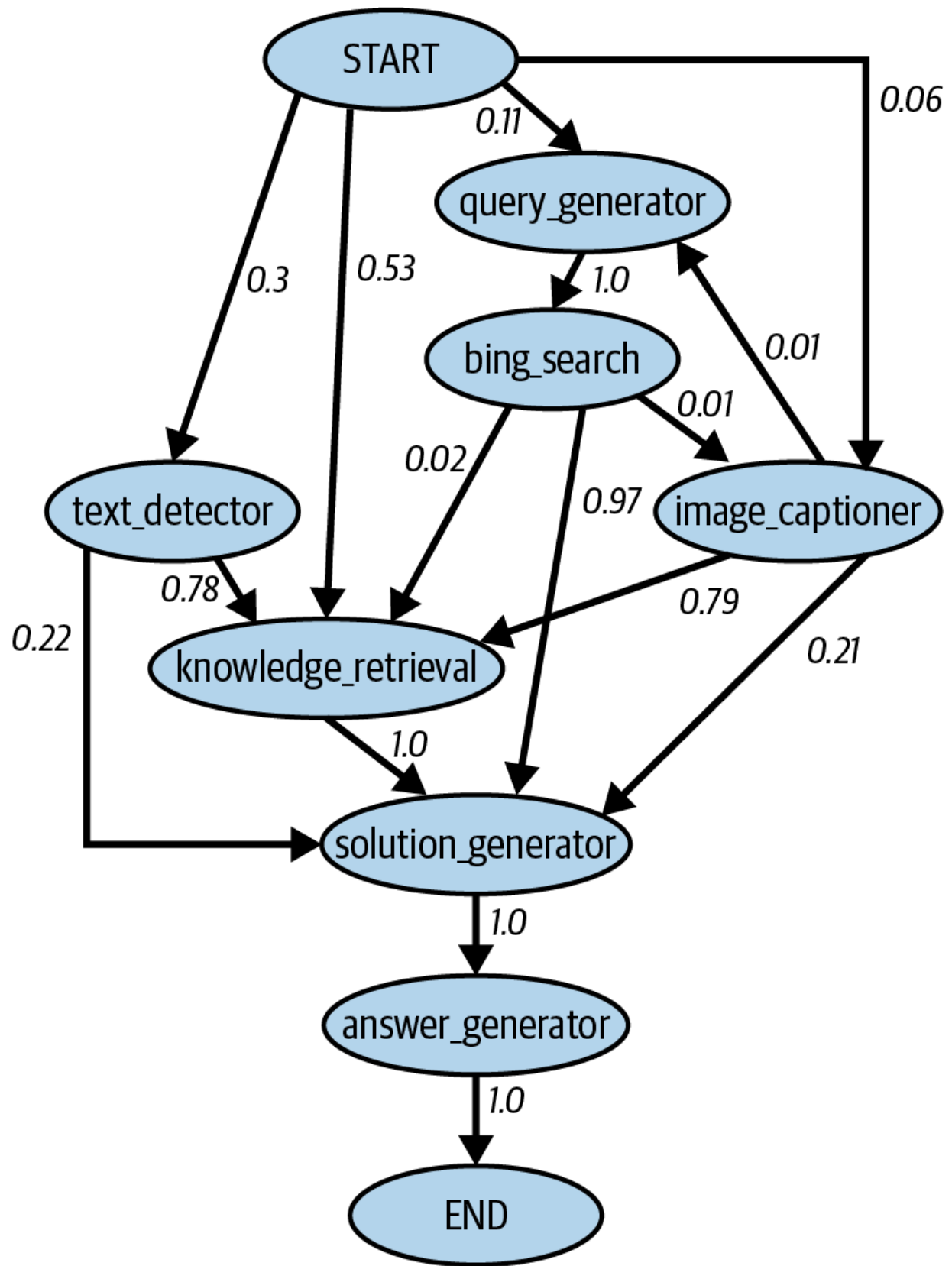


Figure 6-15. A tool transition tree by Lu et al. (2023). Adapted from an original image licensed under

Vogager ([Wang et al., 2023](#)) proposes a skill manager to keep track of new skills (tools) that an agent acquires for later reuse. Each skill is a coding program. When the skill manager determines a newly created skill is to be useful (e.g., because it's successfully helped an agent accomplish a task), it adds this skill to the skill library (conceptually similar to the tool inventory). This skill can be retrieved later to use for other tasks.

Earlier in this section, we mentioned that the success of an agent in an environment depends on its tool inventory and its planning capabilities. Failures in either aspect can cause the agent to fail. The next section will discuss different failure modes of an agent and how to evaluate them.

Agent Failure Modes and Evaluation

Evaluation is about detecting failures. The more complex a task an agent performs, the more possible failure points there are. Other than the failure modes common to all AI applications discussed in Chapters [3](#) and [4](#), agents also have unique failures caused by planning, tool execution, and efficiency. Some of the failures are easier to catch than others.

To evaluate an agent, identify its failure modes and measure how often each of these failure modes happens.

I created a simple benchmark to illustrate these different failure modes that you can see on the book's [GitHub repository](#). There are also agent benchmarks and leaderboards such as the [Berkeley Function Calling Leaderboard](#), the [AgentOps evaluation harness](#), and the [TravelPlanner benchmark](#).

Planning failures

Planning is hard and can fail in many ways. The most common mode of planning failure is tool use failure. The agent might generate a plan with one or more of these errors:

Invalid tool

For example, it generates a plan that contains `bing_search`, but `bing_search` isn't in the agent's tool inventory.

Valid tool, invalid parameters.

For example, it calls `lbs_to_kg` with two parameters. `lbs_to_kg` is in the tool inventory but requires only one parameter, `lbs`.

Valid tool, incorrect parameter values

For example, it calls `lbs_to_kg` with one parameter, `lbs`, but uses the value 100 for lbs when it should be 120.

Another mode of planning failure is goal failure: the agent fails to achieve the goal. This can be because the plan doesn't solve a task, or it solves the task without following the constraints. To illustrate this, imagine you ask the model to plan a two-week trip from San Francisco to Hanoi with a budget of \$5,000. The agent might plan a trip from San Francisco to Ho Chi Minh City, or plan a two-week trip from San Francisco to Hanoi that will be way over the budget.

A common constraint that is often overlooked by agent evaluation is time. In many cases, the time an agent takes matters less, because you can assign a task to an agent and only need to check in when it's done. However, in many cases, the agent becomes less useful with time. For example, if you ask an agent to prepare a grant proposal and the agent finishes it after the grant deadline, the agent isn't very helpful.

An interesting mode of planning failure is caused by errors in reflection. The agent is convinced that it's accomplished a task when it hasn't. For example, you ask the agent to assign 50 people to 30 hotel rooms. The agent might assign only 40 people and insist that the task has been accomplished.

To evaluate an agent for planning failures, one option is to create a planning dataset where each example is a tuple `(task, tool inventory)`. For each task, use the agent to generate a K number of plans. Compute the following metrics:

1. Out of all generated plans, how many are valid?
2. For a given task, how many plans does the agent have to generate, on average, to get a valid plan?
3. Out of all tool calls, how many are valid?
4. How often are invalid tools called?
5. How often are valid tools called with invalid parameters?
6. How often are valid tools called with incorrect parameter values?

Analyze the agent's outputs for patterns. What types of tasks does the agent fail more on? Do you have a hypothesis why? What tools does the model frequently make mistakes with? Some tools might be harder for an agent to use. You can improve an agent's ability to use a challenging tool by better prompting, more examples, or finetuning. If all fail, you might consider swapping this tool for something easier to use.

Tool failures

Tool failures happen when the correct tool is used, but the tool output is wrong. One failure mode is when a tool just gives the wrong outputs. For example, an image captioner returns a wrong description, or an SQL query generator returns a wrong SQL query.

If the agent generates only high-level plans and a translation module is involved in translating from each planned action to executable commands, failures can happen because of translation errors.

Tool failures can also happen because the agent doesn't have access to the right tools for the task. An obvious example is when the task involves retrieving the current stock prices from the internet, and the agent doesn't have access to the internet.

Tool failures are tool-dependent. Each tool needs to be tested independently. Always print out each tool call and its output so that you can inspect and evaluate them. If you have a translator, create benchmarks to evaluate it.

Detecting missing tool failures requires an understanding of what tools should be used. If your agent frequently fails on a specific domain, this might be because it lacks tools for this domain. Work with human domain experts and observe what tools they would use.

Efficiency

An agent might generate a valid plan using the right tools to accomplish a task, but it might be inefficient. Here are a few things you might want to track to evaluate an agent's efficiency:

- How many steps does the agent need, on average, to complete a task?
- How much does the agent cost, on average, to complete a task?
- How long does each action typically take? Are there any actions that are especially time-consuming or expensive?

You can compare these metrics with your baseline, which can be another agent or a human operator. When comparing AI agents to human agents, keep in mind that humans and AI have very different modes of operations, so what's considered efficient for humans might be inefficient for AI, and vice versa. For example, visiting 100 web pages might be inefficient for a human agent who can visit only one page at a time, but trivial for an AI agent that can visit all the web pages at once.

In this chapter, we've discussed in detail how RAG and agent systems function. Both patterns often deal with information that exceeds a model's context limit. A memory system that supplements the model's context in handling information can significantly enhance its capabilities. Let's now explore how a memory system works.

Memory

Memory refers to mechanisms that allow a model to retain and utilize information. A memory system is especially useful for knowledge-rich applications like RAG and multi-step applications like agents. A RAG system relies on memory for its augmented context, which can grow over multiple turns as it retrieves more information. An agentic system needs memory to store instructions, examples, context, tool inventories, plans, tool outputs, reflections, and more. While RAG and agents place greater

demands on memory, it is beneficial for any AI application that requires retaining information.

An AI model typically has three main memory mechanisms:

Internal knowledge

The model itself is a memory mechanism, as it retains the knowledge from the data it was trained on. This knowledge is its *internal knowledge*. A model's internal knowledge doesn't change unless the model itself is updated. The model can access this knowledge in all queries.

Short-term memory

A model's context is a memory mechanism. Previous messages in a conversation can be added to the model's context, allowing the model to leverage them to generate future responses. A model's context can be considered its *short-term memory* as it doesn't persist across tasks (queries). It's fast to access, but its capacity is limited. Therefore, it's often used to store information that is most important for the current task.

Long-term memory

External data sources that a model can access via retrieval, such as in a RAG system, are a memory mechanism. This can be considered the model's *long-term memory*, as it can be persisted across tasks. Unlike

a model's internal knowledge, information in the long-term memory can be deleted without updating the model.

Humans have access to similar memory mechanisms. How to breathe is your internal knowledge. You typically don't forget how to breathe unless you're in serious trouble. Your short-term memory contains information immediately relevant to what you're doing, such as the name of a person you just met. Your long-term memory is augmented with books, computers, notes, etc.

Which memory mechanism to use for your data depends on its frequency of use. Information essential for all tasks should be incorporated into the model's internal knowledge via training or finetuning. Information that is rarely needed should reside in its long-term memory. Short-term memory is reserved for immediate, context-specific information. These three memory mechanisms are illustrated in [Figure 6-16](#).

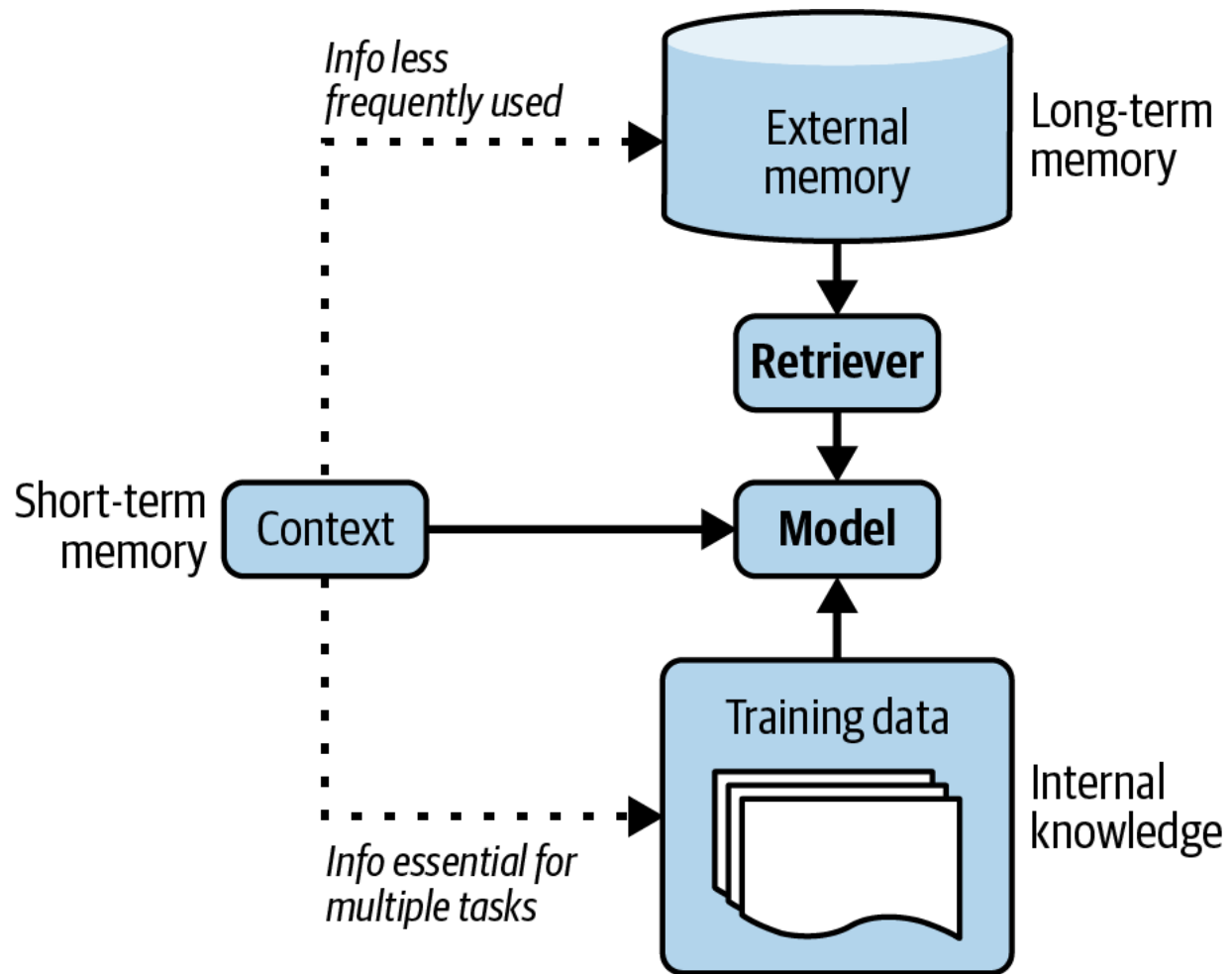


Figure 6-16. The hierarchy of information for an agent.

Memory is essential for humans to operate. As AI applications have evolved, developers have quickly realized that memory is important for AI models, too. Many memory management tools for AI models have been developed, and many model providers have incorporated external memory. Augmenting an AI model with a memory system has many benefits. Here are just a few of them:

Manage information overflow within a session

During the process of executing a task, an agent acquires a lot of new information, which can exceed the agent's maximum context length. The excess information can be stored in a memory system with long-term memories.

Persist information between sessions

An AI coach is practically useless if every time you want the coach's advice, you have to explain your whole life story. An AI assistant would be annoying to use if it keeps forgetting your preferences. Having access to your conversation history can allow an agent to personalize its actions to you. For example, when you ask for book recommendations, if the model remembers that you've previously loved *The Three-Body Problem*, it can suggest similar books.

Boost a model's consistency

If you ask me a subjective question twice, like rating a joke between 1 and 5, I'm much more likely to give consistent answers if I remember my previous answer. Similarly, if an AI model can reference its previous answers, it can calibrate its future answers to be consistent.

Maintain data structural integrity

Because text is inherently unstructured, the data stored in the context of a text-based model is unstructured. You can put structured data in

the context. For example, you can feed a table into the context line-by-line, but there's no guarantee that the model will understand that this is supposed to be a table. Having a memory system capable of storing structured data can help maintain the structural integrity of your data. For example, if you ask an agent to find potential sales leads, this agent can leverage an Excel sheet to store the leads. An agent can also leverage a queue to store the sequence of actions to be performed.

A memory system for AI models typically consists of two functions:

- Memory management: managing what information should be stored in the short-term and long-term memory.
- Memory retrieval: retrieving information relevant to the task from long-term memory.

Memory retrieval is similar to RAG retrieval, as long-term memory is an external data source. In this section, I'll focus on memory management.

Memory management typically consists of two operations: *add* and *delete* memory. If memory storage is limited, deletion might not be necessary. This might work for long-term memory because external memory storage is relatively cheap and easily extensible. However, short-term memory is limited by the model's maximum context length and, therefore, requires a strategy for what to add and what to delete.

Long-term memory can be used to store the overflow from short-term memory. This operation depends on how much space you want to allocate for short-term memory. For a given query, the context input into the model consists of both its short-term memory and information retrieved from its long-term memory. A model's short-term capacity is, therefore, determined by how much of the context should be allocated for information retrieved from long-term memory. For example, if 30% of the context is reserved, then the model can use at most 70% of the context limit for short-term memory. When this threshold is reached, the overflow can be moved to long-term memory.

Like many components previously discussed in this chapter, memory management isn't unique to AI applications. Memory management has been a cornerstone of all data systems, and many strategies have been developed to use memory efficiently.

The simplest strategy is FIFO, first in, first out. The first to be added to the short-term memory will be the first to be moved to the external storage. As a conversation gets longer, API providers like OpenAI might start removing the beginning of the conversation. Frameworks like LangChain might allow the retention of N last messages or N last tokens. In a long conversation, this strategy assumes that the early messages are less relevant to the current discussion. However, this assumption can be fatally wrong. In some conversations, the earliest messages might carry the most information, especially when the early messages state the purpose of the conversation.

While FIFO is straightforward to implement, it can cause the model to lose track of important information.¹⁶

More-sophisticated strategies involve removing redundancy. Human languages contain redundancy to enhance clarity and compensate for potential misunderstandings. If there's a way to automatically detect redundancy, the memory footprint will be reduced significantly.

One way to remove redundancy is by using a summary of the conversation. This summary can be generated using the same or another model.

Summarization, together with tracking named entities, can take you a long way. [Bae et al. \(2022\)](#) took this a step further. After obtaining the summary, the authors wanted to construct a new memory by joining the memory with the key information that the summary missed. The authors developed a classifier that, for each sentence in the memory and each sentence in the summary, determines if only one, both, or neither should be added to the new memory.

[Liu et al. \(2023\)](#), on the other hand, used a reflection approach. After each action, the agent is asked to do two things:

1. Reflect on the information that has just been generated.
2. Determine if this new information should be inserted into the memory, should merge with the existing memory, or should replace some other

information, especially if the other information is outdated and contradicts new information.

When encountering contradicting pieces of information, some people opt to keep the newer ones. Some people ask AI models to judge which one to keep. How to handle contradiction depends on the use case. Having contradictions can cause an agent to be confused but can also help it draw from different perspectives.

Summary

Given the popularity of RAG and the potential of agents, early readers have mentioned that this is the chapter they're most excited about.

This chapter started with RAG, the pattern that emerged first between the two. Many tasks require extensive background knowledge that often exceeds a model's context window. For example, code copilots might need access to entire codebases, and research assistants may need to analyze multiple books. Originally developed to overcome a model's context limitations, RAG also enables more efficient use of information, improving response quality while reducing costs. From the early days of foundation models, it was clear that the RAG pattern would be immensely valuable for a wide range of applications, and it has since been rapidly adopted across both consumer and enterprise use cases.

RAG employs a two-step process. It first retrieves relevant information from external memory and then uses this information to generate more accurate responses. The success of a RAG system depends on the quality of its retriever. Term-based retrievers, such as Elasticsearch and BM25, are much lighter to implement and can provide strong baselines. Embedding-based retrievers are more computationally intensive but have the potential to outperform term-based algorithms.

Embedding-based retrieval is powered by vector search, which is also the backbone of many core internet applications such as search and recommender systems. Many vector search algorithms developed for these applications can be used for RAG.

The RAG pattern can be seen as a special case of agent where the retriever is a tool the model can use. Both patterns allow a model to circumvent its context limitation and stay more up-to-date, but the agentic pattern can do even more than that. An agent is defined by its environment and the tools it can access. In an AI-powered agent, AI is the planner that analyzes its given task, considers different solutions, and picks the most promising one. A complex task can require many steps to solve, which requires a powerful model to plan. A model's ability to plan can be augmented with reflection and a memory system to help it keep track of its progress.

The more tools you give a model, the more capabilities the model has, enabling it to solve more challenging tasks. However, the more automated

the agent becomes, the more catastrophic its failures can be. Tool use exposes agents to many security risks discussed in [Chapter 5](#). For agents to work in the real world, rigorous defensive mechanisms need to be put in place.

Both RAG and agents work with a lot of information, which often exceeds the maximum context length of the underlying model. This necessitates the introduction of a memory system for managing and using all the information a model has. This chapter ended with a short discussion on what this component looks like.

RAG and agents are both prompt-based methods, as they influence the model's quality solely through inputs without modifying the model itself. While they can enable many incredible applications, modifying the underlying model can open up even more possibilities. How to do so will be the topic of the next chapter.

- The model used was a type of [recurrent neural network](#) known as [LSTM](#) (Long Short-Term Memory). LSTM was the dominant architecture of deep learning for natural language processing (NLP) before the transformer architecture took over in 2018.
- Around the same time, another paper, also from Facebook, “How Context Affects Language Models’ Factual Predictions” ([Petroni et al., arXiv, May 2020](#)), showed that augmenting a pre-trained language model with a retrieval system can dramatically improve the model's performance on factual questions.
- Thanks to Chetan Tekur for the example.

- ! Parkinson's Law is usually expressed as "Work expands so as to fill the time available for its completion." I have a similar theory that an application's context expands to fill the context limit supported by the model it uses.
- ! Information retrieval was described as early as the 1920s in Emanuel Goldberg's patents for a "statistical machine" to search documents stored on films. See ["The History of Information Retrieval Research"](#) (Sanderson and Croft, *Proceedings of the IEEE, 100: Special Centennial Issue*, April 2012).
- ! For those interested in learning more about BM25, I recommend this paper by the BM25 authors: ["The Probabilistic Relevance Framework: BM25 and Beyond"](#) (Robertson and Zaragoza, *Foundations and Trends in Information Retrieval* 3 No. 4, 2009)
- ! [Aravind Srinivas, the CEO of Perplexity](#), tweeted that "Making a genuine improvement over BM25 or full-text search is hard".
- ! A RAG retrieval workflow shares many similar steps with the traditional recommender system.
- ! Some teams have told me that their retrieval systems work best when the data is organized in a question-and-answer format.
- ! *Artificial Intelligence: A Modern Approach* (1995) defines an agent as anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.
- A complaint in the early days of agents is that agents are only good for burning through your API credits.
- ! Because most agentic workflows are sufficiently complex to involve multiple components, most agents are multi-agent.
- ! Chameleon ([Lu et al., 2023](#)) calls this translator a program generator.

- ! This reminds me of the actor-critic (AC) agent method ([Konda and Tsitsiklis, 1999](#)) in reinforcement learning.
- ! For human conversations, the opposite might be true if the first few messages are pleasantries.
- ! Usage-based strategies, such as removing the least frequently used information, is more challenging, since you'll need a way to know when a model uses a given piece of information.

Chapter 7. Finetuning

Finetuning is the process of adapting a model to a specific task by further training the whole model or part of the model. Chapters [5](#) and [6](#) discuss prompt-based methods, which adapt a model by giving it instructions, context, and tools. Finetuning adapts a model by adjusting its weights.

Finetuning can enhance various aspects of a model. It can improve the model's domain-specific capabilities, such as coding or medical question answering, and can also strengthen its safety. However, it is most often used to improve the model's instruction-following ability, particularly to ensure it adheres to specific output styles and formats.

While finetuning can help create models that are more customized to your needs, it also requires more up-front investment. A question I hear very often is when to finetune and when to do RAG. After an overview of finetuning, this chapter will discuss the reasons for finetuning and the reasons for not finetuning, as well as a simple framework for thinking about choosing between finetuning and alternate methods.

Compared to prompt-based methods, finetuning incurs a much higher memory footprint. At the scale of today's foundation models, naive finetuning often requires more memory than what's available on a single GPU. This makes finetuning expensive and challenging to do. As discussed throughout this chapter, reducing memory requirements is a primary

motivation for many finetuning techniques. This chapter dedicates one section to outlining factors contributing to a model's memory footprint, which is important for understanding these techniques.

A memory-efficient approach that has become dominant in the finetuning space is PEFT (parameter-efficient finetuning). This chapter explores PEFT and how it differs from traditional finetuning; this chapter also provides an overview of its evolving techniques. I'll focus particularly on one compelling category: adapter-based techniques.

With prompt-based methods, knowledge about how ML models operate under the hood is recommended but not strictly necessary. However, finetuning brings you to the realm of model training, where ML knowledge is required. ML basics are beyond the scope of this book. If you want a quick refresh, the book's [GitHub repository](#) has pointers to helpful resources. In this chapter, I'll cover a few core concepts immediately relevant to the discussion.

This chapter is the most technically challenging one for me to write, not because of the complexity of the concepts, but because of the broad scope these concepts cover. I suspect it might also be technically challenging to read. If, at any point, you feel like you're diving too deep into details that aren't relevant to your work, feel free to skip.

There's a lot to discuss. Let's dive in!

Finetuning Overview

To finetune, you start with a base model that has some, but not all, of the capabilities you need. The goal of finetuning is to get this model to perform well enough for your specific task.

Finetuning is one way to do *transfer learning*, a concept first introduced by [Bozinovski and Fulgosi](#) in 1976. Transfer learning focuses on how to transfer the knowledge gained from one task to accelerate learning for a new, related task. This is conceptually similar to how humans transfer skills: for example, knowing how to play the piano can make it easier to learn another musical instrument.

An early large-scale success in transfer learning was Google’s multilingual translation system ([Johnson et. al, 2016](#)). The model transferred its knowledge of Portuguese–English and English–Spanish translation to directly translate Portuguese to Spanish, even though there were no Portuguese–Spanish examples in the training data.

Since the early days of deep learning, transfer learning has offered a solution for tasks with limited or expensive training data. By training a base model on tasks with abundant data, you can then transfer that knowledge to a target task.

For LLMs, knowledge gained from pre-training on text completion (a task with abundant data) is transferred to more specialized tasks, like legal question answering or text-to-SQL, which often have less available data. This capability for transfer learning makes foundation models particularly valuable.

Transfer learning improves *sample efficiency*, allowing a model to learn the same behavior with fewer examples. A *sample-efficient* model learns effectively from fewer samples. For example, while training a model from scratch for legal question answering may need millions of examples, finetuning a good base model might only require a few hundred.

Ideally, much of what the model needs to learn is already present in the base model, and finetuning just refines the model's behavior. OpenAI's [InstructGPT paper](#) (2022) suggested viewing finetuning as unlocking the capabilities a model already has but that are difficult for users to access via prompting alone.

NOTE

Finetuning isn't the only way to do transfer learning. Another approach is *feature-based transfer*. In this approach, a model is trained to extract features from the data, usually as embedding vectors, which are then used by another model. I mention feature-based transfer briefly in [Chapter 2](#), when discussing how part of a foundation model can be reused for a classification task by *adding a classifier head*.

Feature-based transfer is very common in computer vision. For instance, in the second half of the 2010s, many people used models trained on the ImageNet dataset to extract features from images and use these features in other computer vision tasks such as object detection or image segmentation.

Finetuning is part of a model's training process. It's an extension of model pre-training. Because any training that happens after pre-training is finetuning, finetuning can take many different forms. [Chapter 2](#) already discussed two types of finetuning: supervised finetuning and preference finetuning. Let's do a quick recap of these methods and how you might leverage them as an application developer.

Recall that a model's training process starts with *pre-training*, which is usually done with self-supervision. Self-supervision allows the model to learn from a large amount of unlabeled data. For language models, self-supervised data is typically just *sequences of text* that don't need annotations.

Before finetuning this pre-trained model with expensive task-specific data, you can finetune it with self-supervision using cheap task-related data. For

example, to finetune a model for legal question answering, before finetuning it on expensive annotated (question, answer) data, you can finetune it on raw legal documents. Similarly, to finetune a model to do book summarization in Vietnamese, you can first finetune it on a large collection of Vietnamese text. *Self-supervised finetuning* is also called *continued pre-training*.

As discussed in [Chapter 1](#), language models can be autoregressive or masked. An autoregressive model predicts the next token in a sequence using the previous tokens as the context. A masked model fills in the blank using the tokens both before and after it. Similarly, with supervised finetuning, you can also finetune a model to predict the next token or fill in the blank. The latter, also known as *infilling finetuning*, is especially useful for tasks such as text editing and code debugging. You can finetune a model for infilling even if it was pre-trained autoregressively.

The massive amount of data a model can learn from during self-supervised learning outfits the model with a rich understanding of the world, but it might be hard for users to extract that knowledge for their tasks, or the way the model behaves might be misaligned with human preference. Supervised finetuning uses high-quality annotated data to refine the model to align with human usage and preference.

During *supervised finetuning*, the model is trained using (input, output) pairs: the input can be an instruction and the output can be a response. A

response can be open-ended, such as for the task of book summarization. A response can be also close-ended, such as for a classification task. High-quality instruction data can be challenging and expensive to create, especially for instructions that require factual consistency, domain expertise, or political correctness. [Chapter 8](#) discusses how to acquire instruction data.

A model can also be finetuned with reinforcement learning to generate responses that maximize human preference. Preference finetuning requires comparative data that typically follows the format (instruction, winning response, losing response).

It's possible to finetune a model to extend its context length. *Long-context finetuning* typically requires modifying the model's architecture, such as adjusting the positional embeddings. A long sequence means more possible positions for tokens, and positional embeddings should be able to handle them. Compared to other finetuning techniques, long-context finetuning is harder to do. The resulting model might also degrade on shorter sequences.

[Figure 7-1](#) shows the making of different Code Llama models ([Rozière et al., 2024](#)), from the base model Llama 2, using different finetuning techniques. Using long-context finetuning, they were able to increase the model's maximum context length from 4,096 tokens to 16,384 tokens to accommodate longer code files. In the image, instruction finetuning refers to supervised finetuning.

Finetuning can be done by both model developers and application developers. Model developers typically post-train a model with different finetuning techniques before releasing it. A model developer might also release different model versions, each finetuned to a different extent, so that application developers can choose the version that works best for them.

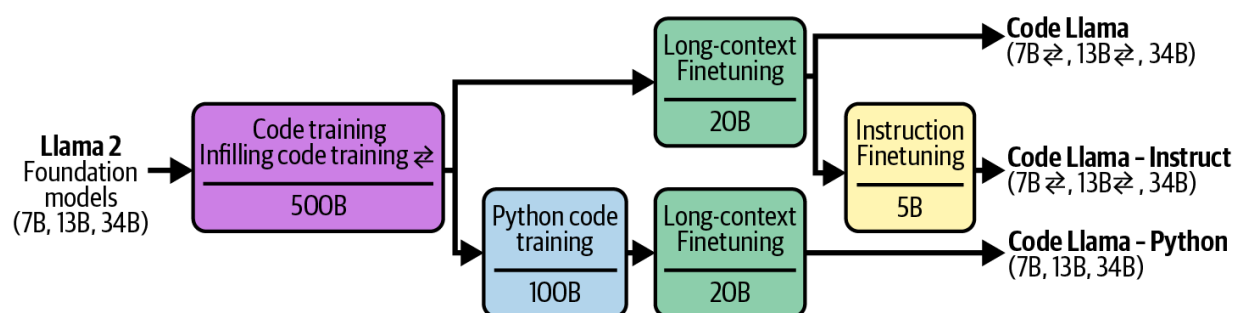


Figure 7-1. Different finetuning techniques used to make different Code Llama models. Image from the Rozière et al. (2024). Adapted from an original image licensed under CC BY 4.0.

As an application developer, you might finetune a pre-trained model, but most likely, you'll finetune a model that has been post-trained. The more refined a model is and the more relevant its knowledge is to your task, the less work you'll have to do to adapt it.

When to Finetune

Before jumping into different finetuning techniques, it's necessary to consider whether finetuning is the right option for you. Compared to prompt-based methods, finetuning requires significantly more resources, not just in data and hardware, but also in ML talent. Therefore, finetuning is

generally attempted *after* extensive experiments with prompt-based methods. However, finetuning and prompting aren't mutually exclusive. Real-world problems often require both approaches.

Reasons to Finetune

The primary reason for finetuning is to improve a model's quality, in terms of both general capabilities and task-specific capabilities. Finetuning is commonly used to improve a model's ability to generate outputs following specific structures, such as JSON or YAML formats.

A general-purpose model that performs well on a wide range of benchmarks might not perform well on your specific task. If the model you want to use wasn't sufficiently trained on your task, finetuning it with your data can be especially useful.

For example, an out-of-the-box model might be good at converting from text to the standard SQL dialect but might fail with a less common SQL dialect. In this case, finetuning this model on data containing this SQL dialect will help. Similarly, if the model works well on standard SQL for common queries but often fails for customer-specific queries, finetuning the model on customer-specific queries might help.

One especially interesting use case of finetuning is bias mitigation. The idea is that if the base model perpetuates certain biases from its training data, exposing it to carefully curated data during finetuning can counteract these

biases ([Wang and Russakovsky, 2023](#)). For example, if a model consistently assigns CEOs male-sounding names, finetuning it on a dataset with many female CEOs can mitigate this bias. [Garimella et al. \(2022\)](#) found that finetuning BERT-like language models on text authored by women can reduce these models' gender biases, while finetuning them on texts by African authors can reduce racial biases.

You can finetune a big model to make it even better, but finetuning smaller models is much more common. Smaller models require less memory, and, therefore, are easier to finetune. They are also cheaper and faster to use in production.

A common approach is to finetune a small model to imitate the behavior of a larger model using data generated by this large model. Because this approach distills the larger model's knowledge into the smaller model, it's called *distillation*. This is discussed in [Chapter 8](#) together with other data synthesis techniques.

A small model, finetuned on a specific task, might outperform a much larger out-of-the-box model on that task. For example, Grammarly found that their finetuned Flan-T5 models ([Chung et al., 2022](#)) outperformed a GPT-3 variant specialized in text editing across a wide range of writing assistant tasks despite being 60 times smaller. The finetuning process used only 82,000 (instruction, output) pairs, which is smaller than the data typically needed to train a text-editing model from scratch.

In the early days of foundation models, when the strongest models were commercial with limited finetuning access, there weren't many competitive models available for finetuning. However, as the open source community proliferates with high-quality models of all sizes, tailored for a wide variety of domains, finetuning has become a lot more viable and attractive.

Reasons Not to Finetune

While finetuning can improve a model in many ways, many of these improvements can also be achieved, to a certain extent, without finetuning. Finetuning can improve a model's performance, but so do carefully crafted prompts and context. Finetuning can help with structured outputs, but many other techniques, as discussed in [Chapter 2](#), can also do that.

First, while finetuning a model for a specific task can improve its performance for that task, it can degrade its performance for other tasks.¹ This can be frustrating when you intend this model for an application that expects diverse prompts.

Imagine you need a model for three types of queries: product recommendations, changing orders, and general feedback. Originally, the model works well for product recommendations and general feedback but poorly for changing orders. To fix this, you finetune the model on a dataset of (query, response) pairs about changing orders. The finetuned model

might indeed perform better for this type of query, but worse for the two other tasks.

What do you do in this situation? You can finetune the model on all the queries you care about, not just changing orders. If you can't seem to get a model to perform well on all your tasks, consider using separate models for different tasks. If you wish to combine these separate models into one to make serving them easier, you can also consider merging them together, as discussed later in this chapter.

If you're just starting to experiment with a project, finetuning is rarely the first thing you should attempt. Finetuning requires high up-front investments and continual maintenance. First, you need data. Annotated data can be slow and expensive to acquire manually, especially for tasks that demand critical thinking and domain expertise. Open source data and AI-generated data can mitigate the cost, but their effectiveness is highly variable.

Second, finetuning requires the knowledge of how to train models. You need to evaluate base models to choose one to finetune. Depending on your needs and resources, options might be limited. While finetuning frameworks and APIs can automate many steps in the actual finetuning process, you still need to understand the different training knobs you can tweak, monitor the learning process, and debug when something is wrong. For example, you need to understand how an optimizer works, what

learning rate to use, how much training data is needed, how to address overfitting/underfitting, and how to evaluate your models throughout the process.

Third, once you have a finetuned model, you'll need to figure out how to serve it. Will you host it yourself or use an API service? As discussed in [Chapter 9](#), inference optimization for large models, especially LLMs, isn't trivial. Finetuning requires less of a technical leap if you're already hosting your models in-house and familiar with how to operate models.

More importantly, you need to establish a policy and budget for monitoring, maintaining, and updating your model. As you iterate on your finetuned model, new base models are being developed at a rapid pace. These base models may improve faster than you can enhance your finetuned model. If a new base model outperforms your finetuned model on your specific task, how significant does the performance improvement have to be before you switch to the new base model? What if a new base model doesn't immediately outperform your existing model but has the potential to do so after finetuning—would you experiment with it?

In many cases, switching to a better model would provide only a small incremental improvement, and your task might be given a lower priority than projects with larger returns, like enabling new use cases.^{[2](#)}

AI engineering experiments should start with prompting, following the best practices discussed in [Chapter 6](#). Explore more advanced solutions only if prompting alone proves inadequate. Ensure you have thoroughly tested various prompts, as a model's performance can vary greatly with different prompts.

Many practitioners I've spoken with share a similar story that goes like this. Someone complains that prompting is ineffective and insists on finetuning. Upon investigation, it turns out that prompt experiments were minimal and unsystematic. Instructions were unclear, examples didn't represent actual data, and metrics were poorly defined. After refining the prompt experiment process, the prompt quality improved enough to be sufficient for their application.³

FINETUNING DOMAIN-SPECIFIC TASKS

Beware of the argument that general-purpose models don't work well for domain-specific tasks, and, therefore, you must finetune or train models for your specific tasks. As general-purpose models become more capable, they also become better at domain-specific tasks and can outperform the domain-specific models.

An interesting early specialized model is BloombergGPT, which was introduced by Bloomberg in March 2023. The strongest models on the market then were all proprietary, and Bloomberg wanted a mid-size model that performed well on financial tasks and could be hosted in-house for use cases with sensitive data. The model, with 50 billion parameters, required 1.3 million A100 GPU hours for training. The estimated cost of the compute was between \$1.3 million and \$2.6 million, excluding data costs ([Wu et al., 2023](#)).

In the same month, OpenAI released GPT-4-0314.⁴ Research by [Li et al. \(2023\)](#) demonstrated that GPT-4-0314 significantly outperformed BloombergGPT across various financial benchmarks. [Table 7-1](#) provides details of two such benchmarks.

Table 7-1. General-purpose models like GPT-4 can outperform financial models in financial domains.

Model	FiQA sentiment analysis (weighted F1)	ConvFinQA (accuracy)
GPT-4-0314 (zero-shot)	87.15	76.48
BloombergGPT	75.07	43.41

Since then, several mid-size models with performance comparable to GPT-4 have been released, including [Claude 3.5 Sonnet](#) (70B parameters), [Llama 3-70B-Instruct](#), and [Qwen2-72B-Instruct](#). The latter two are open weight and can be self-hosted.

Because benchmarks are insufficient to capture real-world performance, it’s possible that BloombergGPT works well for Bloomberg for their specific use cases. The Bloomberg team certainly gained invaluable experience through training this model, which might enable them to better develop and operate future models.

Both finetuning and prompting experiments require systematic processes. Doing prompt experiments enables developers to build an evaluation pipeline, data annotation guideline, and experiment tracking practices that will be stepping stones for finetuning.

One benefit of finetuning, before prompt caching was introduced, was that it can help optimize token usage. The more examples you add to a prompt, the more input tokens the model will use, which increases both latency and cost. Instead of including your examples in each prompt, you can finetune a model on these examples. This allows you to use shorter prompts with the finetuned model, as shown in [Figure 7-2](#).

With prompt caching, where repetitive prompt segments can be cached for reuse, this is no longer a strong benefit. Prompt caching is discussed further in [Chapter 9](#). However, the number of examples you can use with a prompt is still limited by the maximum context length. With finetuning, there's no limit to how many examples you can use.

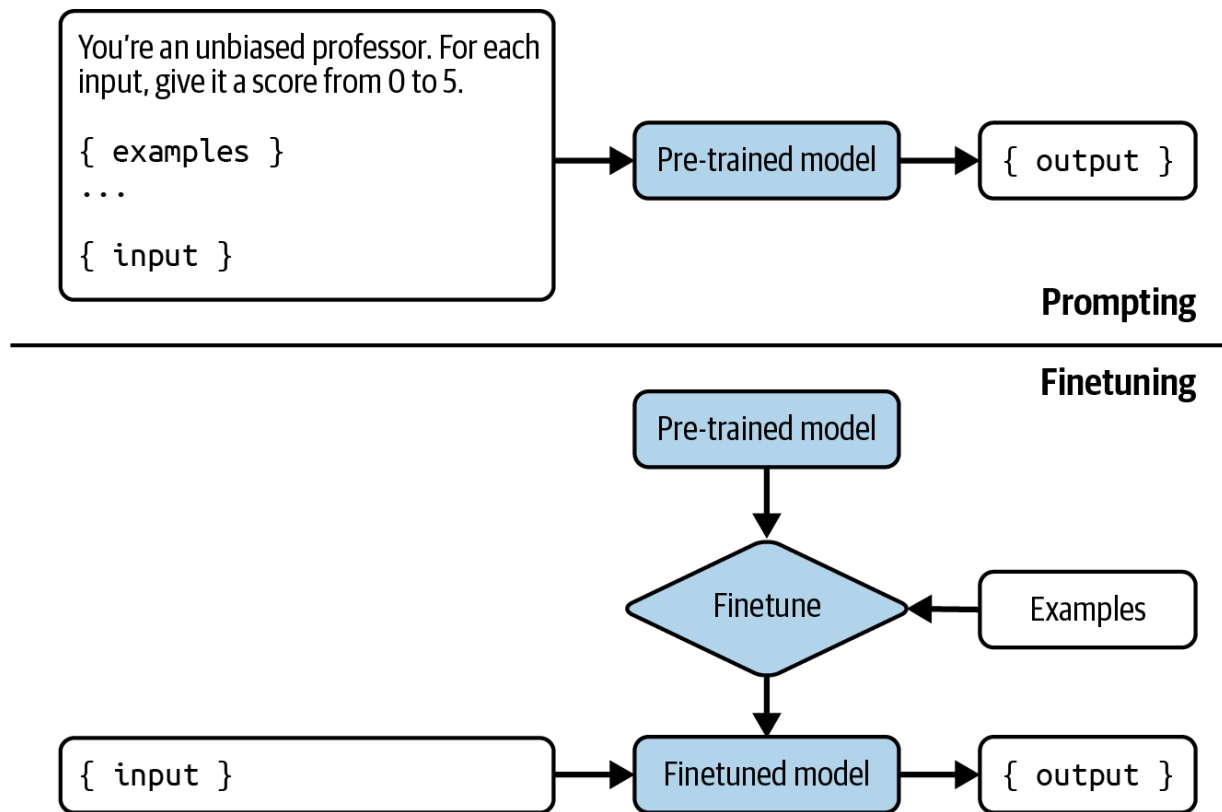


Figure 7-2. Instead of including examples in each prompt, which increases cost and latency, you finetune a model on these examples.

Finetuning and RAG

Once you've maximized the performance gains from prompting, you might wonder whether to do RAG or finetuning next. The answer depends on whether your model's failures are information-based or behavior-based.

If the model fails because it lacks information, a RAG system that gives the model access to the relevant sources of information can help. Information-based failures happen when the outputs are factually wrong or outdated. Here are two example scenarios in which information-based failures happen:

The model doesn't have the information.

Public models are unlikely to have information private to you or your organization. When a model doesn't have the information, it either tells you so or hallucinates an answer.

The model has outdated information.

If you ask: "How many studio albums has Taylor Swift released?" and the correct answer is 11, but the model answers 10, it can be because the model's cut-off date was before the release of the latest album.

The paper ["Fine-Tuning or Retrieval?"](#) by Ovadia et al. (2024) demonstrated that for tasks that require up-to-date information, such as questions about current events, RAG outperformed finetuned models. Not only that, RAG with the base model outperformed RAG with finetuned models, as shown in [Table 7-2](#). This finding indicates that *while finetuning can enhance a model's performance on a specific task, it may also lead to a decline in performance in other areas.*

Table 7-2. RAG outperforms finetuning on a question-answering task about current events, curated by different finetuning approaches the author used.

	Base model	Base model + RAG	FT-reg	FT-p
Mistral-7B	0.481	0.875	0.504	0.588
Llama 2-7B	0.353	0.585	0.219	0.392
Orca 2-7B	0.456	0.876	0.511	0.566

On the other hand, *if the model has behavioral issues, finetuning might help*. One behavioral issue is when the model’s outputs are factually correct but irrelevant to the task. For example, you ask the model to generate technical specifications for a software project to provide to your engineering teams. While accurate, the generated specs lack the details your teams need. Finetuning the model with well-defined technical specifications can make the outputs more relevant.

Another issue is when it fails to follow the expected output format. For example, if you asked the model to write HTML code, but the generated code didn’t compile, it might be because the model wasn’t sufficiently exposed to HTML in its training data. You can correct this by exposing the model to more HTML code during finetuning.

Semantic parsing is a category of tasks whose success hinges on the model's ability to generate outputs in the expected format and, therefore, often requires finetuning. Semantic parsing is discussed briefly in Chapters [2](#) and [6](#). As a reminder, semantic parsing means converting natural language into a structured format like JSON. Strong off-the-shelf models are generally good for common, less complex syntaxes like JSON, YAML, and regex. However, they might not be as good for syntaxes with fewer available examples on the internet, such as a domain-specific language for a less popular tool or a complex syntax.

In short, finetuning is for form, and RAG is for facts. A RAG system gives your model external knowledge to construct more accurate and informative answers. A RAG system can help mitigate your model's hallucinations. Finetuning, on the other hand, helps your model understand and follow syntaxes and styles.^{[5](#)} While finetuning can potentially reduce hallucinations if done with enough high-quality data, it can also worsen hallucinations if the data quality is low.

If your model has both information and behavior issues, start with RAG. RAG is typically easier since you won't have to worry about curating training data or hosting the finetuned models. When doing RAG, start with simple term-based solutions such as BM25 instead of jumping straight into something that requires vector databases.

RAG can also introduce a more significant performance boost than finetuning. Ovadia et al. (2024) showed that for almost all question categories in the [MMLU benchmark](#), RAG outperforms finetuning for three different models: Mistral 7B, Llama 2-7B, and Orca 2-7B.

However, RAG and finetuning aren't mutually exclusive. They can sometimes be used together to maximize your application's performance. In the same experiment, [Ovadia et al. \(2024\)](#) showed that incorporating RAG on top of a finetuned model can boost its performance on the MMLU benchmark 43% of the time. It's important to note that in this experiment, using RAG with finetuned models doesn't improve the performance 57% of the time, compared to using RAG alone.

There's no universal workflow for all applications. [Figure 7-3](#) shows some paths an application development process might follow over time. The arrow indicates what next step you might try. This figure is inspired by an example workflow shown by [OpenAI](#) (2023).

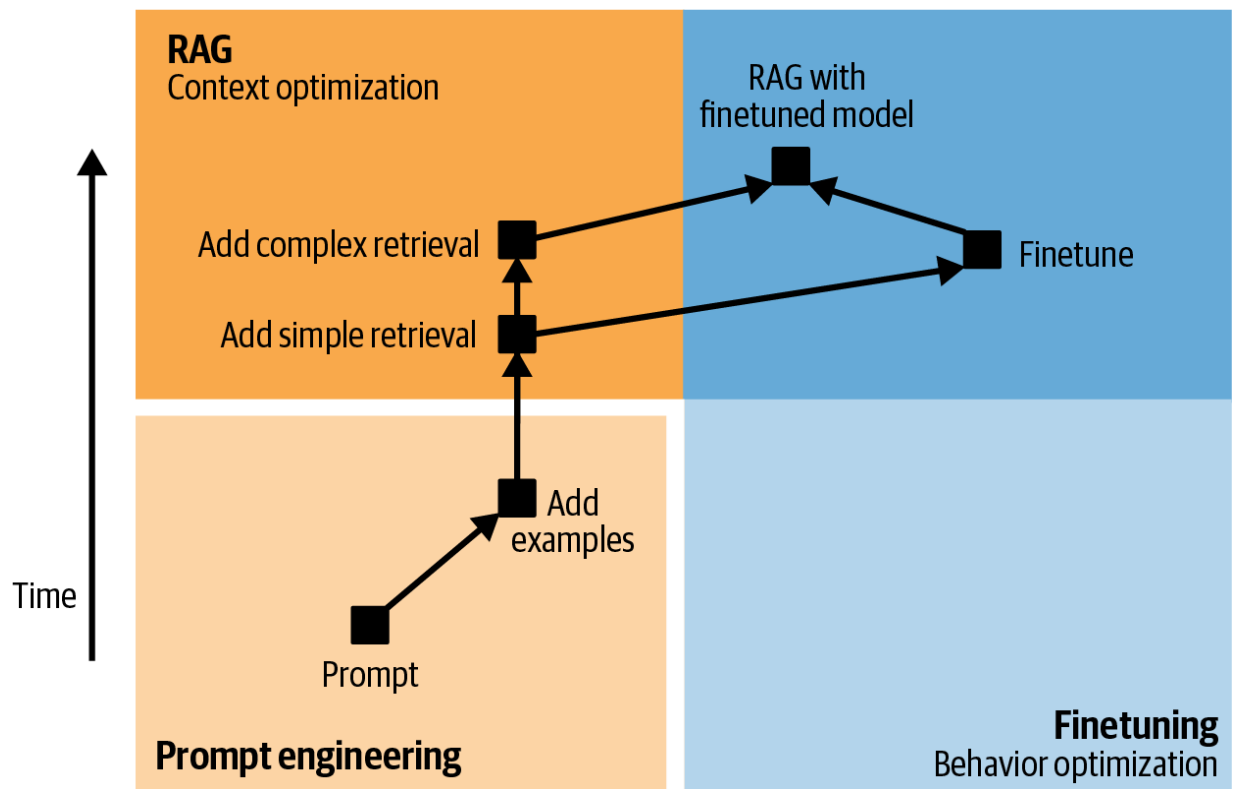


Figure 7-3. Example application development flows. After simple retrieval (such as term-based retrieval), whether to experiment with more complex retrieval (such as hybrid search) or finetuning depends on each application and its failure modes.

So the workflow to adapt a model to a task might work as follows. Note that before any of the adaptation steps, you should define your evaluation criteria and design your evaluation pipeline, as discussed in [Chapter 4](#). This evaluation pipeline is what you'll use to benchmark your progress as you develop your application. Evaluation doesn't happen only in the beginning. It should be present during every step of the process:

1. Try to get a model to perform your task with prompting alone. Use the prompt engineering best practices covered in [Chapter 5](#), including systematically versioning your prompts.

2. Add more examples to the prompt. Depending on the use case, the number of examples needed might be between 1 and 50.
3. If your model frequently fails due to missing information, connect it to data sources that can supply relevant information. When starting with RAG, begin by using basic retrieval methods like term-based search. Even with simple retrieval, adding relevant and accurate knowledge should lead to some improvement in your model's performance.
4. Depending on your model's failure modes, you might explore one of these next steps:
 - a. If the model continues having information-based failures, you might want to try even more advanced RAG methods, such as embedding-based retrieval.
 - b. If the model continues having behavioral issues, such as it keeps generating irrelevant, malformed, or unsafe responses, you can opt for finetuning. Embedding-based retrieval increases inference complexity by introducing additional components into the pipeline, while finetuning increases the complexity of model development but leaves inference unchanged.
5. Combine both RAG and finetuning for even more performance boost.

If, after considering all the pros and cons of finetuning and other alternate techniques, you decide to finetune your model, the rest of the chapter is for you. First, let's look into the number one challenge of finetuning: its memory bottleneck.

Memory Bottlenecks

Because finetuning is memory-intensive, many finetuning techniques aim to minimize their memory footprint. Understanding what causes this memory bottleneck is necessary to understand why and how these techniques work. This understanding, in turn, can help you select a finetuning method that works best for you.

Besides explaining finetuning's memory bottleneck, this section also introduces formulas for back-of-the-napkin calculation of the memory usage of each model. This calculation is useful in estimating what hardware you'd need to serve or finetune a model.

Because memory calculation requires a breakdown of low-level ML and computing concepts, this section is technically dense. If you're already familiar with these concepts, feel free to skip them.

KEY TAKEAWAYS FOR UNDERSTANDING MEMORY BOTTLENECKS

If you decide to skip this section, here are a few key takeaways. If you find any of these takeaways unfamiliar, the concepts in this section should help explain it:

1. Because of the scale of foundation models, memory is a bottleneck for working with them, both for inference and for finetuning. The memory needed for finetuning is typically much higher than the memory needed for inference because of the way neural networks are trained.
2. The key contributors to a model's memory footprint during finetuning are its number of parameters, its number of trainable parameters, and its numerical representations.
3. The more trainable parameters, the higher the memory footprint. You can reduce memory requirement for finetuning by reducing the number of trainable parameters. Reducing the number of trainable parameters is the motivation for PEFT, parameter-efficient finetuning.
4. Quantization refers to the practice of converting a model from a format with more bits to a format with fewer bits. Quantization is a straightforward and efficient way to reduce a model's memory footprint. For a model of 13 billion parameters, using FP32 means 4 bytes per weight or 52 GB for the whole weights. If you can reduce each value to only 2 bytes, the memory needed for the model's weights decreases to 26 GB.

5. Inference is typically done using as few bits as possible, such as 16 bits, 8 bits, and even 4 bits.
 6. Training is more sensitive to numerical precision, so it's harder to train a model in lower precision. Training is typically done in mixed precision, with some operations done in higher precision (e.g., 32-bit) and some in lower precision (e.g., 16-bit or 8-bit).
-

Backpropagation and Trainable Parameters

A key factor that determines a model's memory footprint during finetuning is its number of *trainable parameters*. A trainable parameter is a parameter that can be updated during finetuning. During pre-training, all model parameters are updated. During inference, no model parameters are updated. During finetuning, some or all model parameters may be updated. The parameters that are kept unchanged are *frozen parameters*.

The memory needed for each trainable parameter results from the way a model is trained. As of this writing, neural networks are typically trained using a mechanism called *backpropagation*.⁶ With backpropagation, each training step consists of two phases:

1. Forward pass: the process of computing the output from the input.
2. Backward pass: the process of updating the model's weights using the aggregated signals from the forward pass.

During inference, only the forward pass is executed. During training, both passes are executed. At a high level, the backward pass works as follows:

1. Compare the computed output from the forward pass against the expected output (ground truth). If they are different, the model made a mistake, and the parameters need to be adjusted. The difference between the computed output and the expected output is called the *loss*.
2. Compute how much each trainable parameter contributes to the mistake. This value is called the *gradient*. Mathematically, gradients are computed by taking the derivative of the loss with respect to each trainable parameter. There's one gradient value per trainable parameter.⁷ If a parameter has a high gradient, it significantly contributes to the loss and should be adjusted more.
3. Adjust trainable parameter values using their corresponding gradient. How much each parameter should be readjusted, given its gradient value, is determined by the *optimizer*. Common optimizers include SGD (stochastic gradient descent) and Adam. For transformer-based models, Adam is, by far, the most widely used optimizer.

The forward and backward pass for a hypothetical neural network with three parameters and one nonlinear activation function is visualized in [Figure 7-4](#). I use this dummy neural network to simplify the visualization.

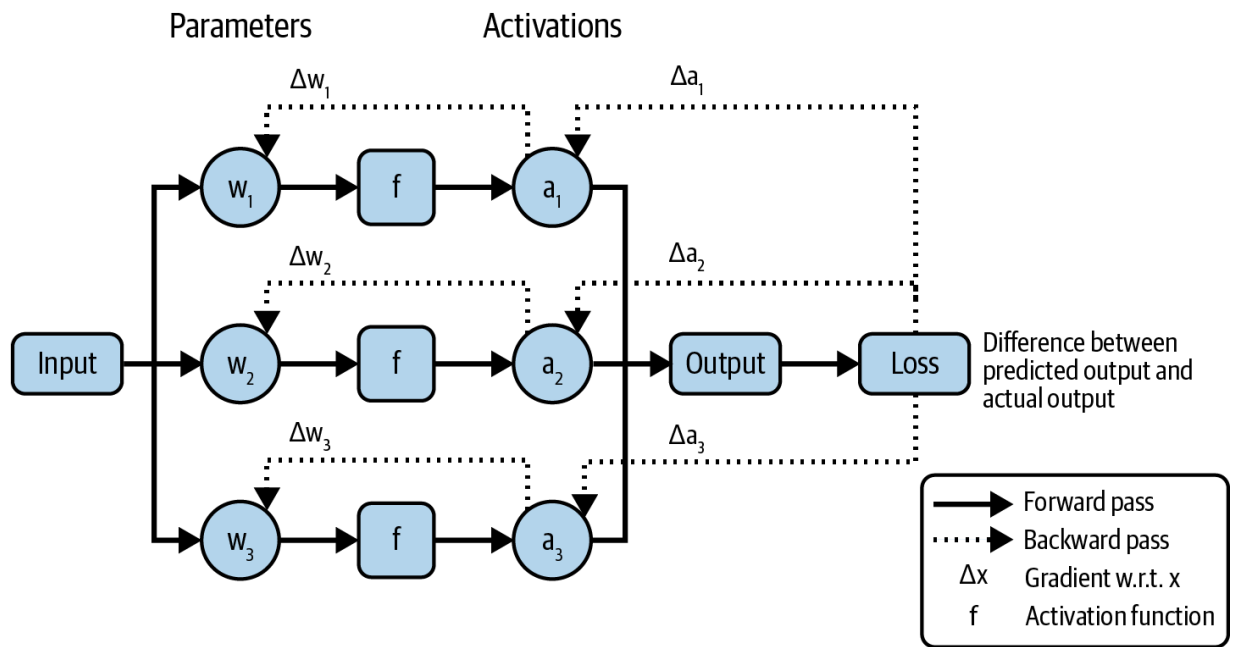


Figure 7-4. The forward and backward pass of a simple neural network.

During the backward pass, each trainable parameter comes with additional values, its gradient, and its optimizer states. Therefore, the more trainable parameters there are, the more memory is needed to store these additional values.

Memory Math

It's useful to know how much memory a model needs so that you can use the right hardware for it. Often, you might already have the hardware and need to calculate whether you can afford to run a certain model. If a model requires 30 GB of memory to do inference, a chip with 24 GB of memory won't be sufficient.

A model's memory footprint depends on the model as well as the workload and the different optimization techniques used to reduce its memory usage. Because it's impossible to account for all optimization techniques and workloads, in this section, I'll outline only the formulas for approximate calculations, which should give you a rough idea of how much memory you need to operate a model, both during inference and training.

NOTE

Inference and training having distinct memory profiles is one of the reasons for the divergence in chips for training and inference, as discussed in [Chapter 9](#).

Memory needed for inference

During inference, only the forward pass is executed. The forward pass requires memory for the model's weights. Let N be the model's parameter count and M be the memory needed for each parameter; the memory needed to load the model's parameters is:

$$N \times M$$

The forward pass also requires memory for activation values. Transformer models need memory for key-value vectors for the attention mechanism. The memory for both activation values and key-value vectors grows linearly with sequence length and batch size.

For many applications, the memory for activation and key-value vectors can be assumed to be 20% of the memory for the model's weights. If your application uses a longer context or larger batch size, the actual memory needed will be higher. This assumption brings the model's memory footprint to:

$$N \times M \times 1.2$$

Consider a 13B-parameter model. If each parameter requires 2 bytes, the model's weights will require $13\text{B} \times 2 \text{ bytes} = 26 \text{ GB}$. The total memory for inference will be $26 \text{ GB} \times 1.2 = 31.2 \text{ GB}$.

A model's memory footprint grows rapidly with its size. As models become bigger, memory becomes a bottleneck for operating them.⁸ A 70B-parameter model with 2 bytes per parameter will require a whopping 140 GB of memory just for its weights.⁹

Memory needed for training

To train a model, you need memory for the model's weights and activations, which has already been discussed. Additionally, you need memory for gradients and optimizer states, which scales with the number of trainable parameters.

Overall, the memory needed for training is calculated as:

- Training memory = model weights + activations + gradients + optimizer states

TIP

During the backward pass, each trainable parameter requires one value for gradient plus zero to two values for optimizer states, depending on the optimizer:

- A vanilla SGD optimizer has no state.
- A momentum optimizer stores one value per trainable parameter.
- An Adam optimizer stores two values per trainable parameter.

Imagine you're updating all parameters in a 13B-parameter model using the Adam optimizer. Because each trainable parameter has three values for its gradient and optimizer states, if it takes two bytes to store each value, the memory needed for gradients and optimizer states will be:

$$13 \text{ billion} \times 3 \times 2 \text{ bytes} = 78 \text{ GB}$$

However, if you only have 1B trainable parameters, the memory needed for gradients and optimizer states will be only:

$$1 \text{ billion} \times 3 \times 2 \text{ bytes} = 6 \text{ GB}$$

One important thing to note is that in the previous formula, I assumed that the memory needed for activations is less than the memory needed for the model's weights. However, in reality, the activation memory can be much larger. If activations are stored for gradient computation, the memory needed for activations can dwarf the memory needed for the model's weights. [Figure 7-5](#) shows the memory needed for activations compared to the memory needed for the model's weights for different Megatron models at different scales, according to the paper [“Reducing Activation Recomputation in Large Transformer Models”](#), by Korthikanti et al. (2022).

One way to reduce the memory needed for activations is not to store them. Instead of storing activations for reuse, you recompute activations when necessary. This technique is called *gradient checkpointing* or *activation recomputation*. While this reduces the memory requirements, it increases the time needed for training due to the recomputation. ¹⁰

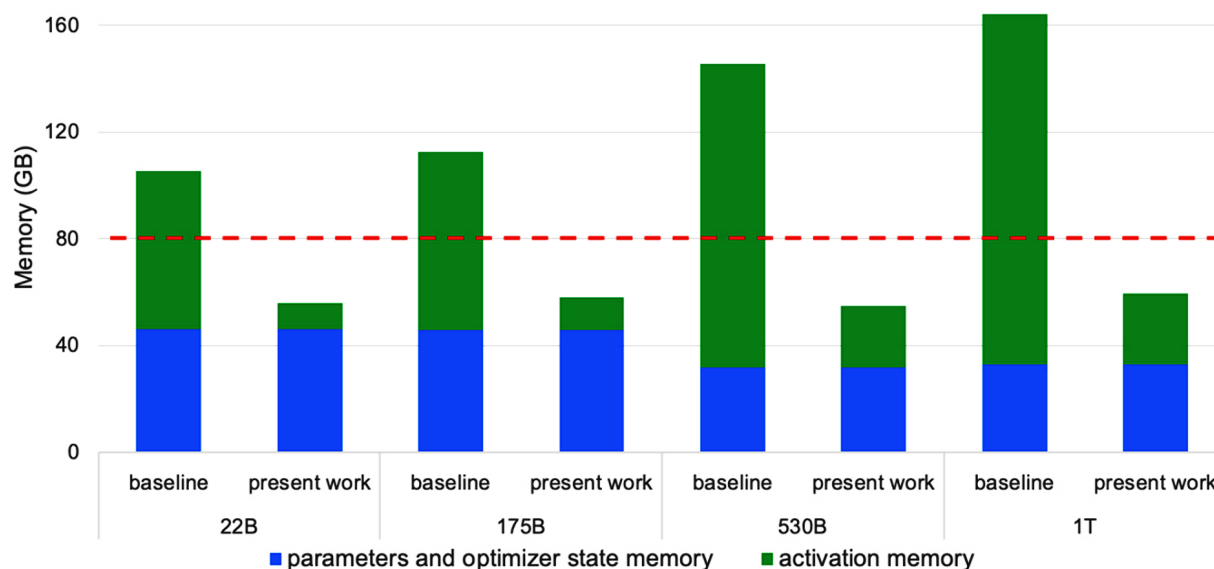


Figure 7-5. The memory needed for activations can dwarf the memory needed for the model's weights. Image from Korthikanti et al., 2022.

Numerical Representations

In the memory calculation so far, I've assumed that each value takes up two bytes of memory. The memory required to represent each value in a model contributes directly to the model's overall memory footprint. If you reduce the memory needed for each value by half, the memory needed for the model's weights is also reduced by half.

Before discussing how to reduce the memory needed for each value, it's useful to understand numerical representations. Numerical values in neural networks are traditionally represented as float numbers. The most common family of floating point formats is the FP family, which adheres to the Institute of Electrical and Electronics Engineers (IEEE) standard for Floating-Point Arithmetic (IEEE 754):

- FP32 uses 32 bits (4 bytes) to represent a float. This format is called single precision.
- FP64 uses 64 bits (8 bytes) and is called double precision.
- FP16 uses 16 bits (2 bytes) and is called half precision.

While FP64 is still used in many computations—as of this writing, FP64 is the default format for NumPy and pandas—it’s rarely used in neural networks because of its memory footprint. FP32 and FP16 are more common. Other popular floating point formats in AI workloads include *BF16* (BFloat16) and *TF32* (TensorFloat-32). BF16 was designed by Google to optimize AI performance on TPUs and TF32 was designed by NVIDIA for GPUs.¹¹

Numbers can also be represented as integers. Even though not yet as common as floating formats, integer representations are becoming increasingly popular. Common integer formats are INT8 (8-bit integers) and INT4 (4-bit integers).¹²

Each float format usually has 1 bit to represent the number’s sign, i.e., negative or positive. The rest of the bits are split between *range* and *precision*:¹³

Range

The number of range bits determines the range of values the format can represent. More bits means a wider range. This is similar to how

having more digits lets you represent a wider range of numbers.

Precision

The number of precision bits determines how precisely a number can be represented. Reducing the number of precision bits makes a number less precise. For example, if you convert 10.1234 to a format that can support only two decimal digits, this value becomes 10.12, which is less precise than the original value.

[Figure 7-6](#) shows different floating point formats along with their range and precision bits. ¹⁴

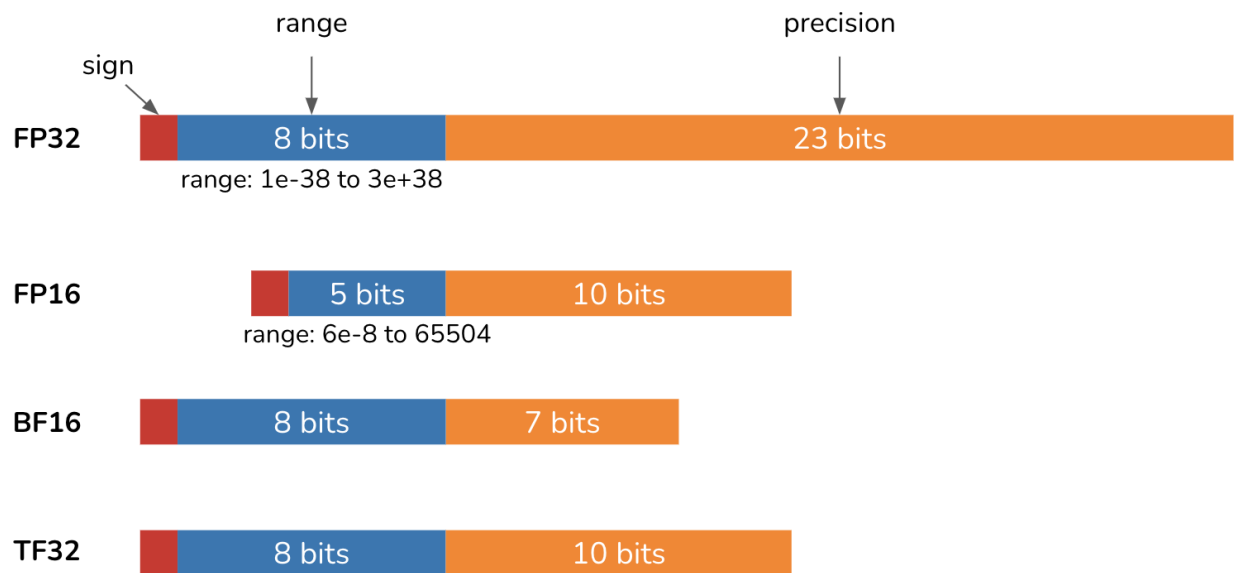


Figure 7-6. Different numerical formats with their range and precision.

Formats with more bits are considered *higher precision*. Converting a number with a high-precision format into a low-precision format (e.g., from FP32 to FP16) means *reducing its precision*. Reducing precision can cause

a value to change or result in errors. [Table 7-3](#) shows how FP32 values can be converted into FP16, BF16, and TF32.

Table 7-3. Convert from FP32 values to lower-precision formats. Resultant inaccuracies are in italics.

FP32	FP16	BF16	TF32
0.0123456789	0.0123443603515625	0.0123291	0.01234436035
0.123456789	0.12347412109375	0.123535	0.12341308593
1.23456789	1.234375	1.23438	1.234375
12.3456789	12.34375	12.375	12.34375
123.456789	123.4375	123.5	123.4375
1234.56789	1235.0	1232.0	1234.0
12345.6789	12344.0	12352.0	12344.0
123456.789	INF ^a	123392.0	123456.0
1234567.89	INF	1236990.0	1233920.0

^a Values out of bound in FP16 are rounded to infinity.

Note in [Table 7-3](#) that even though BF16 and FP16 have the same number of bits, BF16 has more bits for range and fewer bits for precision. This

allows BF16 to represent large values that are out-of-bound for FP16. However, this also makes BF16 less precise than FP16. For example, 1234.56789 is 1235.0 in FP16 (0.035% value change) but 1232.0 in BF16 (0.208% value change).

WARNING

When using a model, make sure to load the model in the format it's intended for. Loading a model into the wrong numerical format can cause the model to change significantly. For example, Llama 2 had its weights set to BF16 when it came out. However, many teams loaded the model in FP16 and were subsequently frustrated to find the model's quality much worse than advertised.¹⁵ While this misunderstanding wasted a lot of people's time, the upside is that it forced many people to learn about numerical representations.

The right format for you depends on the distribution of numerical values of your workload (such as the range of values you need), how sensitive your workload is to small numerical changes, and the underlying hardware.¹⁶

Quantization

The fewer bits needed to represent a model's values, the lower the model's memory footprint will be. A 10B-parameter model in a 32-bit format requires 40 GB for its weights, but the same model in a 16-bit format will require only 20 GB. Reducing precision, also known as quantization, is a cheap and extremely effective way to reduce a model's memory footprint. It's straightforward to do and generalizes over tasks and architectures. In

the context of ML, low precision generally refers to any format with fewer bits than the standard FP32.

QUANTIZATION VERSUS REDUCED PRECISION

Strictly speaking, it's quantization only if the target format is integer. However, in practice, quantization is used to refer to all techniques that convert values to a lower-precision format. In this book, I use quantization to refer to precision reduction, to keep it consistent with the literature.

To do quantization, you need to decide what to quantize and when:

What to quantize

Ideally, you want to quantize whatever is consuming most of your memory, but it also depends on what you can quantize without hurting performance too much. As discussed in [“Memory Math”](#), major contributors to a model's memory footprint during inference are the model's weights and activations.¹⁷ Weight quantization is more common than activation quantization, since weight activation tends to have a more stable impact on performance with less accuracy loss.

When to quantize

Quantization can happen during training or post-training. Post-training quantization (PTQ) means quantizing a model after it's been fully trained. PTQ is by far the most common. It's also more relevant to AI application developers who don't usually train models.

Inference quantization

In the early days of deep learning, it was standard to train and serve models using 32 bits with FP32. Since the late 2010s, it has become increasingly common to serve models in 16 bits and in even lower precision. For example, [Dettmers et al. \(2022\)](#) have done excellent work quantizing LLMs into 8 bits with LLM.int8() and 4 bits with QLoRA ([Dettmers et al., 2023](#)).

A model can also be served in *mixed precision*, where values are reduced in precision when possible and maintained in higher precision when necessary. To serve models on the devices, [Apple](#) (2024) leveraged a quantization scheme that uses a mixture of 2-bit and 4-bit formats, averaging 3.5 bits-per-weight. Also in 2024, in anticipation of 4-bit neural networks, NVIDIA announced their new GPU architecture, [Blackwell](#), that supports model inference in 4-bit float.

Once you get to 8 bits and under, numerical representations get more tricky. You can keep parameter values as floats using one of the [minifloat](#) formats, such as FP8 (8 bits) and FP4 (4 bits).¹⁸ More commonly, however,

parameter values are converted into an integer format, such as INT8 or INT4.

Quantization is effective, but there's a limit to how far it can go. You can't have fewer than 1 bit per value, and some have attempted the 1-bit representation, e.g., BinaryConnect ([Courbariaux et al., 2015](#)), Xnor-Net ([Rastegari et al., 2016](#)), and BitNet ([Wang et al., 2023](#)).¹⁹

In 2024, Microsoft researchers ([Ma et al.](#)) declared that we're entering the era of 1-bit LLMs by introducing BitNet b1.58, a transformer-based language model that requires only 1.58 bits per parameter and whose performance is comparable to 16-bit Llama 2 ([Touvron et al., 2023](#)) up to 3.9B parameters, as shown in [Table 7-4](#).

Table 7-4. BitNet b1.58's performance compared to that of Llama 2 16-bit on different benchmarks ar

Model	Size	ARCe	ARCc	HS
Llama LLM	700M	54.7	23.0	37.0
BitNet b1.58	700M	51.8	21.4	35.1
Llama LLM	1.3B	56.9	23.5	38.5
BitNet b1.58	1.3B	54.9	24.2	37.7
Llama LLM	3B	62.1	25.6	43.3
BitNet b1.58	3B	61.4	28.3	42.9
BitNet b1.58	3.9B	64.2	28.7	44.2

Reduced precision not only reduces the memory footprint but also often improves computation speed. First, it allows a larger batch size, enabling the model to process more inputs in parallel. Second, reduced precision speeds up computation, which further reduces inference latency and training time. To illustrate this, consider the addition of two numbers. If we perform the addition bit by bit, and each takes t nanoseconds, it'll take $32t$ nanoseconds for 32 bits but only $16t$ nanoseconds for 16 bits. However, reducing precision doesn't always reduce latency due to the added computation needed for format conversion.

There are downsides to reduced precision. Each conversion often causes a small value change, and many small changes can cause a big performance change. If a value is outside the range the reduced precision format can represent, it might be converted to infinity or an arbitrary value, causing the model's quality to further degrade. How to reduce precision with minimal impact on model performance is an active area of research, pursued by model developers as well as by hardware makers and application developers.

Inference in lower precision has become a standard. A model is trained using a higher-precision format to maximize performance, then its precision is reduced for inference. Major ML frameworks, including PyTorch, TensorFlow, and Hugging Face's transformers, offer PTQ for free with a few lines of code.

Some edge devices only support quantized inference. Therefore, frameworks for on-device inference, such as TensorFlow Lite and PyTorch Mobile, also offer PTQ.

Training quantization

Quantization during training is not yet as common as PTQ, but it's gaining traction. There are two distinct goals for training quantization:

1. To produce a model that can perform well in low precision during inference. This is to address the challenge that a model's quality might

degrade during post-training quantization.

2. To reduce training time and cost. Quantization reduces a model's memory footprint, allowing a model to be trained on cheaper hardware or allowing the training of a larger model on the same hardware.

Quantization also speeds up computation, which further reduces costs.

A quantization technique might help achieve one or both of these goals.

Quantization-aware training (QAT) aims to create a model with high quality in low precision for inference. With QAT, the model simulates low-precision (e.g., 8-bit) behavior during training, which allows the model to learn to produce high-quality outputs in low precision. However, QAT doesn't reduce a model's training time since its computations are still performed in high precision. QAT can even increase training time due to the extra work of simulating low-precision behavior.

On the other hand, training a model directly in lower precision can help with both goals. People attempted to train models in reduced precision as early as 2016; see [Hubara et al. \(2016\)](#) and [Jacob et al. \(2017\)](#). [Character.AI \(2024\)](#) shared that they were able to train their models entirely in INT8, which helped eliminate the training/serving precision mismatch while also significantly improving training efficiency. However, training in lower precision is harder to do, as backpropagation is more sensitive to lower precision.

Lower-precision training is often done in [mixed precision](#), where a copy of the weights is kept in higher precision but other values, such as gradients and activations, are kept in lower precision.²¹ You can also have less-sensitive weight values computed in lower precision and more-sensitive weight values computed in higher precision. For example, LLM-QAT ([Liu et al., 2023](#)) quantizes weights and activations into 4 bits but keeps embeddings in 16 bits.

The portions of the model that should be in lower precision can be set automatically using the [automatic mixed precision](#) (AMP) functionality offered by many ML frameworks.

It's also possible to have different phases of training in different precision levels. For example, a model can be trained in higher precision but finetuned in lower precision. This is especially common with foundation models, where the team training a model from scratch might be an organization with sufficient compute for higher precision training. Once the model is published, developers with less compute access can finetune that model in lower precision.

Finetuning Techniques

I hope that the previous section has made clear why finetuning large-scale models is so memory-intensive. The more memory finetuning requires, the

fewer people who can afford to do it. Techniques that reduce a model's memory footprint make finetuning more accessible, allowing more people to adapt models to their applications. This section focuses on memory-efficient finetuning techniques, which centers around parameter-efficient finetuning.

I'll also cover model merging, an exciting but more experimental approach to creating custom models. While model merging is generally not considered finetuning, I include it in this section because it's complementary to finetuning. Finetuning tailors one model to specific needs. Model merging combines multiple models, often finetuned models, for the same purpose.

While combining multiple models isn't a new concept, new types of models and finetuning techniques have inspired many creative model-merging techniques, making this section especially fun to write about.

Parameter-Efficient Finetuning

In the early days of finetuning, models were small enough that people could finetune entire models. This approach is called *full finetuning*. In full finetuning, the number of trainable parameters is exactly the same as the number of parameters.

Full finetuning can look similar to training. The main difference is that training starts with randomized model weights, whereas finetuning starts

with model weights that have been previously trained.

As discussed in [“Memory Math”](#), the more trainable parameters there are, the more memory is needed. Consider a 7B-parameter model:

- If you use a 16-bit format like FP16, loading the model’s weights alone requires 14 GB for memory.
- Full finetuning this model with the Adam optimizer, also in a 16-bit format, requires an additional $7\text{B} \times 3 \times 2 \text{ bytes} = 42 \text{ GB}$ of memory.
- The total memory needed for the model’s weights, gradients, and optimizer states is then $14 \text{ GB} + 42 \text{ GB} = 56 \text{ GB}$.

56 GB exceeds the memory capacity of most consumer GPUs, which typically come with 12–24 GB of memory, with higher-end GPUs offering up to 48 GB. And this memory estimation doesn’t yet take into account the memory required for activations.

NOTE

To fit a model on a given hardware, you can either reduce the model’s memory footprint or find ways to use the hardware’s memory more efficiently. Techniques like quantization and PEFT help minimize the total memory footprint. Techniques that focus on making better use of hardware memory include *CPU offloading*. Instead of trying to fit the whole model on GPUs, you can offload the excess memory onto CPUs, as demonstrated by DeepSpeed ([Rasley et al., 2020](#)).

We also haven't touched on the fact that full finetuning, especially supervised finetuning and preference finetuning, typically requires a lot of high-quality annotated data that most people can't afford. Due to the high memory and data requirements of full finetuning, people started doing *partial finetuning*. In partial finetuning, only some of the model's parameters are updated. For example, if a model has ten layers, you might freeze the first nine layers and finetune only the last layer,²² reducing the number of trainable parameters to 10% of full finetuning.

While partial finetuning can reduce the memory footprint, it's *parameter-inefficient*. Partial finetuning requires many trainable parameters to achieve performance close to that of full finetuning. A study by [Houlsby et al. \(2019\)](#) shows that with BERT large ([Devlin et al., 2018](#)), you'd need to update approximately 25% of the parameters to achieve performance comparable to that of full finetuning on the GLUE benchmark ([Wang et al., 2018](#)). [Figure 7-7](#) shows the performance curve of partial finetuning with different numbers of trainable parameters.

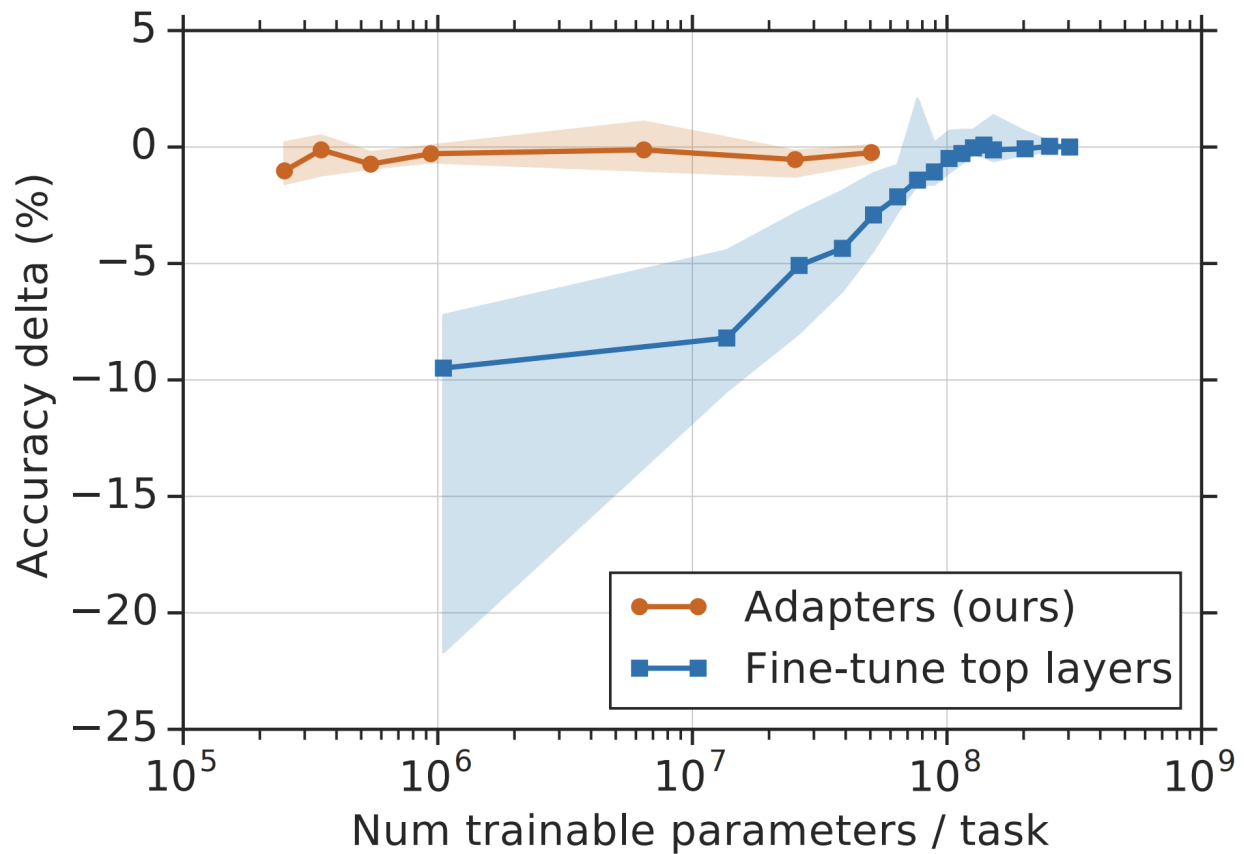


Figure 7-7. The blue line shows that partial finetuning requires many trainable parameters to achieve a performance comparable to full finetuning. Image from Houlsby et al. (2019).

This brings up the question: How to achieve performance close to that of full finetuning while using significantly fewer trainable parameters? Finetuning techniques resulting from this quest are parameter-efficient. There's no clear threshold that a finetuning method has to pass to be considered parameter-efficient. However, in general, a technique is considered parameter-efficient if it can achieve performance close to that of full finetuning while using several orders of magnitude fewer trainable parameters.

The idea of PEFT (parameter-efficient finetuning) was introduced by Houlsby et al. (2019). The authors showed that by inserting additional parameters into the model in the right places, you can achieve strong finetuning performance using a small number of trainable parameters. They inserted two adapter modules into each transformer block of a BERT model, as shown in [Figure 7-8](#).

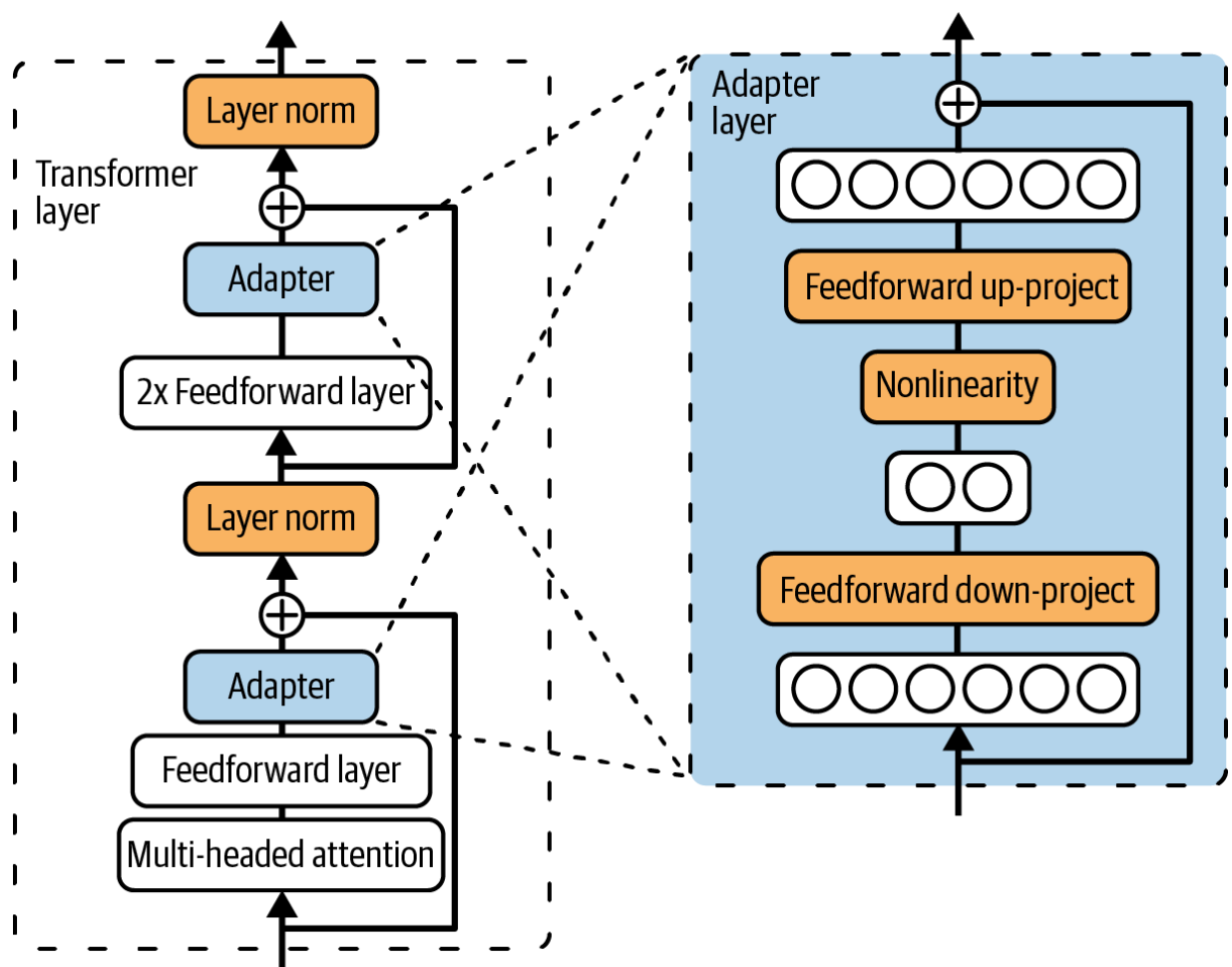


Figure 7-8. By inserting two adapter modules into each transformer layer for a BERT model and updating only the adapters, Houlsby et al. (2019) were able to achieve strong finetuning performance using a small number of trainable parameters.

During finetuning, they kept the model's original parameters unchanged and only updated the adapters. The number of trainable parameters is the number of parameters in the adapters. On the GLUE benchmark, they achieved a performance within 0.4% of full finetuning using only 3% of the number of trainable parameters. The orange line in [Figure 7-7](#) shows the performance delta between full finetuning and finetuning using different adapter sizes.

However, the downside of this approach is that it increases the inference latency of the finetuned model. The adapters introduce additional layers, which add more computational steps to the forward pass, slowing inference.

PEFT enables finetuning on more affordable hardware, making it accessible to many more developers. PEFT methods are generally not only parameter-efficient but also sample-efficient. While full finetuning may need tens of thousands to millions of examples to achieve notable quality improvements, some PEFT methods can deliver strong performance with just a few thousand examples.

Given PEFT's obvious appeal, PEFT techniques are being rapidly developed. The next section will give an overview of these techniques before diving deeper into the most common PEFT technique: LoRA.

PEFT techniques

The existing prolific world of PEFT generally falls into two buckets: *adapter-based methods* and *soft prompt-based methods*. However, it's likely that newer buckets will be introduced in the future.

Adapter-based methods refer to all methods that involve additional modules to the model weights, such as the one developed by [Houlsby et al. \(2019\)](#). Because adapter-based methods involve adding parameters, they are also called *additive methods*.

As of this writing, LoRA ([Hu et al., 2021](#)) is by far the most popular adapter-based method, and it will be the topic of the following section. Other adapter-based methods include BitFit ([Zaken et al., 2021](#)), which came out around the same time LoRA did. Newer adapter methods include IA3 ([Liu et al., 2022](#)), whose efficient mixed-task batching strategy makes it particularly attractive for multi-task finetuning. It's been shown to outperform LoRA and even full finetuning in some cases. LongLoRA ([Chen et al., 2023](#)) is a LoRA variant that incorporates attention-modification techniques to expand context length.

If adapter-based methods add trainable parameters to the model's architecture, soft prompt-based methods modify how the model processes the input by introducing special trainable tokens. These additional tokens are fed into the model alongside the input tokens. They are called *soft*

prompts because, like the inputs (hard prompts), soft prompts also guide the model's behaviors. However, soft prompts differ from hard prompts in two ways:

- Hard prompts are human-readable. They typically contain *discrete* tokens such as “I”, “write”, “a”, and “lot”. In contrast, soft prompts are continuous vectors, resembling embedding vectors, and are not human-readable.
- Hard prompts are static and not trainable, whereas soft prompts can be optimized through backpropagation during the tuning process, allowing them to be adjusted for specific tasks.

Some people describe soft prompting as a crossover between prompt engineering and finetuning. [Figure 7-9](#) visualizes how you can use soft prompts together with hard prompts to guide a model's behaviors.

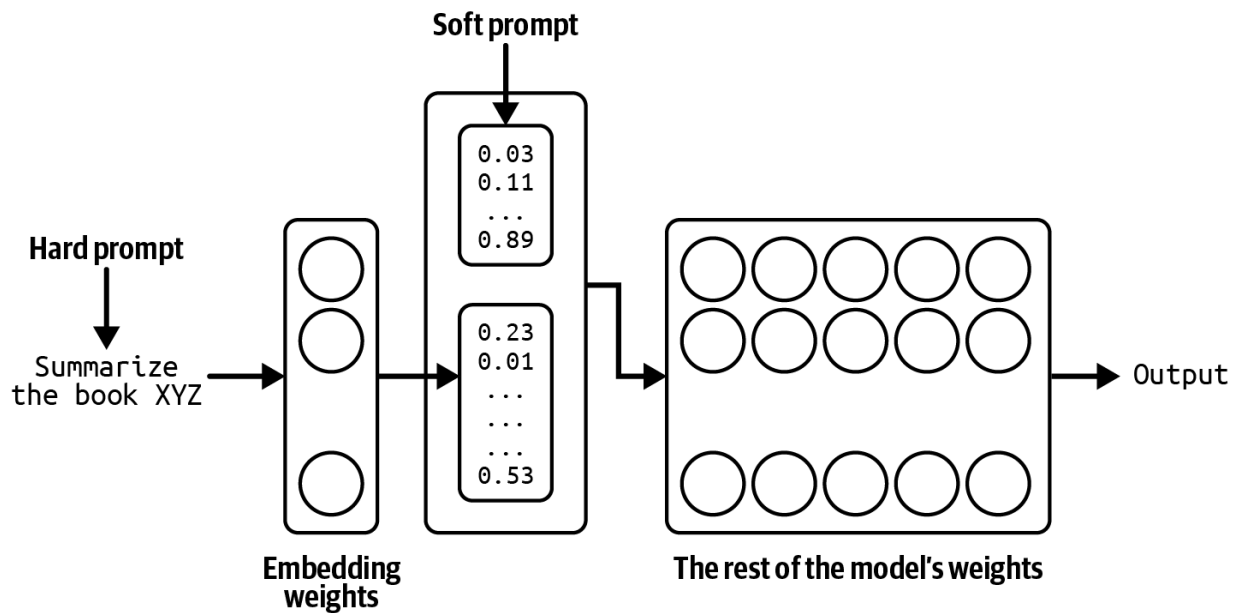


Figure 7-9. Hard prompts and soft prompts can be combined to change a model's behaviors.

Soft prompt tuning as a subfield is characterized by a series of similar-sounding techniques that can be confusing, such as prefix-tuning ([Li and Liang, 2021](#)), P-Tuning ([Liu et al., 2021](#)), and prompt tuning ([Lester et al., 2021](#)).²³ They differ mainly on the locations where the soft prompts are inserted. For example, prefix tuning prepends soft prompt tokens to the input at every transformer layer, whereas prompt tuning prepends soft prompt tokens to only the embedded input. If you want to use any of them, many PEFT frameworks will implement them out of the box for you.

To get a sense of what PEFT methods are being used, I analyzed over 1,000 open issues on the [GitHub repository huggingface/peft](#) in October 2024. The assumption is that if someone uses a technique, they are more likely to report issues or ask questions about it. [Figure 7-10](#) shows the result. For “P-

Tuning”, I searched for keywords “p_tuning” and “p tuning” to account for different spellings.

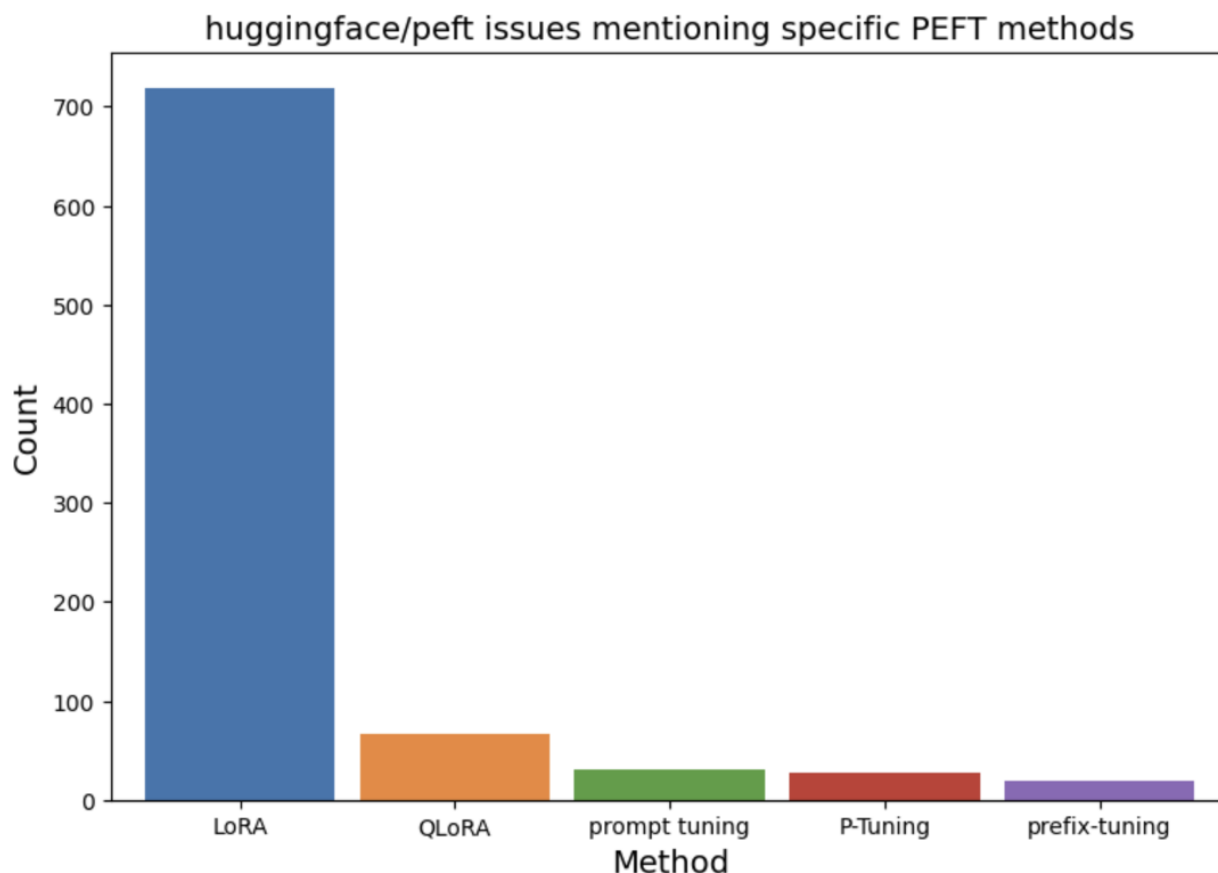


Figure 7-10. The number of issues corresponding to different finetuning techniques from the GitHub repository huggingface/peft. This is a proxy to estimate the popularity of each technique.

From this analysis, it’s clear that LoRA dominates. Soft prompts are less common, but there seems to be growing interest from those who want more customization than what is afforded by prompt engineering but who don’t want to invest in finetuning.

Because of LoRA’s popularity, the next section focuses on how LoRA works and how it solves the challenge posed by early adapter-based

methods. Even if you don't use LoRA, this deep dive should provide a framework for you to explore other finetuning methods.

LoRA

Unlike the original adapter method by [Houlsby et al. \(2019\)](#), LoRA (Low-Rank Adaptation) ([Hu et al., 2021](#)) incorporates additional parameters in a way that doesn't incur extra inference latency. Instead of introducing additional layers to the base model, LoRA uses modules that can be merged back to the original layers.

You can apply LoRA to individual weight matrices. Given a weight matrix, LoRA decomposes this matrix into the product of two smaller matrices, then updates these two smaller matrices before merging them back to the original matrix.

Consider the weight matrix W of the dimension $n \times m$. LoRA works as follows:

1. First, choose the dimension of the smaller matrices. Let r be the chosen value. Construct two matrices: A (dimension $n \times r$) and B (dimension $r \times m$). Their product is W_{AB} , which is of the same dimension as W . r is the LoRA *rank*.
2. Add W_{AB} to the original weight matrix W to create a new weight matrix W' . Use W' in place of W as part of the model. You can use a

hyperparameter α to determine how much W_{AB} should contribute to the new matrix: $W\hat{a} = W + \frac{\alpha}{r} W_{AB}$

3. During finetuning, update only the parameters in A and B . W is kept intact.

[Figure 7-11](#) visualizes this process.

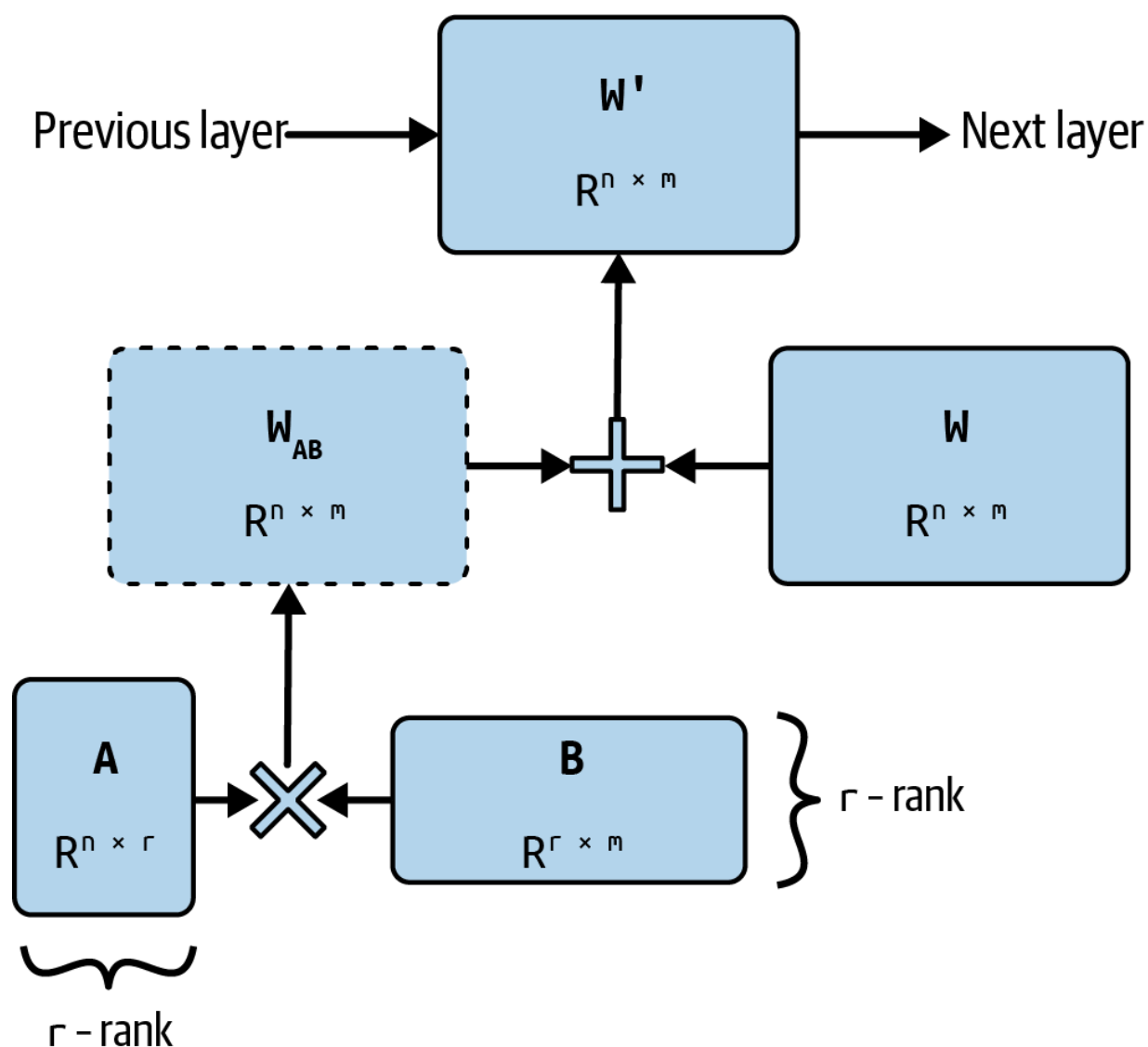


Figure 7-11. To apply LoRA to a weight matrix W , decompose it into the product of two matrices A and B . During finetuning, only A and B are updated. W is kept intact.

NOTE

LoRA (Low-Rank Adaptation) is built on the concept of *low-rank factorization*, a long-standing dimensionality reduction technique. The key idea is that you can factorize a large matrix into a product of two smaller matrices to reduce the number of parameters, which, in turn, reduces both the computation and memory requirements. For example, a 9×9 matrix can be factorized into the product of two matrices of dimensions 9×1 and 1×9 . The original matrix has 81 parameters, but the two product matrices have only 18 parameters combined.

The number of columns in the first factorized matrix and the number of columns in the second factorized matrix correspond to the rank of the factorization. The original matrix is *full-rank*, while the two smaller matrices represent a low-rank approximation.

While factorization can significantly reduce the number of parameters, it's lossy because it only approximates the original matrix. The higher the rank, the more information from the original matrix the factorization can preserve.

Like the original adapter method, LoRA is parameter-efficient and sample-efficient. The factorization enables LoRA to use even fewer trainable parameters. The LoRA paper showed that, for GPT-3, LoRA achieves comparable or better performance with full finetuning on several tasks while using only ~4.7M trainable parameters, 0.0027% of full finetuning.

Why does LoRA work?

Parameter-efficient methods like LoRA have become so popular that many people take them for granted. *But why is parameter efficiency possible at all?* If a model requires a lot of parameters to learn certain behaviors during

pre-training, shouldn't it also require a lot of parameters to change its behaviors during finetuning?

The same question can be raised for data. If a model requires a lot of data to learn a behavior, shouldn't it also require a lot of data to meaningfully change this behavior? How is it possible that you need millions or billions of examples to pre-train a model, but only a few hundreds or thousands of examples to finetune it?

Many papers have argued that while LLMs have many parameters, they have very low intrinsic dimensions; see [Li et al. \(2018\)](#); [Aghajanyan et al. \(2020\)](#); and [Hu et al. \(2021\)](#). They showed that *pre-training implicitly minimizes the model's intrinsic dimension*. Surprisingly, larger models tend to have lower intrinsic dimensions after pre-training. This suggests that pre-training acts as a compression framework for downstream tasks. In other words, the better trained an LLM is, the easier it is to finetune the model using a small number of trainable parameters and a small amount of data.

You might wonder, if low-rank factorization works so well, *why don't we use LoRA for pre-training as well?* Instead of pre-training a large model and applying low-rank factorization only during finetuning, could we factorize a model from the start for pre-training? Low-rank pre-training can significantly reduce the model's number of parameters, significantly reducing the model's pre-training time and cost.

Throughout the 2010s, many people tried training low-rank neural networks, exemplified in studies such as “Low-Rank Matrix Factorization for Deep Neural Network Training with High-Dimensional Output Targets” ([Sainath et al., 2013](#)), “Semi-Orthogonal Low-Rank Matrix Factorization for Deep Neural Networks” ([Povey et al., 2018](#)), and “Speeding up Convolutional Neural Networks with Low Rank Expansions” ([Jaderberg et al., 2014](#)).

Low-rank factorization has proven to be effective at smaller scales. For example, by applying various factorization strategies, including replacing 3×3 convolution with 1×1 convolution, SqueezeNet ([Iandola et al., 2016](#)) achieves AlexNet-level accuracy on ImageNet using 50 times fewer parameters.

More recent attempts to train low-rank LLMs include ReLoRA ([Lialin et al., 2023](#)) and GaLore ([Zhao et al., 2024](#)). ReLoRA works for transformer-based models of up to 1.3B parameters. GaLore achieves performance comparable to that of a full-rank model at 1B parameters and promising performance at 7B parameters.

It’s possible that one day not too far in the future, researchers will develop a way to scale up low-rank pre-training to hundreds of billions of parameters. However, if [Aghajanyan et al.’s argument](#) is correct—that pre-training implicitly compresses a model’s intrinsic dimension—full-rank pre-training is still necessary to sufficiently reduce the model’s intrinsic dimension to a

point where low-rank factorization can work. It would be interesting to study exactly how much full-rank training is necessary before it's possible to switch to low-rank training.

LoRA configurations

To apply LoRA, you need to decide what weight matrices to apply LoRA to and the rank of each factorization. This section will discuss the considerations for each of these decisions.

LoRA can be applied to each individual weight matrix. The efficiency of LoRA, therefore, depends not only on what matrices LoRA is applied to but also on the model's architecture, as different architectures have different weight matrices.

While there have been examples of LoRA with other architectures, such as convolutional neural networks ([Dutt et al., 2023](#); [Zhong et al., 2024](#); [Aleem et al., 2024](#)), LoRA has been primarily used for transformer models.²⁴

LoRA is most commonly applied to the four weight matrices in the attention modules: the query (W_q), key (W_k), value (W_v), and output projection (W_o) matrices.

Typically, LoRA is applied uniformly to all matrices of the same type within a model. For example, applying LoRA to the query matrix means applying LoRA to all query matrices in the model.

Naively, you can apply LoRA to all these attention matrices. However, often, you're constrained by your hardware's memory and can accommodate only a fixed number of trainable parameters. Given a fixed budget of trainable parameters, what matrices should you apply LoRA to, to maximize performance?

When finetuning GPT-3 175B, Hu et al. (2021) set their trainable parameter budget at 18M, which is 0.01% of the model's total number of parameters. This budget allows them to apply LoRA to the following:

1. One matrix with the rank of 8
2. Two matrices with the rank of 4
3. All four matrices with the rank of 2

NOTE

GPT-3 175B has 96 transformer layers with a model dimension of 12,288. Applying LoRA with rank = 2 to all four matrices would yield $(12,288 \times 2 \times 2) \times 4 = 196,608$ trainable parameters per layer, or 18,874,368 trainable parameters for the whole model.

They found that applying LoRA to all four matrices with rank = 2 yields the best performance on the WikiSQL ([Zhong et al., 2017](#)) and MultiNLI (Multi-Genre Natural Language Inference) benchmarks ([Williams et al., 2017](#)). [Table 7-5](#) shows their results. However, the authors suggested that if you can choose only two attention matrices, the query and value matrices generally yield the best results.

Table 7-5. LoRA performance with the budget of 18M trainable parameters. Results from LoRA (Hu et al., 2022)

Number of trainable parameters = 18M				
Weight type	W_q	W_k	W_v	W_o
Rank r	8	8	8	8
WikiSQL ($\pm$ 0.5%)	70.4	70.0	73.0	73.2
MultiNLI ($\pm$ 0.1%)	91.0	90.8	91.0	91.3

Empirical observations suggest that applying LoRA to more weight matrices, including the feedforward matrices, yields better results. For example, Databricks showed that the biggest performance boost they got was from applying LoRA to all feedforward layers ([Sooriyarachchi, 2023](#)). [Fomenko et al. \(2024\)](#) noted that feedforward-based LoRA can be complementary to attention-based LoRA, though attention-based LoRA typically offers greater efficacy within memory constraints.

The beauty of LoRA is that while its performance depends on its rank, studies have shown that *a small r , such as between 4 and 64, is usually sufficient for many use cases*. A smaller r means fewer LoRA parameters, which translates to a lower memory footprint.

The LoRA authors observed that, to their surprise, increasing the value of r doesn't increase finetuning performance. This observation is consistent with Databricks' report that "increasing r beyond a certain value may not yield any discernible increase in quality of model output" (Sooriyarachchi, 2023).²⁵ Some argue that a higher r might even hurt as it can lead to overfitting. However, in some cases, a higher rank might be necessary. [Raschka \(2023\)](#) found that $r = 256$ achieved the best performance on his tasks.

Another LoRA hyperparameter you can configure is the value α that determines how much the product W_{AB} should contribute to the new matrix during merging: $W\hat{a} = W + \frac{\alpha}{r} W_{AB}$. In practice, I've often seen α chosen so that the ratio $\alpha:r$ is typically between 1:8 and 8:1, but the optimal ratio varies. For example, if r is small, you might want α to be larger, and if r is large, you might want α to be smaller. Experimentation is needed to determine the best (r, α) combination for your use case.

Serving LoRA adapters

LoRA not only lets you finetune models using less memory and data, but it also simplifies serving multiple models due to its modularity. To understand this benefit, let's examine how to serve a LoRA-finetuned model.

In general, there are two ways to serve a LoRA-finetuned model:

1. Merge the LoRA weights A and B into the original model to create the new matrix W' prior to serving the finetuned model. Since no extra computation is done during inference, no extra latency is added.
2. Keep W , A , and B separate during serving. The process of merging A and B back to W happens during inference, which adds extra latency.

The first option is generally better if you have only one LoRA model to serve, whereas the second is generally better for *multi-LoRA serving*—serving multiple LoRA models that share the same base model. [Figure 7-12](#) visualizes multi-LoRA serving if you keep the LoRA adapters separate.

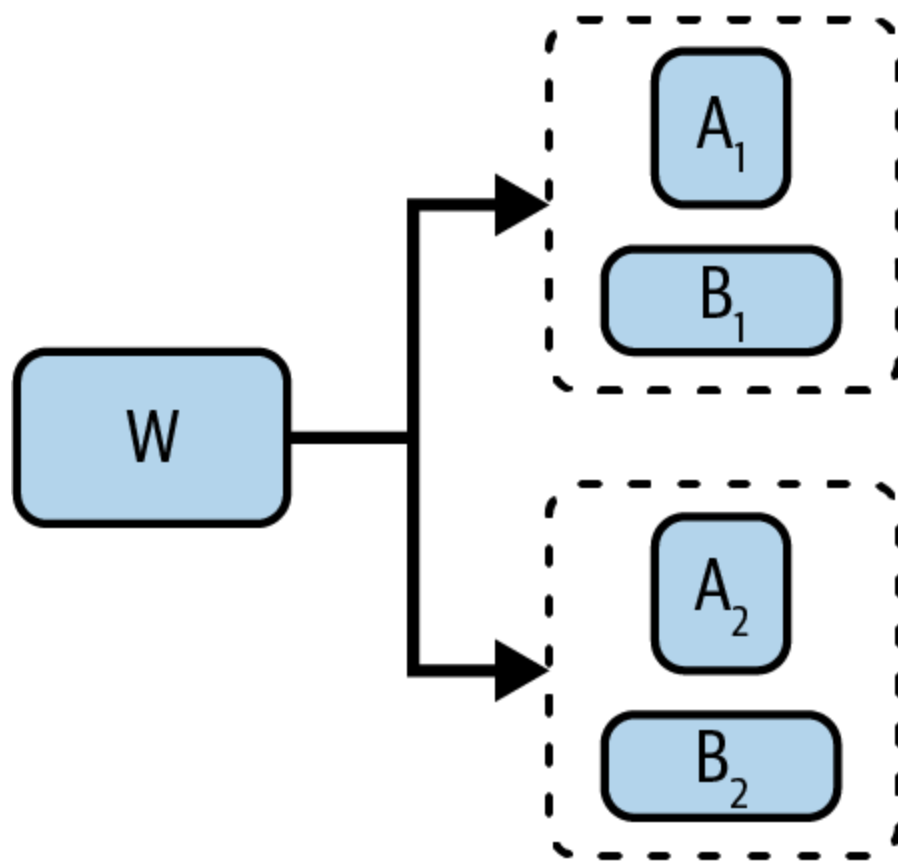


Figure 7-12. Keeping LoRA adapters separate allows reuse of the same full-rank matrix W in multi-LoRA serving.

For multi-LoRA serving, while option 2 adds latency overhead, it significantly reduces the storage needed. Consider the scenario in which you finetune a model for each of your customers using LoRA. With 100 customers, you end up with 100 finetuned models, all sharing the same base model. With option 1, you have to store 100 full-rank matrices W' . With option 2, you only have to store one full-rank matrix W , and 100 sets of smaller matrices (A, B) .

To put this in perspective, let's say that the original matrix W is of the dimension 4096×4096 (16.8M parameters). If the LoRA's rank is 8, the number of parameters in A and B is $4096 \times 8 \times 2 = 65,536$:

- In option 1, 100 full-rank matrices W' totals $16.8\text{M} \times 100 = 1.68\text{B}$ parameters.
- In option 2, one full-rank matrix W and 100 sets of small matrices (A, B) totals: $16.8\text{M} + 65,536 \times 100 = 23.3\text{M}$ parameters.

Option 2 also makes it faster to switch between tasks. Let's say you're currently serving customer X using this customer's model. To switch to serving customer Y , instead of loading this customer's full weight matrix, you only need to load Y 's LoRA adapter, which can significantly reduce the loading time. While keeping A and B separate incurs additional latency, there are optimization techniques to minimize the added latency. The [book's GitHub repository](#) contains a walkthrough of how to do so.

Multi-LoRA serving makes it easy to combine multiple specialized models. Instead of having one big powerful model for multiple tasks, you can have one LoRA adapter for each task. For example, Apple used multiple [LoRA adapters](#) to adapt the same 3B-parameter base model to different iPhone features (2024). They utilized quantization techniques to further reduce the memory footprint of this base model and adapters, allowing the serving of all of them on-device.

The modularity of LoRA adapters means that LoRA adapters can be shared and reused. There are publicly available finetuned LoRA adapters that you can use the way you'd use pre-trained models. You can find them on [Hugging Face](#)²⁶ or initiatives like [AdapterHub](#).

You might be wondering: “LoRA sounds great, but what’s the catch?” The main drawback of LoRA is that it doesn’t offer performance as strong as full finetuning. It’s also more challenging to do than full finetuning as it involves modifying the model’s implementation, which requires an understanding of the model’s architecture and coding skills. However, this is usually only an issue for less popular base models. PEFT frameworks—such as [Hugging Face’s PEFT](#), [Axolotl](#), [unsloth](#), and [LitGPT](#)—likely support LoRA for popular base models right out of the box.

Quantized LoRA

The rapid rise of LoRA has led to the development of numerous LoRA variations. Some aim to reduce the number of trainable parameters even further. However, as illustrated in [Table 7-6](#), the memory of a LoRA adapter is minimal compared to the memory of the model’s weights. Reducing the number of LoRA parameters decreases the overall memory footprint by only a small percentage.

Table 7-6. The memory needed by LoRA weights compared to that needed by the model’s weights.

	Model’s weights memory (16 bits)	LoRA trainable params (r=2, query & key matrices)	LoRA adapter memory (16 bits)
Llama 2 (13B)	26 GB	3.28M	6.55 MB
GPT-3 (175B)	350 GB	18.87M	37.7 MB

Rather than trying to reduce LoRA’s number of parameters, you can reduce the memory usage more effectively by quantizing the model’s weights, activations, and/or gradients during finetuning. An early promising quantized version of LoRA is QLoRA ([Dettmers et al., 2023](#)).²⁷ In the original LoRA paper, during finetuning, the model’s weights are stored using 16 bits. QLoRA stores the model’s weights in 4 bits but dequantizes

(converts) them back into BF16 when computing the forward and backward pass.

The 4-bit format that QLoRA uses is NF4 (NormalFloat-4), which quantizes values based on the insight that pre-trained weights usually follow a normal distribution with a median of zero. On top of 4-bit quantization, QLoRA also uses paged optimizers to automatically transfer data between the CPU and GPU when the GPU runs out of memory, especially with long sequence lengths. These techniques allow a 65B-parameter model to be finetuned on a single 48 GB GPU.

The authors finetuned a variety of models, including Llama 7B to 65B, in the 4-bit mode. The resulting family of models, called Guanaco, showed competitive performance on both public benchmarks and comparative evaluation. [Table 7-7](#) shows the Elo ratings of Guanaco models, GPT-4, and ChatGPT in May 2023, as judged by GPT-4. While Guanaco 65B didn't outperform GPT-4, it was often preferred to ChatGPT.

Table 7-7. Elo ratings of Guanaco models compared to popular models in May 2023 using GPT-4 as a judge. The experiment is from QLoRA (Dettmers et al., 2023).

Model	Size	Elo
GPT-4	-	1348 $\pm$ 1
Guanaco 65B	41 GB	1022 $\pm$ 1
Guanaco 33B	21 GB	992 $\pm$ 1
Vicuna 13B	26 GB	974 $\pm$ 1
ChatGPT	-	966 $\pm$ 1
Guanaco 13B	10 GB	916 $\pm$ 1
Bard	-	902 $\pm$ 1
Guanaco 7B	6 GB	879 $\pm$ 1

The main limitation of QLoRA is that NF4 quantization is expensive. While QLoRA can reduce the memory footprint, it might increase training time due to the extra time required by quantization and dequantization steps.

Due to its memory-saving promise, quantized LoRA is an active area of research. Other than QLoRA, quantized LoRA works include QA-LoRA ([Xu et al., 2023](#)), ModuLoRA ([Yin et al., 2023](#)), and IR-QLoRA ([Qin et al., 2024](#)).

Model Merging and Multi-Task Finetuning

If finetuning allows you to create a custom model by altering a single model, model merging allows you to create a custom model by combining multiple models. Model merging offers you greater flexibility than finetuning alone. You can take two available models and merge them together to create a new, hopefully more useful, model. You can also finetune any or all of the constituent models before merging them together.

While you don't have to further finetune the merged model, its performance can often be improved by finetuning. Without finetuning, model merging can be done without GPUs, making merging particularly attractive to indie model developers that don't have access to a lot of compute.

The goal of model merging is to create a single model that provides more value than using all the constituent models separately. The added value can come from improved performance. For example, if you have two models that are good at different things on the same task, you can merge them into a single model that is better than both of them on that task. Imagine one model that can answer the first 60% of the questions and another model that can answer the last 60% of the questions. Combined, perhaps they can answer 80% of the questions.

The added value can also come from a reduced memory footprint, which leads to reduced costs. For example, if you have two models that can do

different tasks, they can be merged into one model that can do both tasks but with fewer parameters. This is particularly attractive for adapter-based models. Given two models that were finetuned on top of the same base model, you can combine their adapters into a single adapter.

One important use case of model merging is multi-task finetuning. Without model merging, if you want to finetune a model for multiple tasks, you generally have to follow one of these approaches:

Simultaneous finetuning

You create a dataset with examples for all the tasks and finetune the model on this dataset to make the model learn all the tasks simultaneously. However, because it's generally harder to learn multiple skills at the same time, this approach typically requires more data and more training.

Sequential finetuning

You can finetune the model on each task separately but sequentially. After training a model on task A, train it on task B, and so on. The assumption is that it's easier for models to learn one task at a time. Unfortunately, neural networks are prone to catastrophic forgetting ([Kirkpatrick et al., 2016](#)). A model can forget how to do an old task when it's trained on a new task, leading to a significant performance drop on earlier tasks.

Model merging offers another method for multi-task finetuning. You can finetune the model on different tasks separately but in parallel. Once done, these different models are merged together. Finetuning on each task separately allows the model to learn that task better. Because there's no sequential learning, there's less risk of catastrophic forgetting.

Model merging is also appealing when you have to deploy models to devices such as phones, laptops, cars, smartwatches, and warehouse robots. On-device deployment is often challenging because of limited on-device memory capacity. Instead of squeezing multiple models for different tasks onto a device, you can merge these models together into one model that can perform multiple tasks while requiring much less memory.

On-device deployment is necessary for use cases where data can't leave the device (often due to privacy), or where there's limited or unreliable internet access. On-device deployment can also significantly reduce inference costs. The more computation you can offload to user devices, the less you have to pay to data centers. ²⁸

Model merging is one way to do *federated learning* ([McMahan et al., 2016](#)), in which multiple devices train the same model using separate data. For example, if you deploy model X to multiple devices, each copy of X can continue learning separately from the on-device data. After a while, you have multiple copies of X, all trained on different data. You can merge these

copies together into one new base model that contains the learning of all constituent models.

The idea of combining models together to obtain better performance started with *model ensemble methods*. According to [Wikipedia](#), ensembling combines “multiple learning algorithms to obtain better predictive performance than could be obtained from any of the constituent learning algorithms alone.” If model merging typically involves mixing parameters of constituent models together, ensembling typically combines only model outputs while keeping each constituent model intact.

For example, in ensembling, given a query, you might use three models to generate three different answers. Then, a final answer is generated based on these three answers, using a simple majority vote or another trainable ML module.²⁹ While ensembling can generally improve performance, it has a higher inference cost since it requires multiple inference calls per request.

[Figure 7-13](#) compares ensembling and model merging. Just like model ensembles used to dominate leaderboards, many models on top of the [Hugging Face’s Open LLM Leaderboard](#) are merged models.

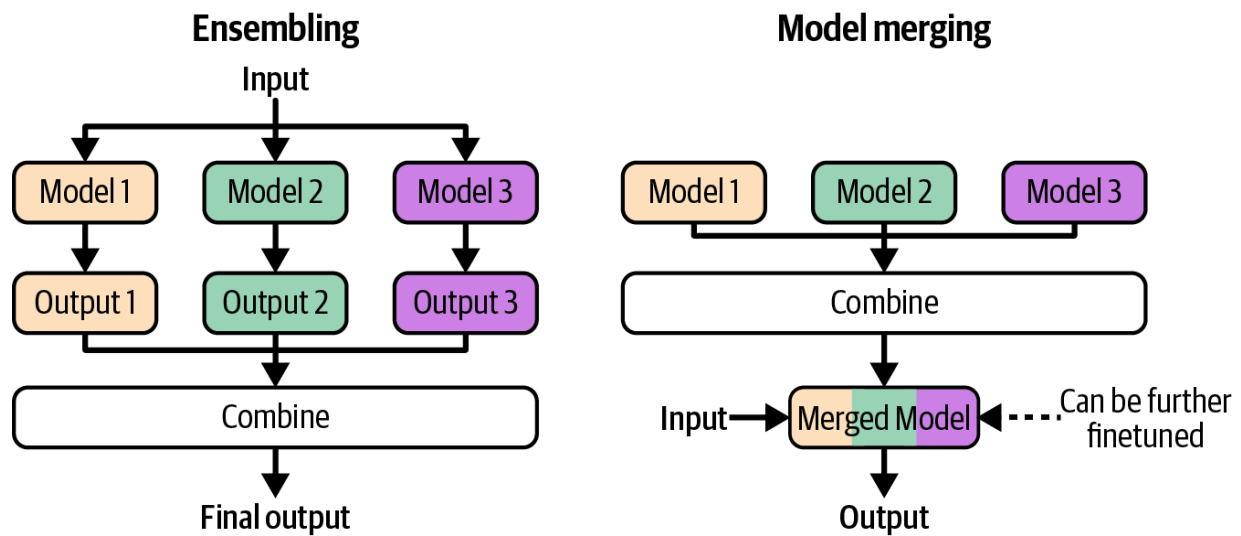


Figure 7-13. How ensembling and model merging work.

Many model-merging techniques are experimental and might become outdated as the community gains a better understanding of the underlying theory. For this reason, I'll focus on the high-level merging approaches instead of any individual technique.

Model merging approaches differ in how the constituent parameters are combined. Three approaches covered here are summing, layer stacking, and concatenation. [Figure 7-14](#) shows their high-level differences.

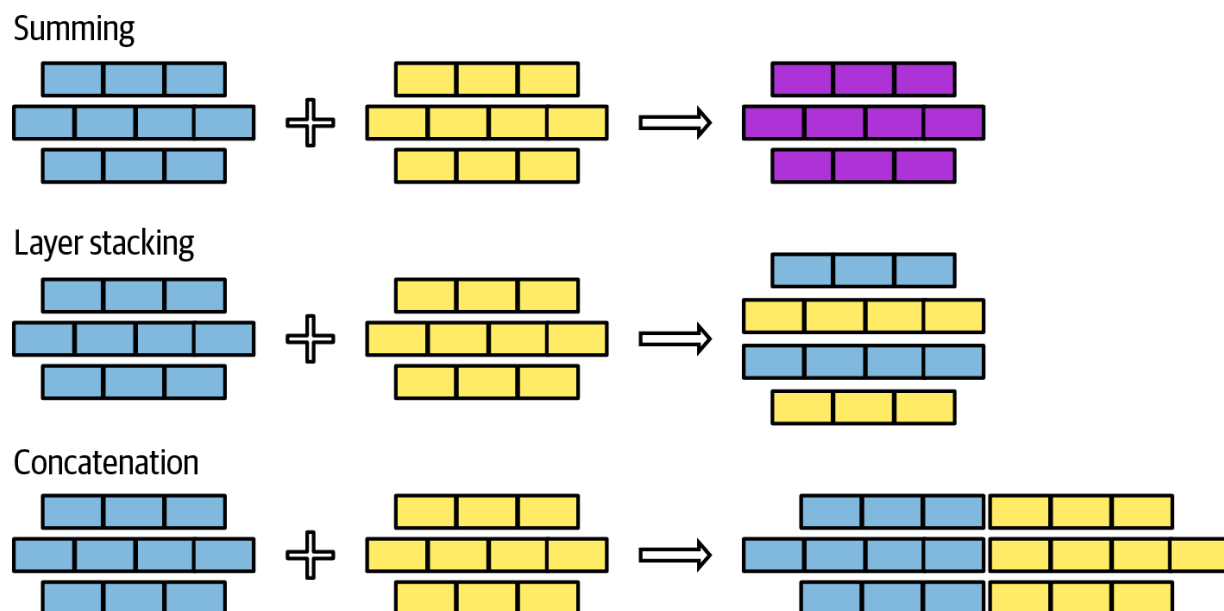


Figure 7-14. Three main approaches to model merging: summing, layer stacking, and concatenation.

You can mix these approaches when merging models, e.g., summing some layers and stacking others. Let's explore each of these approaches.

Summing

This approach involves adding the weight values of constituent models together. I'll discuss two summing methods: linear combination and spherical linear interpolation. If the parameters in two models are in different scales, e.g., one model's parameter values are much larger than the other's, you can rescale the models before summing so that their parameter values are in the same range.

Linear combination

Linear combination includes both an average and a weighted average.

Given two models, A and B, their weighted average is:

$$\text{Merge}(A, B) = \frac{W_A A + W_B B}{W_A + W_B}$$

[Figure 7-15](#) shows how to linearly combine two layers when $w_A = w_B = 1$.

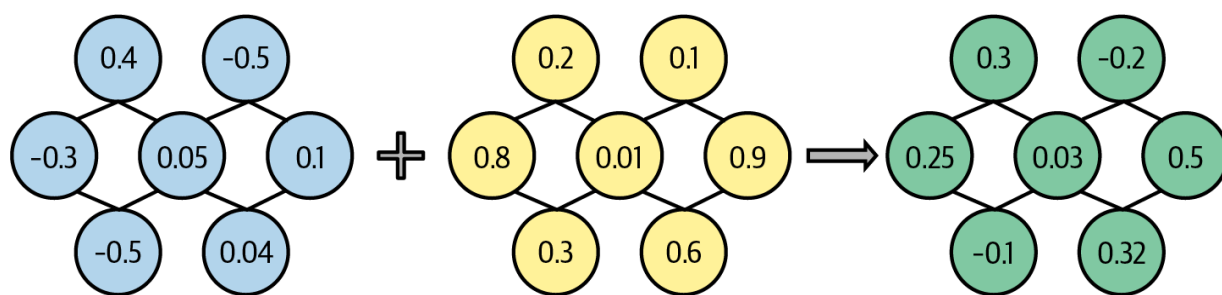


Figure 7-15. Merging parameters by averaging them.

Linear combination works surprisingly well, given how simple it is.³⁰ The idea that multiple models can be linearly combined to create a better one was studied as early as the early 1990s ([Perrone, 1993](#)). Linear combination is often used in federated learning ([Wang et al., 2020](#)).

You can linearly combine entire models or parts of models. Model soups ([Wortsman et al., 2022](#)) showed how averaging the entire weights of multiple finetuned models can improve accuracy without increasing inference time. However, it's more common to merge models by linearly combining specific components, such as their adapters.

While you can linearly combine any set of models, *linear combination is the most effective for models finetuned on top of the same base model*. In this case, linear combination can be viewed through the concept of *task vectors*. The idea is that once you've finetuned a model for a specific task, subtracting the base model from it should give you a vector that captures the essence of the task. Task vectors are also called *delta parameters*. If you finetune using LoRA, you can construct the task vector from the LoRA weights.

Task vectors allow us to do *task arithmetic* ([Ilharco et al., 2022](#)), such as adding two task vectors to combine task capabilities or subtracting a task vector to reduce specific capabilities. Task subtraction can be useful for removing undesirable model behaviors, such as invasive capabilities like facial recognition or biases obtained during pre-training.

Linear combination is straightforward when the components to be merged are of the same architecture and of the same size. However, it can also work for models that don't share the same architecture or the same size. For example, if one model's layer is larger than that of the other model, you can project one or both layers into the same dimension.

Some people proposed aligning models before averaging to ensure that functionally related parameters are averaged together, such as in "Model Fusion via Optimal Transport" ([Singh and Jaggi, 2020](#)), "Git Re-Basin: Merging Models Modulo Permutation Symmetries" ([Ainsworth et al.,](#)

[2022](#)), and “Merging by Matching Models in Task Parameter Subspaces” ([Tam et al., 2023](#)). While it makes sense to combine aligned parameters, aligning parameters can be challenging to do, and, therefore, this approach is less common on naive linear combinations.

Spherical linear interpolation (SLERP)

Another common model summing method is SLERP, which is based on the mathematical operator of the same name, Spherical LinEar interPolation.

NOTE

Interpolation means estimating unknown values based on known values. In the case of model merging, the unknown value is the merged model, and the known values are the constituent models. Linear combination is one interpolation technique. SLERP is another.

Because the formula for SLERP is mathy, and model-merging tools typically implement it for you, I won’t go into the details here. Intuitively, you can think of each component (vector) to be merged as a point on a sphere. To merge two vectors, you first draw the shortest path between these two points along the sphere’s surface. This is similar to drawing the shortest path between two cities along the Earth’s surface. The merged vector of these two vectors is a point along their shortest path. Where exactly the point falls along the path depends on the interpolation factor, which you can set to be between 0 and 1. Factor values less than 0.5 bring

the merged vector closer to the first vector, which means that the first task vector will contribute more to the result. A factor of 0.5 means that you pick a point exactly halfway. This middle point is the blue point in [Figure 7-16](#).

SLERP, as a mathematical operation, is defined with only two vectors, which means that you can merge only two vectors at a time. If you want to merge more than two vectors, you can potentially do SLERP sequentially, i.e., merging A with B, and then merging that result with C.

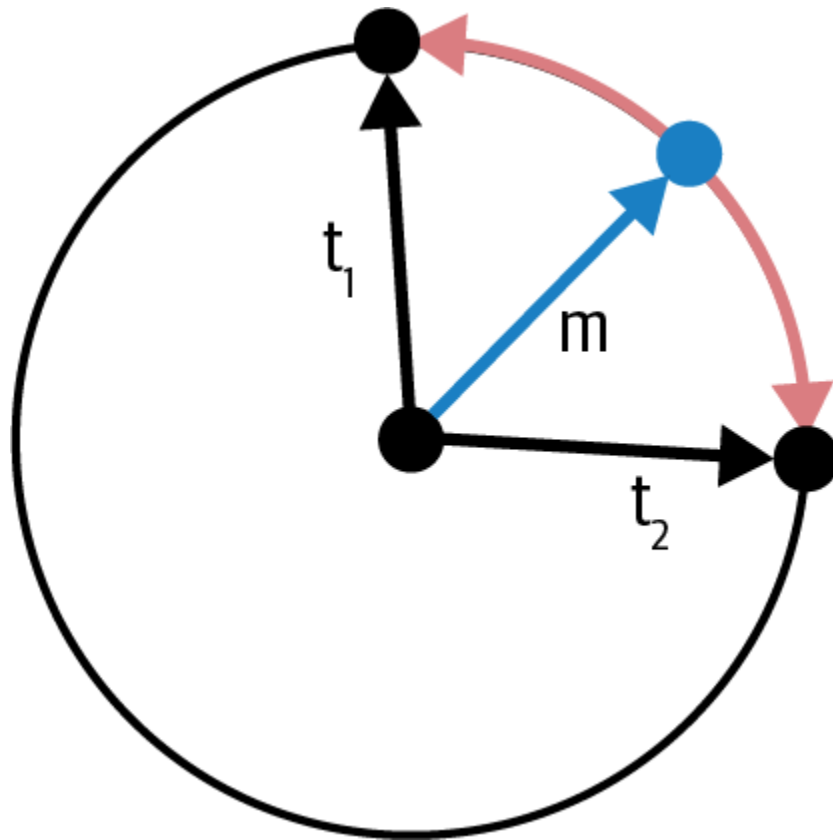


Figure 7-16. How SLERP works for two vectors t_1 and t_2 . The red line is their shortest path on the spherical surface. Depending on the interpolation, the merged vector can be any point along this path. The blue vector is the resulting merged vector when the interpolation factor is 0.5.

Pruning redundant task-specific parameters

During finetuning, many models' parameters are adjusted. However, most of these adjustments are minor and don't significantly contribute to the model's performance on the task.³¹ Adjustments that don't contribute to the model's performance are considered *redundant*.

In the paper “TIES-Merging: Resolving Interference When Merging Models”, [Yadav et al. \(2023\)](#) showed that you can reset a large portion of task vector parameters with minimal performance degradation, as shown in [Figure 7-17](#). Resetting means changing the finetuned parameter to its original value in the base model, effectively setting the corresponding task vector parameter to zero. (Recall that the task vector can be obtained by subtracting the base model from the finetuned model.)

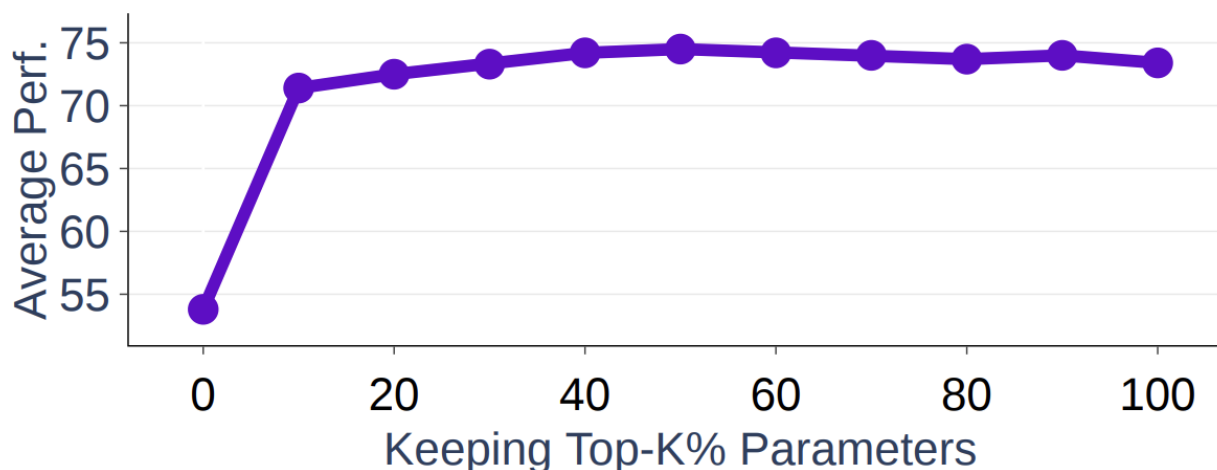


Figure 7-17. In Yadav et al.'s experiments, keeping the top 20% of the task vector parameters gives comparable performance to keeping 100% of the parameters.

These redundant parameters, while not harmful to one model, might be harmful to the merged model. Merging techniques such as TIES (Yadav et al., 2023) and DARE ([Yu et al., 2023](#)) first prune the redundant parameters from task vectors before merging them.³² Both papers showed that this practice can significantly improve the quality of the final merged models. The more models there are to merge, the more important pruning is because there are more opportunities for redundant parameters in one task to interfere with other tasks.³³

Layer stacking

In this approach, you take different layers from one or more models and stack them on top of each other. For example, you might take the first layer from model 1 and the second layer from model 2. This approach is also called *passthrough* or *frankenmerging*. It can create models with unique architectures and numbers of parameters. Unlike the merging by summing approach, the merged models resulting from layer stacking typically require further finetuning to achieve good performance.

One early success of frankenmerging is [Goliath-120B](#) (alpindale, 2023), which was merged from two finetuned Llama 2-70B models, [Xwin](#) and [Euryale](#). It took 72 out of 80 layers from each model and merged them together.

Layer stacking can be used to train mixture-of-experts (MoE) models, as introduced in “Sparse Upcycling: Training Mixture-of-Experts from Dense Checkpoints” ([Komatsuzaki et al., 2022](#)). Rather than training an MOE from scratch, you take a pre-trained model and make multiple copies of certain layers or modules. A router is then added to send each input to the most suitable copy. You then further train the merged model along with the router to refine their performance. [Figure 7-18](#) illustrates this process.

Komatsuzaki et al. showed that layer stacking can produce models that outperform MoE models trained from scratch. Using this approach, Together AI mixed six weaker open source models together to create Mixture-of-Agents, which achieved comparable performance to OpenAI’s GPT-4o in some benchmarks ([Wang et al., 2024](#)).

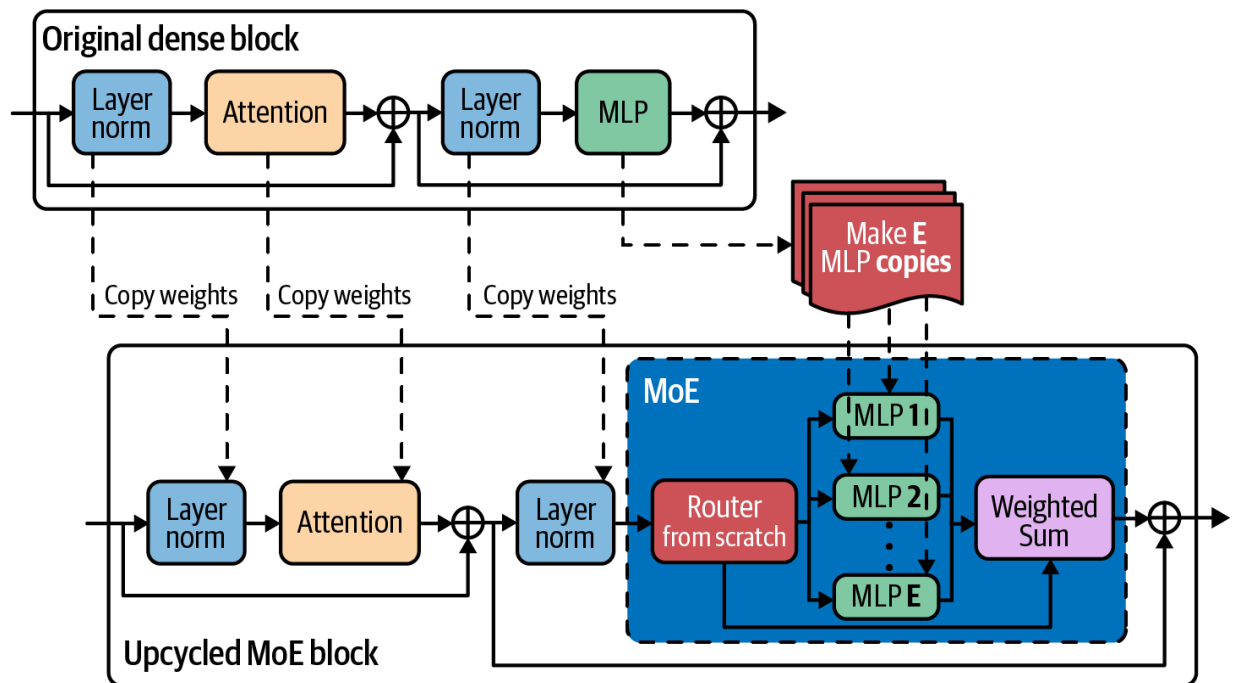


Figure 7-18. You can create an MoE model from a pre-trained model. Image adapted from Komatsuzaki et al. (2022).

An interesting use case of layer stacking is *model upscaling*. Model upscaling is the study of how to create larger models using fewer resources. Sometimes, you might want a bigger model than what you already have, presumably because bigger models give better performance. For example, your team might have originally trained a model to fit on your 40 GB GPU. However, you obtained a new machine with 80 GB, which allows you to serve a bigger model. Instead of training a new model from scratch, you can use layer stacking to create a larger model from the existing model.

One approach to layer upscaling is *depthwise scaling*. [Kim et al. \(2023\)](#) used this technique to create SOLAR 10.7B from one 7B-parameter model with 32 layers. The procedure works as follows:

1. Make a copy of the original pre-trained model.
2. Merge these two copies by summing certain layers (summing two layers and turning them into one layer) and stacking the rest. The layers to be summed are carefully selected to match the target model size. For SOLAR 10.7B, 16 layers are summed, leaving the final model with $32 \times 2 - 16 = 48$ layers.
3. Further train this upscaled model toward the target performance.

[Figure 7-19](#) illustrates this process.

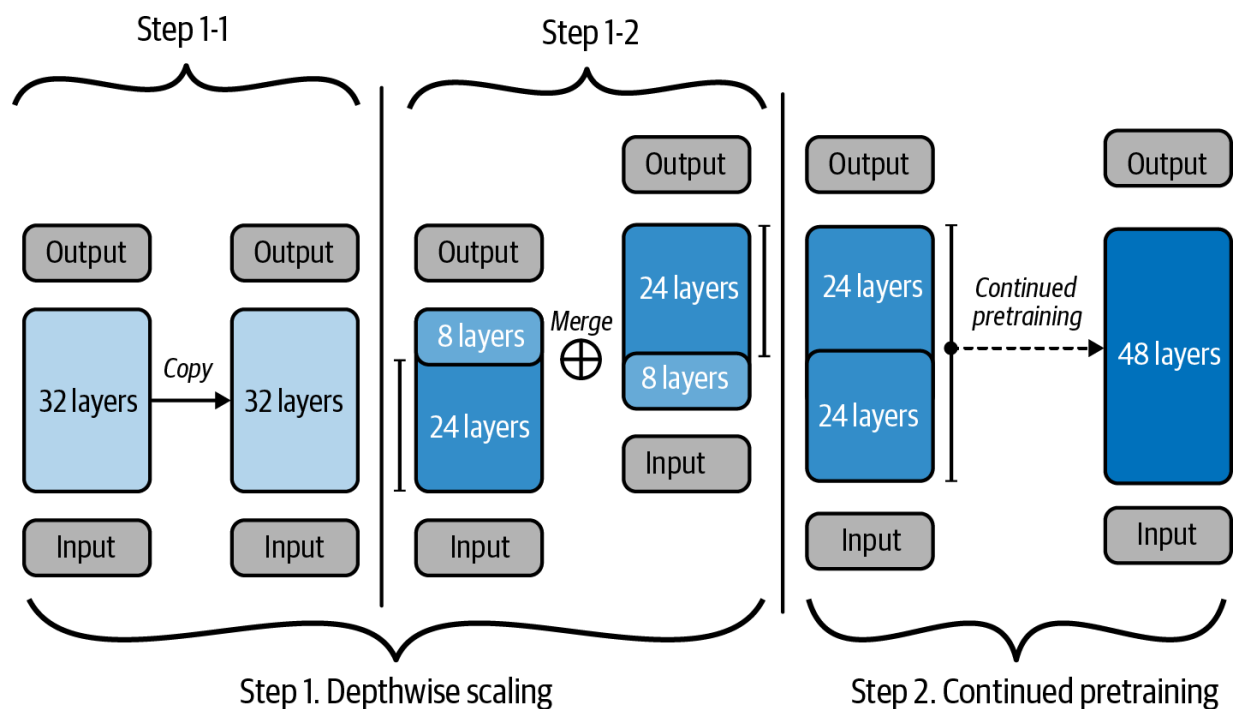


Figure 7-19. Use depthwise scaling to create a 48-layer model from a 32-layer model. The image is licensed under CC BY 4.0 and was slightly modified for readability.

Concatenation

Instead of adding the parameters of the constituent models together in different manners, you can also concatenate them. The merged component's number of parameters will be the sum of the number of parameters from all constituent components. If you merge two LoRA adapters of ranks r_1 and r_2 , the merged adapter's rank will be $r_1 + r_2$, as shown in [Figure 7-20](#).

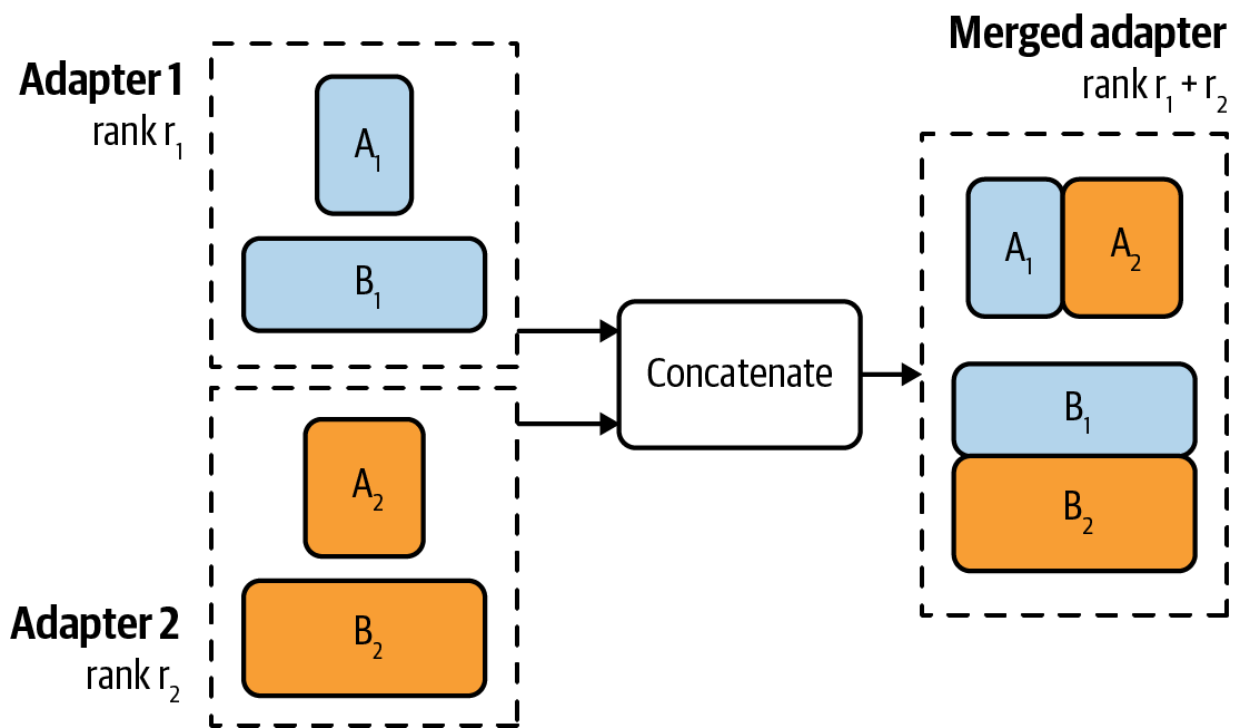


Figure 7-20. If you merge two LoRA adapters using concatenation, the rank of the merged adapter will be the sum of both adapters' ranks.

Concatenation isn't recommended because it doesn't reduce the memory footprint compared to serving different models separately. Concatenation might give better performance, but the incremental performance might not be worth the number of extra parameters. ³⁴

Finetuning Tactics

This chapter has discussed multiple finetuning approaches, what problems they solve, and how they work. In this last section, I'll focus on more practical finetuning tactics.

Finetuning frameworks and base models

While many things around finetuning—deciding whether to finetune, acquiring data, and maintaining finetuned models—are hard, the actual process of finetuning is more straightforward. There are three things you need to choose: a base model, a finetuning method, and a framework for finetuning.

Base models

[Chapter 4](#) already covered the criteria for model selection that can be applied to both prompt-based methods and finetuning. Some of the criteria discussed include model size, licenses, and benchmark performance. At the beginning of an AI project, when you're still exploring the feasibility of your task, it's useful to start with the most powerful model you can afford. If this model struggles to produce good results, weaker models are likely to perform even worse. If the strongest model meets your needs, you can then explore weaker models, using the initial model as a benchmark for comparison.

For finetuning, the starting models vary for different projects. [OpenAI's finetuning best practices document](#) gives examples of two development paths: the progression path and the distillation path.

The progression path looks like this:

1. Test your finetuning code using the cheapest and fastest model to make sure the code works as expected.³⁵
2. Test your data by finetuning a middling model. If the training loss doesn't go down with more data, something might be wrong.
3. Run a few more experiments with the best model to see how far you can push performance.
4. Once you have good results, do a training run with all models to map out the price/performance frontier and select the model that makes the most sense for your use case.

The distillation path might look as follows:

1. Start with a small dataset and the strongest model you can afford. Train the best possible model with this small dataset. Because the base model is already strong, it requires less data to achieve good performance.
2. Use this finetuned model to generate more training data.
3. Use this new dataset to train a cheaper model.

Because finetuning usually comes after experiments with prompt engineering, by the time you start to finetune, ideally, you should have a

pretty good understanding of different models' behaviors. You should plan your finetuning development path based on this understanding.

Finetuning methods

Recall that adapter techniques like LoRA are cost-effective but typically don't deliver the same level of performance as full finetuning. If you're just starting with finetuning, try something like LoRA, and attempt full finetuning later.

The finetuning methods to use also depend on your data volume. Depending on the base model and the task, full finetuning typically requires at least thousands of examples and often many more. PEFT methods, however, can show good performance with a much smaller dataset. If you have a small dataset, such as a few hundred examples, full finetuning might not outperform LoRA.

Take into account how many finetuned models you need and how you want to serve them when deciding on a finetuning method. Adapter-based methods like LoRA allow you to more efficiently serve multiple models that share the same base model. With LoRA, you only need to serve a single full model, whereas full finetuning requires serving multiple full models.

Finetuning frameworks

The easiest way to finetune is to use a finetuning API where you can upload data, select a base model, and get back a finetuned model. Like model inference APIs, finetuning APIs can be provided by model providers, cloud service providers, and third-party providers. A limitation of this approach is that you're limited to the base models that the API supports. Another limitation is that the API might not expose all the knobs you can use for optimal finetuning performance. Finetuning APIs are suitable for those who want something quick and easy, but they might be frustrating for those who want more customization.

You can also finetune using one of many great finetuning frameworks available, such as [LLaMA-Factory](#), [unsloth](#), [PEFT](#), [Axolotl](#), and [LitGPT](#). They support a wide range of finetuning methods, especially adapter-based techniques. If you want to do full finetuning, many base models provide their open source training code on GitHub that you can clone and run with your own data. [Llama Police](#) has a more comprehensive and up-to-date list of finetuning frameworks and model repositories.

Doing your own finetuning gives you more flexibility, but you'll have to provision the necessary compute. If you do only adapter-based techniques, a mid-tier GPU might suffice for most models. If you need more compute, you can choose a framework that integrates seamlessly with your cloud provider.

To finetune a model using more than one machine, you'll need a framework that helps you do distributed training, such as [DeepSpeed](#), [PyTorch Distributed](#), and [ColossalAI](#).

Finetuning hyperparameters

Depending on the base model and the finetuning method, there are many hyperparameters you can tune to improve finetuning efficiency. For specific hyperparameters for your use case, check out the documentation of the base model or the finetuning framework you use. Here, I'll cover a few important hyperparameters that frequently appear.

Learning rate

The learning rate determines how fast the model's parameters should change with each learning step. If you think of learning as finding a path toward a goal, the learning rate is the step size. If the step size is too small, it might take too long to get to the goal. If the step size is too big, you might overstep the goal, and, hence, the model might never converge.

A universal optimal learning rate doesn't exist. You'll have to experiment with different learning rates, typically between the range of $1e-7$ to $1e-3$, to see which one works best. A common practice is to take the learning rate at the end of the pre-training phase and multiply it with a constant between 0.1 and 1.

The loss curve can give you hints about the learning rate. If the loss curve fluctuates a lot, it's likely that the learning rate is too big. If the loss curve is stable but takes a long time to decrease, the learning is likely too small. Increase the learning rate as high as the loss curve remains stable.

You can vary learning rates during the training process. You can use larger learning rates in the beginning and smaller learning rates near the end. Algorithms that determine how learning rates should change throughout the training process are called learning rate schedules.

Batch size

The batch size determines how many examples a model learns from in each step to update its weights. A batch size that is too small, such as fewer than eight, can lead to unstable training.³⁶ A larger batch size helps aggregate the signals from different examples, resulting in more stable and reliable updates.

In general, the larger the batch size, the faster the model can go through training examples. However, the larger the batch size, the more memory is needed to run your model. Thus, batch size is limited by the hardware you use.

This is where you see the cost versus efficiency trade-off. More expensive compute allows faster finetuning.

As of this writing, compute is still a bottleneck for finetuning. Often, models are so large, and memory is so constrained, that only small batch sizes can be used. This can lead to unstable model weight updates. To address this, instead of updating the model weights after each batch, you can accumulate gradients across several batches and update the model weights once enough reliable gradients are accumulated. This technique is called *gradient accumulation*.³⁷

When compute cost isn't the most important factor, you can experiment with different batch sizes to see which gives the best model performance.

Number of epochs

An epoch is a pass over the training data. The number of epochs determines how many times each training example is trained on.

Small datasets may need more epochs than large datasets. For a dataset with millions of examples, 1–2 epochs might be sufficient. A dataset with thousands of examples might still see performance improvement after 4–10 epochs.

The difference between the training loss and the validation loss can give you hints about epochs. If both the training loss and the validation loss still steadily decrease, the model can benefit from more epochs (and more data). If the training loss still decreases but the validation loss increases, the

model is overfitting to the training data, and you might try lowering the number of epochs.

Prompt loss weight

For instruction finetuning, each example consists of a prompt and a response, both of which can contribute to the model's loss during training. During inference, however, prompts are usually provided by users, and the model only needs to generate responses. Therefore, response tokens should contribute more to the model's loss during training than prompt tokens.

The prompt model weight determines how much prompts should contribute to this loss compared to responses. If this weight is 100%, prompts contribute to the loss as much as responses, meaning that the model learns equally from both. If this weight is 0%, the model learns only from responses. Typically, this weight is set to 10% by default, meaning that the model should learn some from prompts but mostly from responses.

Summary

Outside of the evaluation chapters, finetuning has been the most challenging chapter to write. It touched on a wide range of concepts, both old (transfer learning) and new (PEFT), fundamental (low-rank factorization) and experimental (model merging), mathematical (memory calculation) and tactical (hyperparameter tuning). Arranging all these

different aspects into a coherent structure while keeping them accessible was difficult.

The process of finetuning itself isn't hard. Many finetuning frameworks handle the training process for you. These frameworks can even suggest common finetuning methods with sensible default hyperparameters.

However, the context surrounding finetuning is complex. It starts with whether you should even finetune a model. This chapter started with the reasons for finetuning and the reasons for not finetuning. It also discussed one question that I have been asked many times: when to finetune and when to do RAG.

In its early days, finetuning was similar to pre-training—both involved updating the model's entire weights. However, as models increased in size, full finetuning became impractical for most practitioners. The more parameters to update during finetuning, the more memory finetuning needs. Most practitioners don't have access to sufficient resources (hardware, time, and data) to do full finetuning with foundation models.

Many finetuning techniques have been developed with the same motivation: to achieve strong performance on a minimal memory footprint. For example, PEFT reduces finetuning's memory requirements by reducing the number of trainable parameters. Quantized training, on the other hand,

mitigates this memory bottleneck by reducing the number of bits needed to represent each value.

After giving an overview of PEFT, the chapter zoomed into LoRA—why and how it works. LoRA has many properties that make it popular among practitioners. On top of being parameter-efficient and data-efficient, it’s also modular, making it much easier to serve and combine multiple LoRA models.

The idea of combining finetuned models brought the chapter to model merging; its goal is to combine multiple models into one model that works better than these models separately. This chapter discussed the many use cases of model merging, from on-device deployment to model upscaling, and general approaches to model merging.

A comment I often hear from practitioners is that finetuning is easy, but getting data for finetuning is hard. Obtaining high-quality annotated data, especially instruction data, is challenging. The next chapter will dive into these challenges.

- Some people call this phenomenon an alignment tax ([Bai et al., 2020](#)), but this term can be confused with penalties against human preference alignment.
- Many businesses resist changing technologies they consider “good enough.” If all companies were quick to adopt more optimal solutions, fax machines would have become obsolete by now.

- ! I've also noticed a few cases when engineers know that finetuning isn't strictly necessary but still insist on doing it because they want to learn how to finetune. As an engineer who likes learning new skills, I appreciate this mindset. However, if you're in a leadership position, it can be hard to differentiate whether finetuning is needed or wanted.
- ! 0314 denotes the date this GPT-4 version came out, March 14, 2024. The specific date stamp matters because different versions vary significantly in performance.
- ! Some people, such as the authors of the Llama 3.1 paper ([Dubey et al., 2024](#)), adhere to "the principle that post-training should align the model to 'know what it knows' rather than add knowledge."
- ! Other than backpropagation, a promising approach to training neural networks is evolutionary strategy. One example, described by [Maheswaranathan et al.](#), combines random search with surrogate gradients, instead of using real gradients, to update model weights. Another interesting approach is direct feedback alignment ([Arild Nøkland, 2016](#)).
- If a parameter is not trainable, it doesn't need to be updated and, therefore, there's no need to compute its gradient.
- ! Some might say that you're not doing AI until you've seen a "RuntimeError: CUDA out of memory" error.
- ! To learn more about inference memory calculation, check out Carol Chen's ["Transformer Inference Arithmetic"](#), kipply's blog (March 2022).
- ! To learn more about training memory calculation, check out EleutherAI's ["Transformer Math 101"](#) (Anthony et al., April 2023).
- Google introduced BFloat16 as ["the secret to high performance on Cloud TPUs"](#).
- ! Integer formats are also called *fixed point* formats.

- ! Range bits are called *exponents*. Precision bits are called significands.
- ! Note that usually the number at the end of a format's name signifies how many bits it occupies, but TF32 actually has 19 bits, not 32 bits. I believe it was named so to suggest its functional compatibility with FP32. But honestly, why it's called TF32 and not TF19 keeps me up at night. An ex-coworker at NVIDIA volunteered his conjecture that people might be skeptical of weird formats (19-bit), so naming this format TF32 makes it look more friendly.
- ! The FP16 and BF16 confusion continued with Llama 3.1. See X and Threads discussions: [1](#); [2](#), [3](#), [4](#); and llama.cpp's [benchmark between BF16 and FP16](#), [Bloke's writeup](#), and [Raschka's writeup](#).
- ! Designing numerical formats is a fascinating discipline. Being able to create a lower-precision format that doesn't compromise a system's quality can make that system much cheaper and faster, enabling new use cases.
- ! Another major contributor to the memory footprint of transformer-based models is the KV cache, which is discussed in [Chapter 9](#).
- ! The smallest possible float size that follows all IEEE principles is 4-bit.
- ! The authors of the Xnor-Net paper spun off Xnor.ai, a startup that focused on model compression. [In early 2020, it was acquired by Apple for a reported \\$200M](#).
- ! During training, the model's weights are updated via multiple steps. Small rounding changes can compound during the training process, making it difficult for the model to achieve the desirable performance. On top of that, loss values require precise computation. Small changes in the loss value can point parameter updates in the wrong direction.
- ! Personal anecdote: much of my team's work at NVIDIA was on mixed precision training. See ["Mixed Precision Training for NLP and Speech Recognition with OpenSeq2Seq"](#) (Huyen et al., NVIDIA Developer Technical Blog, October 2018).

- ! In partial finetuning, it's common to finetune the layers closest to the output layer because those layers are usually more task-specific, whereas earlier layers tend to capture more general features.
- ! I've never met a single person who could explain to me, on the spot, the differences between these techniques.
- ! To effectively use LoRA for a model, it's necessary to understand that model's architecture. [Chapter 2](#) already covered the weight composition of some transformer-based models. For the exact weight composition of a model, refer to its paper.
- ! As of this writing, some finetuning frameworks like [Fireworks](#) only allow a maximum LoRA rank of 32. However, this constraint is unlikely due to performance and more likely due to their hardware's memory constraint.
- ! Search for these adapters by tags "adapter", "peft", or "LoRA".
- ! QLoRA isn't the only quantized LoRA work. Many research labs have been working on quantized LoRA without publicly discussing it.
- ! My book, [Designing Machine Learning Systems](#) has a section on "ML on the Cloud and on the Edge."
- ! You can read more about ensemble methods in my book [Designing Machine Learning Systems](#).
- ! Averaging works not just with weights but also with embeddings. For example, given a sentence, you can use a word embedding algorithm to generate an embedding vector for each word in the sentence, then average all these word embeddings into a sentence embedding. When I started out in ML, I couldn't believe that averaging seems to just work. It's magical when simple components, when used correctly, can create something so wonderfully perplexing, like AI.
- ! The assumption is that the parameters that undergo the most substantial changes during finetuning are the ones most crucial for the target task.

- ! TIES is abbreviated from “TrIm, Elect Sign, and merge,” while DARE is from “Drop And REscale.” I know, these abbreviations pain me too.
- ! When task vectors are pruned, they become more sparse, but the finetuned model doesn’t. Pruning, in this case, isn’t to reduce the memory footprint or inference latency, but to improve performance.
- ! I debated for a long time whether to include the concatenation technique in this book, and decided to include it for completeness.
- ! In college, I made the painful mistake of letting my model train overnight, only to have it crash after eight hours because I tried to save the checkpoint in a nonexistent folder. All that progress was lost.
- ! While it’s commonly acknowledged that small batch sizes lead to unstable training, I wasn’t able to find good explanations for why that’s the case. If you have references about this, please feel free to send them my way.
- ! I tried to find the first paper where gradient accumulation was introduced but couldn’t. Its use in deep learning was mentioned as early as 2016 in [“Ako: Decentralised Deep Learning with Partial Gradient Exchange”](#) (Watcharapichat et al., *Proceedings of the Seventh ACM Symposium on Cloud Computing*, 2016). The concept seems to come from distributed training, where gradients computed on different machines need to be accumulated and used to update the model’s weights.

Chapter 8. Dataset Engineering

The quality of a model depends on the quality of its training data. The best ML team in the world with infinite compute can't help you finetune a good model if you don't have data. The goal of dataset engineering is to create a dataset that allows you to train the best model, ideally within your allocated budget.

As fewer companies can afford to develop models from scratch, more are turning to data to differentiate their AI performance. As models demand more data, data handling becomes more challenging and demands more investments in talent and infrastructure.¹

Data operations have evolved from side tasks that people handle when they have time to dedicated roles. Many AI companies now employ data labelers, dataset creators, and data quality engineers, either integrated into or working alongside their core engineering teams.

If the model landscape is confusing enough with numerous offerings, the data landscape is even more complex, with an ever-growing array of datasets and techniques being introduced. This chapter gives you an overview of the data landscape and considerations to take into account when building your own dataset.

It begins with data curation, addressing questions like What data do you need? How much? What does it mean for data to be of high quality? It then discusses techniques for data synthesis and processing. Data curation, generation, and processing don't follow a linear path. You'll likely have to go back and forth between different steps.

For the same model, different training phases aim to teach the model different capabilities, and, therefore, require datasets with different attributes. For example, data quantity for pre-training is often measured in the number of tokens, whereas data quantity for supervised finetuning is often measured in the number of examples. However, at a high level, their curation processes follow the same principle. This chapter focuses on post-training data because that's more relevant to application developers. However, I'll also include lessons from pre-training data when these lessons are insightful for post-training.

There are best practices you can follow and tools that you can use to automate parts of the process. However, data will mostly just be toil, tears, and sweat.

A DATA-CENTRIC VIEW OF AI

The increasing focus on data during AI development has given rise to *data-centric AI*, as opposed to *model-centric AI*:

- Model-centric AI tries to improve AI performance by enhancing the models themselves. This involves designing new architectures, increasing the sizes of the models, or developing new training techniques.
- Data-centric AI tries to improve AI performance by enhancing the data. This involves developing new data processing techniques and creating high-quality datasets that allow better models to be trained with fewer resources.

In the early days of deep learning, many AI benchmarks were model-centric. Given a dataset like ImageNet, people try to train the best possible model using the same dataset. In recent years, more benchmarks have become data-centric. Given the same model, people try to develop a dataset that gives this model the best performance.

In 2021, Andrew Ng launched a [data-centric AI competition](#) where participants needed to improve upon the same base dataset by applying techniques such as fixing incorrect labels, adding edge case examples, augmenting data, etc.

In 2023, DataComp ([Gadre et al., 2023](#)) hosted a [competition](#) whose goal was to create the best dataset for training a CLIP model ([Radford et al., 2021](#)). A standardized script trains a CLIP model on each submitted dataset. The quality of a dataset is evaluated based on its resulting model's performance on 38 downstream tasks. In 2024, they hosted a similar competition to evaluate datasets for language models with scales from 412M to 7B parameters ([Li et al., 2024](#)). Other similar data-centric benchmarks include DataPerf ([MLCommons, 2023](#)) and dcbench ([Eyuboglu and Karlaš, 2022](#)).

The model-centric and data-centric division helps guide research. In reality, however, meaningful technological progress often requires investment in both model and data improvements.

Data Curation

While not all issues with AI models can be solved with data, data is often a key part of the solution. The right data can make the model more capable, safer, and able to handle longer contexts. Conversely, poor data can cause the model to increase biases and hallucinations. Mistakes in data can harm the model and waste resources.

Data curation is a science that requires understanding how the model learns and what resources are available to help it learn. Dataset builders should

work closely with application and model developers. In a small team, they might be the same person—the person responsible for training a model is also responsible for acquiring the data for it. However, organizations with high data demands often employ specialized roles.²

What data you need depends on your task and what you want to teach the model. For self-supervised finetuning, you need sequences of data. For instruction finetuning, you need data in the (instruction, response) format. For preference finetuning, you need data in the (instruction, winning response, losing response) format. To train a reward model, you can use the same data format as preference finetuning or use data with annotated scores for each of your examples in the ((instruction, response), score) format.

Training data should exhibit the behaviors you want your model to learn. Acquiring high-quality data annotations is always challenging, but it’s even more challenging if you want to teach models complex behaviors such as chain-of-thought (CoT) reasoning and tool use. Let’s go over these two examples to understand why:

Chain-of-thought

As discussed in [Chapter 5](#), CoT prompting nudges the model to work through a problem step-by-step before producing the final answer. To teach a model to generate step-by-step responses, its training data should include CoT responses. “Scaling Instruction-Finetuned Language Models” ([Chun et al., 2024](#)) shows that incorporating step-

by-step responses in the finetuning data greatly enhances the performance of models of various sizes on CoT tasks, with accuracy nearly doubling for certain tasks.

Generating multi-step responses can be tedious and time-consuming—explaining how to solve a math problem step-by-step is much more challenging than simply giving the final answer. To illustrate this, here are two examples, one with only the final answer and one with CoT. Both are from Chun et al. (2024):

Instruction: Please answer the following question. What is the boiling point of Nitrogen?

Response (without CoT): -320.4F

CoT instruction: Answer the following question by reasoning step-by-step. The cafeteria had 23 apples. If they used 20 for lunch and bought 6 more, how many apples do they have?

Response (with CoT): The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$.

As a result, CoT datasets are less common compared to other instruction datasets.

Tool use

Given the vast amount of knowledge a model acquires during pre-training, many models might intuitively know how to use certain tools. However, a model's tool use ability can be improved by

showing it tool use examples. It's common to use domain experts to create tool use data, where each prompt is a task that requires tool use, and its response is the actions needed to perform that task. For example, if you want data to finetune a model to act as a personal assistant, you might want to ask professional personal assistants what types of tasks they usually perform, how they perform them, and what tools they need. If you ask human experts to explain how they do things, they might miss certain steps, either because of faulty memory or because they might think these steps aren't important. It's often necessary to observe how humans perform these tasks to ensure accuracy.

However, what's efficient for humans might not be efficient for AI, and vice versa. As a result, human annotations might not be ideal for AI agents. For example, a human might prefer a web interface, whereas it's easier for a model to use an API. To search for something, a human might first open a browser, copy and paste that query into the search bar, and click on each result. Meanwhile, a model can just send a request to the search API with the query and process all the results at once. For this reason, many rely on simulations and other synthetic techniques to generate tool use data, as explored later in this chapter.

Tool use data might also require special formats. In typical conversation data, the user and AI take turns, with each turn

containing one message. However, for tool use, the AI might need to generate multiple messages each turn, with each message sent to a different location. For example, it might send one message to the code interpreter and one message to the user (such as to inform the user what it's doing). To support this, Llama 3 authors ([Dubey et al., 2024](#)) designed a multi-message chat format that consists of message headers that specify the source and destination of each message, and special termination tokens to specify where the human and AI turns start.

When curating data for applications with conversation interfaces, you need to consider whether you require single-turn data, multi-turn data, or both. Single-turn data helps train a model to respond to individual instructions. Multi-turn data, on the other hand, teaches the model how to solve tasks—many real-world tasks involve back-and-forth. For instance, when given a query, a model may need to first clarify the user's intent before addressing the task. After the model's response, the user might provide corrections or additional information for the next step.

Single-turn data is simpler and, therefore, easier to obtain. Multi-turn data often requires purpose-built scenarios or more involved interactions to capture.

Data curation isn't just about creating new data to help a model learn new behaviors but is also about removing existing data to help a model unlearn

bad behaviors. Imagine you work on a chatbot like ChatGPT and you hear user complaints that the chatbot is a bit arrogant, annoying users and wasting their tokens. For example, when a user asks it to verify if a statement is factually correct, the chatbot responds with: “The statement is correct, but its style can be improved to be better.” It then continues to produce an unsolicited rewriting of the statement.

You investigate and find that in the training data, there are several examples of annotations with unsolicited suggestions. You put in a request to remove these examples from the training data and another request to acquire new examples that demonstrate fact-checking without unsolicited rewriting.

Each application might require data of different characteristics. Different training phases also require different data mixes. At a high level, however, data curation follows the three criteria: data quality, data coverage, and data quantity.

To give an intuition about these terms, if you think of model training as cooking, the data fed into the model is the ingredients. Data quality is equivalent to the quality of the ingredients—you can’t have good food if your ingredients are spoiled. Data coverage is equivalent to having the right mix of ingredients (e.g., you shouldn’t have too much or too little sugar). Data quantity is about how many ingredients you should have. Let’s explore these terms in detail.

Data Quality

A small amount of high-quality data can outperform a large amount of noisy data, e.g., data that is irrelevant or inconsistent. The creators of the Yi model family found that 10K carefully crafted instructions are superior to hundreds of thousands of noisy instructions ([Young et al., 2024](#)).

Similarly, “LIMA: Less Is More for Alignment” ([Zhou et al., 2023](#)) shows that a 65B-parameter Llama model, finetuned with 1,000 carefully curated prompts and responses, can produce answers that are either equivalent or strictly preferred to GPT-4 in 43% of cases, as judged by human annotators. However, the downside of having too few data examples is that LIMA is not as robust as product-grade models.

The [Llama 3 team](#) also arrived at the same conclusion. Notably, they found that human-generated data is more prone to errors and inconsistencies, particularly for nuanced safety policies. This led them to develop AI-assisted annotation tools to ensure high data quality.

Most people understand the importance of data quality, but what does it mean for data to be high-quality? The short answer is that data is considered high-quality if it helps you do your job efficiently and reliably. The long answers, however, differ for different people.³ In general, data can be considered high-quality if it has the following six characteristics: relevant,

aligned with task requirements, consistent, correctly formatted, unique, and compliant. Some specific use cases might have other requirements:

Relevant

The training examples should be relevant to the task you're training the model to do. For example, if the task is to answer legal questions today, a legal dataset from the 19th century might not be relevant. However, if the task is about the legal system in the 19th century, this dataset is highly relevant.

Aligned with task requirements

The annotations should align with the task's requirements. For example, if the task requires factual consistency, the annotations should be factually correct. If the task requires creativity, the annotations should be creative. If the task demands not just a score but also a justification for that score, the annotations should include both scores and justifications. But if the task demands concise answers, the annotations should be concise.

I used “aligned” instead of “accurate” or “correct” because, depending on the task, an accurate or correct response might not be what a user wants.

Consistent

Annotations should be consistent across examples and annotators. If you ask two annotators to annotate the same example, their annotations shouldn't be too different. If the task is to score essays from 1 to 5, would two essays with the same score be of the same quality? Inconsistent annotations can confuse the model, making it harder for the model to learn.

Having a good annotation guideline is essential for having annotations that are both aligned with task requirements and consistent.

Correctly formatted

All examples should follow the format expected by the model. Redundant formatting tokens can interfere with the model's learning, and, therefore, they should be removed. For example, if you scrape product reviews from a website, you should remove HTML tags. Beware of trailing white spaces, new lines, inconsistent casing, and numerical formats.⁴

Sufficiently unique

This refers to unique examples in your data.⁵ In the context of model training, duplications can introduce biases and cause data contamination. I use “sufficiently unique” because specific use cases can tolerate different levels of duplications.

Compliant

Data should be compliant with all relevant internal and external policies (including laws and regulations). For example, if you're not allowed to use PII data to train your models, your data shouldn't contain any PII data.

Before setting out to create data, it's important to think about what each of these characteristics means for you. The techniques discussed in this section aim to produce data with these characteristics.

Data Coverage

A model's training data should cover the range of problems you expect it to solve. Real-world users often have a wide range of problems, and the way they express those problems can vary significantly. Having data that captures the diverse usage patterns of your application is key for the model to perform well. Coverage requires sufficient *data diversity*, which is why many refer to this attribute as data diversity.

For example, if some users construct detailed instructions with abundant references while some other users prefer short instructions, your finetuning data should include both detailed and short instructions. If user queries typically have typos, you should include examples with typos. If your application works with multiple programming languages, your training data should include the programming languages your users care about.

Different applications have different dimensions of diversity. For example, a French-to-English tool doesn't need language diversity but might benefit from diversity in topics, lengths, and speaking styles. On the other hand, a chatbot that recommends products to global customers doesn't necessarily need domain diversity, but linguistic and cultural diversity will be important.

For general-purpose use cases like chatbots, the finetuning data should be diverse, representing a wide range of topics and speaking patterns. [Ding et al., \(2023\)](#) believe that the most straightforward way to further improve the performance of chat language models is to increase the quality and diversity of data employed in the training process. To develop Nemotron ([Adler et al., 2024](#)), NVIDIA researchers focused on creating a dataset with task diversity, topic diversity, and instruction diversity, which includes instructions for different output formats, instructions with different output lengths, and instructions for open-ended answers as well as yes-or-no answers. "The Data Addition Dilemma" ([Shen et al., 2024](#)) demonstrated that in some cases, adding more heterogeneous data can lead to worse performance.

Meta shared that [Llama 3](#) doesn't deviate significantly from older Llama versions in terms of model architecture. Llama 3's performance gains are "primarily driven by improvements in data quality and diversity as well as by increased training scale." The Llama 3 paper has rich details on data coverage through all three phases of training: pre-training, supervised

finetuning, and preference finetuning. While this chapter focuses on post-training data, it's useful to look at the *data mix* for the same model across all different training phases to compare and highlight the considerations for each phase.

A diversity axis that is consistent in all three phases is domain diversity, though what exactly *diverse* means differs, as shown in [Table 8-1](#). This table shows only high-level domains and doesn't include finer-grained topics, like "geometry", which is a sub-category in math. Post-training data also has different diversity axes not shown in the table, such as the number of tokens (both for context and response) and the number of turns. Llama 3 uses synthetic data for post-training, so another dimension is the ratio of human-generated data to AI-generated data.

Table 8-1. For Llama 3, different training phases have different optimal domain mixes.

	Pre-training	Supervised finetuning	Preference finetuning
General knowledge (English)	50%	52.66%	81.99%
Math and reasoning	25%	21.19%	5.89%
Coding	17%	14.89%	6.93%
Multilingual	8%	3.01%	5.19%
Exam-like	X	8.14%	X
Long context	X	0.11%	X

It's interesting to note that during pre-training and supervised finetuning, the number of combined math, reasoning, and code tokens accounts for almost half of the training data. While I don't know exactly what percentage of the internet data is math and code, I believe that it's far below 50%. Llama 3 authors shared that *annealing* the model on small amounts of high-quality code and math data (training the model using an increasingly smaller learning rate with increasingly more code and math data) can boost the performance of their models on key benchmarks. This confirms a

common belief that high-quality code and math data is more effective than natural language text in boosting the model’s reasoning capabilities.

The percentage of code and math data during preference finetuning is much smaller (12.82% combined), likely because the goal is to reflect the real distribution of user preferences.

This brings up a question: How do we decide on the right data mix? A simple approach is to choose a data mix that accurately reflects the real-world application usage. You can also use experiments to find optimal data mixes. For example, Meta performed scaling law experiments similar to what is discussed in [“Scaling extrapolation”](#). For each candidate data mix, they trained several small models on a data mix and used that to predict the performance of a large model on that mix. The final model mix is the best-guess mix derived from the experiment results.

To evaluate the impact of data diversity and quality, [Zhou et al. \(2023\)](#) carried out an interesting experiment where they trained a 7B-parameter language model on three datasets of the same size—2,000 examples—but with different characteristics. The first is high-quality but not diverse. The second is diverse but low-quality. The third is both diverse and high-quality. [Figure 8-1](#) shows the generation quality of the three resulting models.

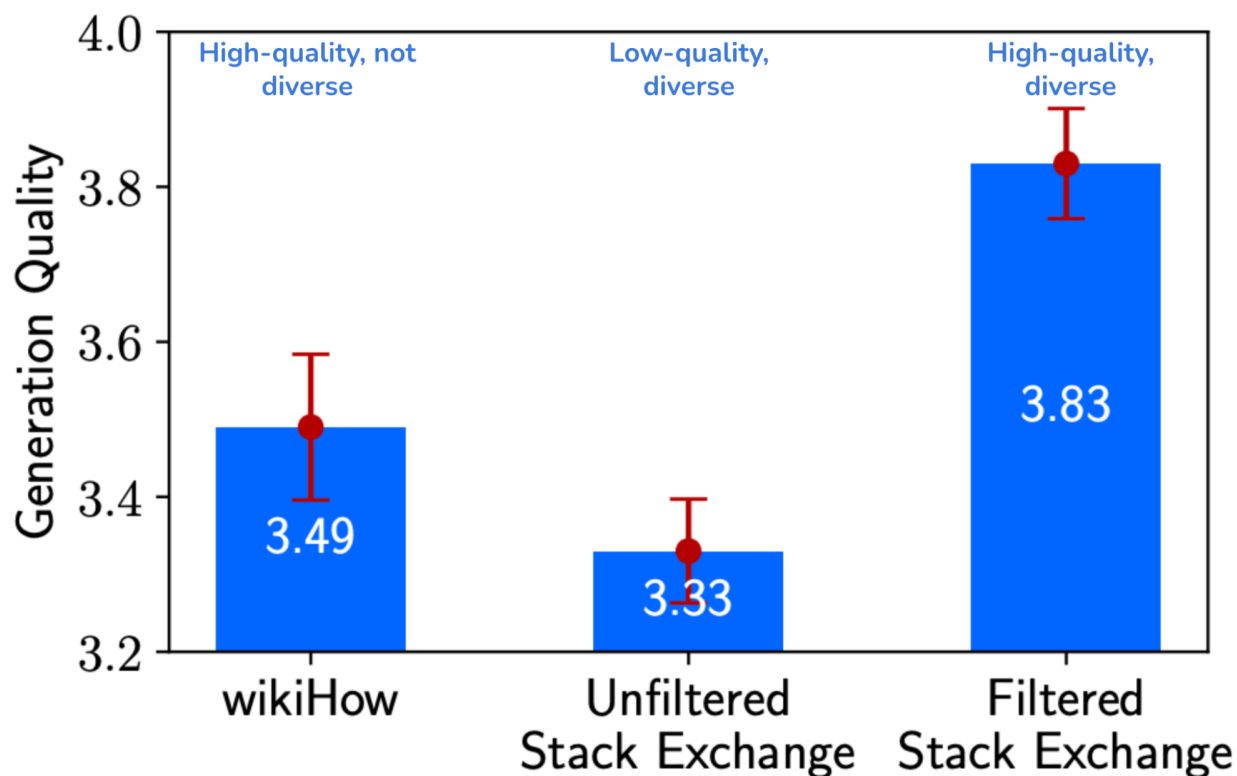


Figure 8-1. A 7B-parameter model, finetuned on a dataset that is both high-quality and diverse, outperforms that same model finetuned on a dataset that is either diverse or high-quality. Image from Zhou et al. (2023). The image is licensed under CC BY 4.0.

Data Quantity

Asking how much data you need is like asking how much money you need. The answer varies widely from one situation to the next. At one extreme, [Jeremy Howard and Jonathan Whitaker](#) did a fun experiment to show that LLMs can learn from a single example. At another extreme, some teams have finetuned models with millions of examples.

While millions of examples sounds like a lot, it's small compared to the data typically needed to train a foundation model from scratch. For reference, Llama 2 and Llama 3 were trained using 2 trillion and 16 trillion

tokens, respectively. If each example is 2,000 tokens, it'd be equivalent to 1 billion and 15 billion examples.

NOTE

You might wonder: if I have millions of examples, shouldn't I just train a model from scratch? You can and should evaluate whether training a model from scratch would improve your performance. While finetuning on top of a pre-trained model is typically more efficient than training from scratch, there are situations when finetuning can be worse, especially when you have a lot of training data. This is due to a phenomenon called *ossification*, where pre-training can *ossify* (i.e., freeze) the model weights so that they don't adapt as well to the finetuning data ([Hernandez et al., 2021](#)). Smaller models are more susceptible to ossification than larger models.

Other than data quality and data diversity, three other factors influence how much data you need:

Finetuning techniques

Full finetuning promises to give the best performance, but it requires orders of magnitude more data than PEFT methods like LoRA. If you have tens of thousands to millions of (instruction, response) pairs, you might want to attempt full finetuning. If you have only a few hundred or a few thousand examples, PEFT might work best.

Task complexity

A simple task, such as classifying whether a product review is positive or negative, will require much less data than a complex task,

such as a question answering about financial filings.

Base model's performance

The closer the base model is to the desirable performance, the fewer examples are needed to get there. Assuming that bigger base models are better, you might need fewer examples to finetune big models.

This is the opposite of pre-training, where bigger models need more training data.

[OpenAI's finetuning.guide](#) shows that if you have fewer examples (100), more advanced models give you better finetuning performance. This is likely because the more advanced models already perform better out of the box. However, after finetuning on a lot of examples (550,000), all five models in the experiment performed similarly, as illustrated in [Figure 8-2](#).

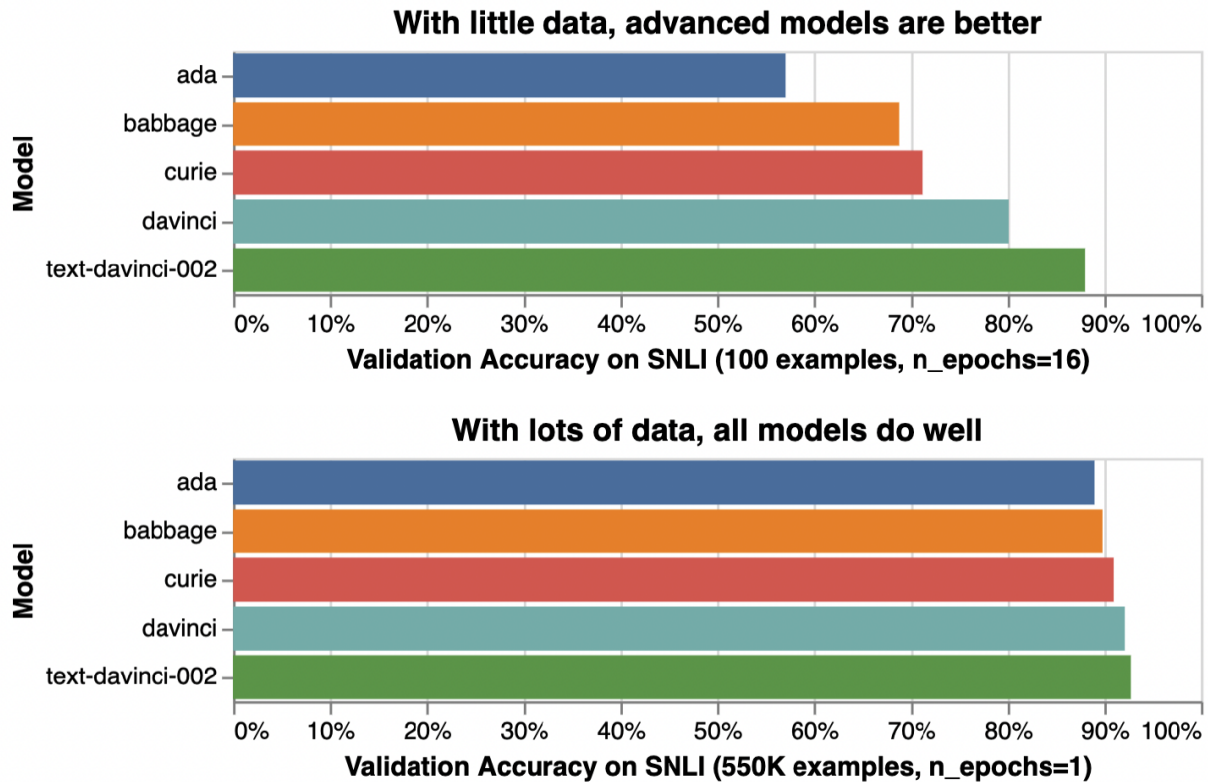


Figure 8-2. With 100 examples, more advanced models give much better performance after finetuning. With 550,000 examples, all models give similar performance after finetuning. Experiments done by Stanford Natural Language Inference (SNLI) Corpus.

In short, if you have a small amount of data, you might want to use PEFT methods on more advanced models. If you have a large amount of data, use full finetuning with smaller models.

Before investing in curating a large dataset, you might want to start with a small, well-crafted dataset (e.g., 50 examples) to see if finetuning can improve the model. If this small dataset is sufficient to achieve your desirable performance, that's great. Clear improvements suggest that more data will improve the performance even more. If no improvement is observed with small data, a bigger dataset will rarely do the trick.

However, be careful before concluding that finetuning with a small dataset doesn't improve a model. Many things, other than data, can impact finetuning's results, such as the choice of hyperparameters (e.g., the learning rate is too high or too low), data quality, poorly crafted prompts, etc. *In the vast majority of cases, you should see improvements after finetuning with 50–100 examples.*

TIP

It's possible to reduce the amount of high-quality data needed by first finetuning your model using lower-quality or less-relevant data. Here are three examples of this approach:

Self-supervised → supervised

You want to finetune a model to answer legal questions. Your (question, answer) set is small, but you have many legal documents. You can first finetune your model on legal documents in a self-supervised manner, then further finetune the model on (question, answer) pairs.

Less-relevant data → relevant data

You want to finetune a model to classify sentiments for product reviews, but you have little product sentiment data and much more tweet sentiment data. You can first finetune your model to classify tweet sentiments, then further finetune it to classify product sentiments.

Synthetic data → real data

You want to finetune a model to predict medical conditions from medical reports. Due to the sensitive nature of this task, your data is limited. You can use AI models to synthesize a large amount of data to finetune your model first, then further finetune it on your real data. This approach is harder to get right, as you'll have to do two distinct finetuning jobs while coordinating the transitioning between them. If you don't know what you're doing, you might end up using more compute just to produce a model worse than what you would've gotten by just finetuning with high-quality data.⁶

Experimenting with a small dataset can help you estimate how much more data you'll need. You can finetune a model on subsets of your current dataset—e.g., 25%, 50%, 100%—and plot how performance scales with dataset size. A steep performance gain slope with increasing dataset size means that you can expect significant performance improvement by

doubling your data. A plateau slope means that doubling your data will give only a small improvement. [Figure 8-3](#) shows an example of this plot.

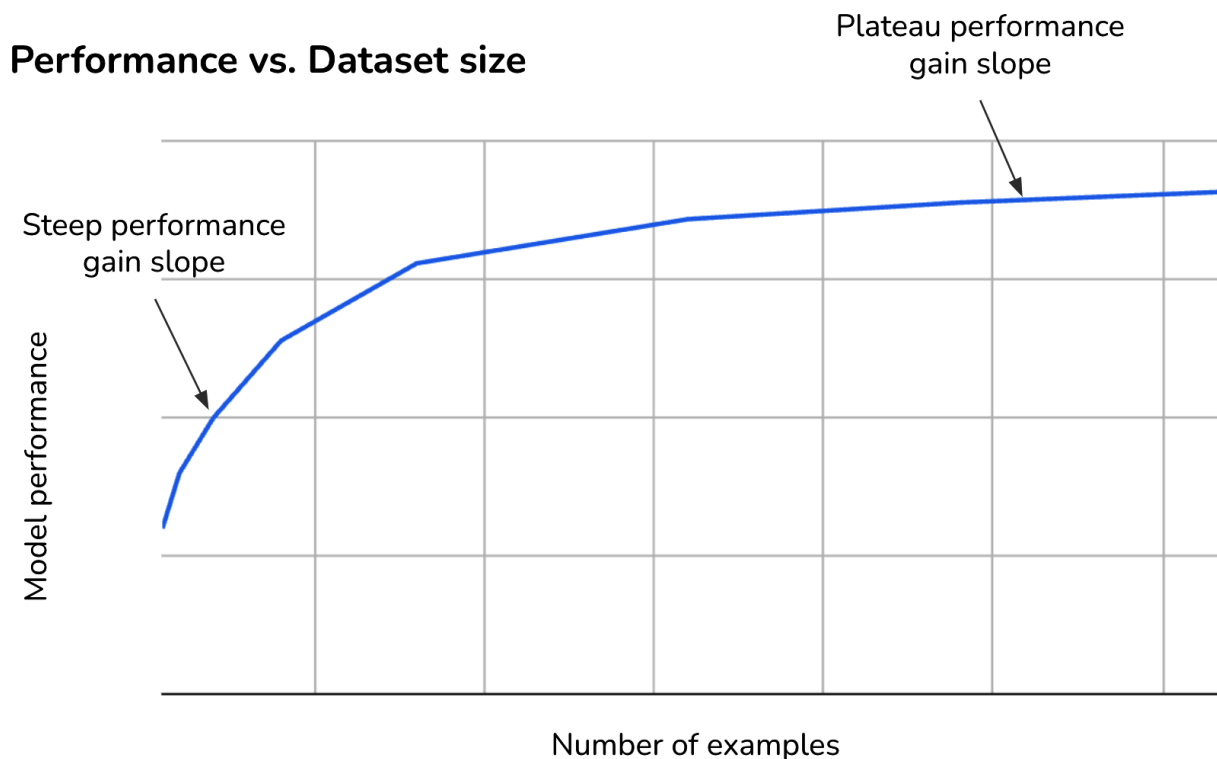


Figure 8-3. The performance gain curve with different dataset sizes can help you estimate the impact of additional training examples on your model's performance.

The performance gain curve shown in [Figure 8-3](#) is fairly typical. In most cases, additional training examples yield diminishing returns: the same number of examples typically gives a lower performance boost as the dataset grows. For example, the first 1,000 examples might improve a model's accuracy by ten percentage points, but the next 1,000 examples might only improve it by five.

While a larger number of finetuning examples generally improves a model's performance, the diversity of the examples matters, too. The paper "Scaling

Instruction-Finetuned Language Models” ([Chung et al., 2022](#)) shows that model performance increased significantly when the number of finetuning tasks increased from 9 to 282. Beyond 282 tasks, the performance gains started to plateau, though there were still positive but incremental improvements up to 1,836 tasks, as shown in [Figure 8-4](#). This suggests that the model benefits greatly from exposure to a diverse set of tasks during finetuning.

The diversity of data can be reflected in task types (such as summarization and question answering), topic diversity (such as fashion, finance, and technology), and the expected output formats (such as JSON outputs or yes-or-no answers).

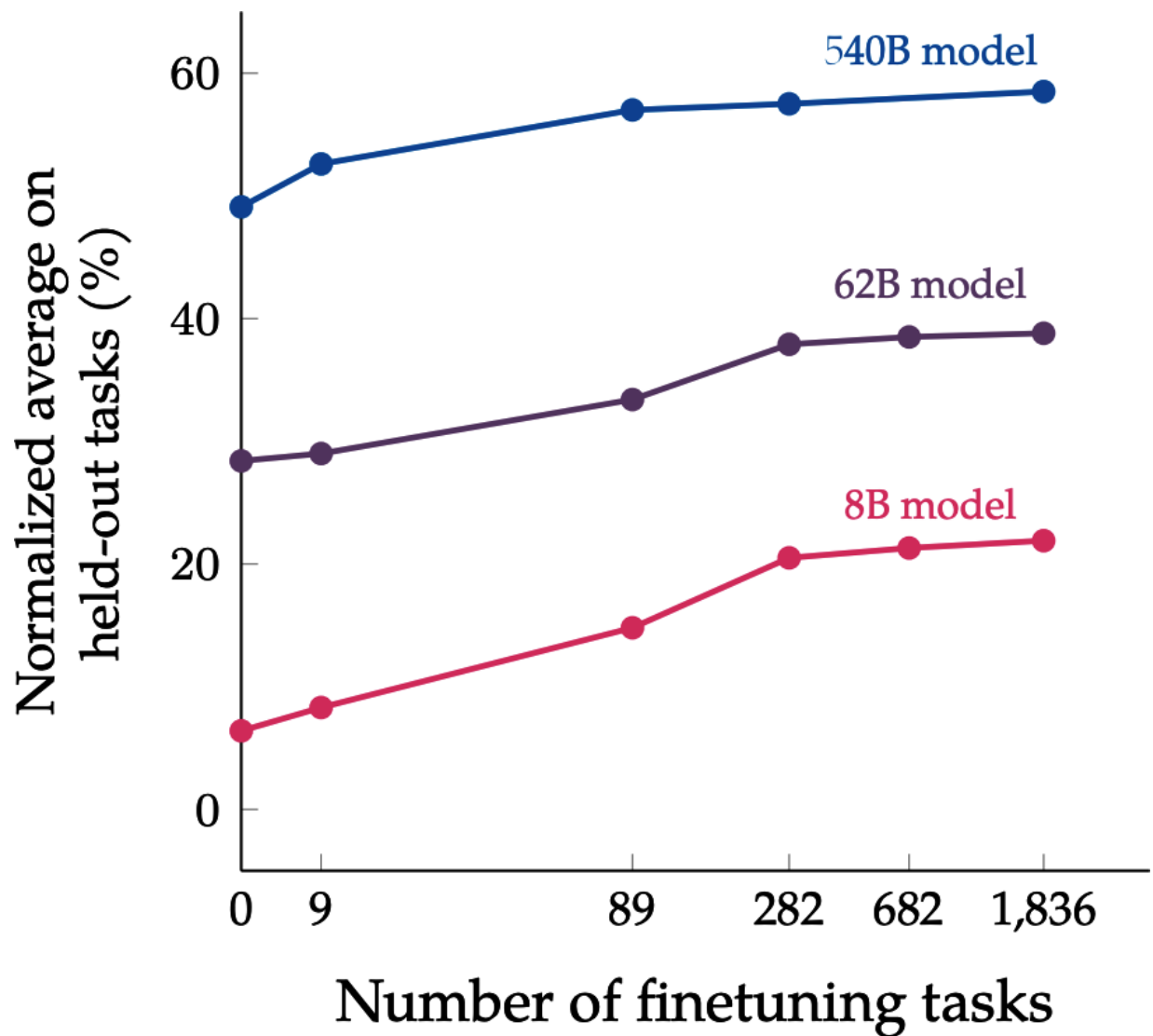


Figure 8-4. Diversity in finetuning number, measured by the number of tasks, can impact model performance. Image from “Scaling Instruction-Finetuned Language Models” (Chung et al., 2022). The image is licensed under CC BY 4.0.

How much data to use for finetuning is determined not just by what you need but also by what you can afford. If you budget \$10,000 for data annotation and each example costs \$2 to annotate, you can have at most 5,000 examples. You might also need to balance the budget for data and compute. Spending more money on data leaves you less money for compute, and vice versa.

Data Acquisition and Annotation

The goal of data acquisition is to produce a sufficiently large dataset with the quality and diversity you need, while ensuring that your data practices respect user privacy and comply with regulations. Data acquisition involves gathering data through methods such as sourcing public data, purchasing proprietary data, annotating data, and synthesizing data. There's a niche but growing field of research in *data acquisition strategy*: how to best acquire a dataset that meets specific requirements given a budget.

The most important source of data, however, is typically data from your own application. If you can figure out a way to create a *data flywheel* that leverages data generated by your users to continually improve your product, you will gain a significant advantage.⁷ Application data is ideal because it's perfectly relevant and aligned with your task. In other words, it matches the distribution of the data that you care about, which is incredibly hard to achieve with other data sources. User-generated data can be user content, system-generated data from user usage, or user feedback. How to design your user feedback system is discussed in [Chapter 10](#).

Before investing in creating your own data, check available datasets first. Data marketplaces are vast and offer both open source and proprietary data. If you're lucky, some of them might be exactly what you need. However, it's often a mix-and-match approach. A dataset can be developed from

multiple data sources via multiple acquisition channels. For example, the process of creating an (instruction, response) dataset might look as follows:

1. Find available datasets with the desirable characteristics. You might find one promising dataset with 10,000 examples.
2. Remove low-quality instructions. Let's say this leaves you with 9,000 examples.
3. Set aside the instructions with low-quality responses. Let's say you find 3,000 such examples. This leaves you with 6,000 examples of high-quality instructions and high-quality responses.
4. Manually write responses for the 3,000 high-quality instructions. Now your dataset has a total of 9,000 high-quality examples.
5. Realizing that there's not enough data for topic X, manually create a set of 100 instruction templates about X. Use an AI model to synthesize 2,000 instructions using these 10 templates.
6. Manually annotate these 2,000 synthetic instructions. Now your dataset has a total of 11,000 examples.

This is, of course, an oversimplification of the actual dataset curation process, with the vast majority of steps hidden to conserve paper and save readers from tedium. For example, there might be several steps in which you realize that many of the annotations aren't helpful, so you have to update the annotation guidelines and re-annotate your data. Worse, you might find that some of them are factually incorrect, so you have to hire another set of annotators to fact-check your original annotations. Or you

might find that having 100 synthetic instructions per template hurts your data's diversity, so you have to create more templates and generate fewer instructions per template. And so on.

RESOURCES FOR PUBLICLY AVAILABLE DATASETS

Here are a few resources where you can look for publicly available datasets. While you should take advantage of available data, you should never fully trust it. Data needs to be thoroughly inspected and validated.

Always check a dataset's license before using it. Try your best to understand where the data comes from. Even if a dataset has a license that allows commercial use, it's possible that part of it comes from a source that doesn't:

1. [Hugging Face](#) and [Kaggle](#) each host hundreds of thousands of datasets.
2. Google has a wonderful and underrated [Dataset Search](#).
3. Governments are often great providers of open data. [Data.gov](#) hosts hundreds of thousands of datasets, and [data.gov.in](#) hosts tens of thousands.
4. University of Michigan's [Institute for Social Research](#) ICPSR has data from tens of thousands of social studies.
5. [UC Irvine's Machine Learning Repository](#) and [OpenML](#) are two older dataset repositories, each hosting several thousand datasets.
6. The [Open Data Network](#) lets you search among tens of thousands of datasets.
7. Cloud service providers often host a small collection of open datasets; the most notable one is [AWS's Open Data](#).
8. ML frameworks often have small pre-built datasets that you can load while using the framework, such as [TensorFlow datasets](#).

9. Some evaluation harness tools host evaluation benchmark datasets that are sufficiently large for PEFT finetuning. For example, [Eleuther AI's lm-evaluation-harness](#) hosts 400+ benchmark datasets, averaging 2,000+ examples per dataset.
 10. The [Stanford Large Network Dataset Collection](#) is a great repository for graph datasets.
-

Often, you might need to annotate your own data for finetuning. Annotation is challenging not just because of the annotation process but also due to the complexity of creating clear annotation guidelines. For example, you need to explicitly state what a good response looks like, and what makes it good. Can a response be correct but unhelpful? What's the difference between responses that deserve a score of 3 and 4? Annotation guidelines are needed for both manual and AI-powered annotations.

Some teams, including [LinkedIn](#), have reported that annotation guidelines were among the most challenging parts of their AI engineering pipeline. It's alarming how often people abandon careful annotation halfway due to the time and effort required, hoping instead that their models will figure out the right responses on their own. Many models are strong enough that they can occasionally succeed, but relying on models to figure that out might be too risky for many applications.

The good news is that these guidelines are the same as those for evaluation data, as discussed in [Chapter 4](#). This is another argument for why you

should invest more time in curating evaluation guidelines and data. If you're lucky, your evaluation examples can be augmented or used as seed examples to synthesize new data. In the next section we'll discuss how to do so.

Data Augmentation and Synthesis

Together with compute and talent, data is the hardest challenge of AI. It's been a long-term goal of the whole industry to be able to generate data programmatically. Two processes commonly used are *data augmentation* and *data synthesis*:

- Data augmentation creates new data from existing data (which is real). For example, given a real image of a cat, you can flip it to create a new image of the same cat.⁸
- Data synthesis generates data to mimic the properties of real data. For example, you can simulate how a mouse moves through a web page to generate data for what bot movements would look like.

In other words, augmented data is derived from real data, whereas synthetic data isn't real. However, since the goal of both augmentation and synthesis is to automate data creation, sometimes the two terms are used interchangeably. In this chapter, I'll often use data synthesis to refer to both.

Artificially generated data has a long history in software engineering. It was originally used to generate fake data for testing purposes. For example, libraries like [*Faker*](#) and [*Chance*](#) let you generate data in simple formats such as names, addresses, phone numbers, and email addresses for testing. Let's say you've built a program to parse shipping addresses. You can use fake data generators to generate addresses in different countries and states with different formats to make sure your program can parse all of them.

With AI being capable of generating data indistinguishable from that generated by humans, it's possible to synthesize much more sophisticated data, such as doctor's notes, contracts, financial statements, product descriptions, images, video commercials, etc. This makes it easier to generate data and enables more synthetic data use cases.

While synthetic data promises to significantly reduce the pressure for human-generated data, synthetic data doesn't completely replace human data. In many use cases, as discussed in ["Limitations to AI-generated data"](#), mixing human- and AI-generated data often produces the best value.

Why Data Synthesis

Synthetic data is appealing for many reasons. You can synthesize data to improve the golden data trio: quantity, coverage, and quality. You can also synthesize data to mitigate privacy concerns and distill models:

To increase data quantity

The biggest reason for data synthesis is that it allows you to produce data at scale, promising an abundant supply of data for training and testing AI models. More data, in theory, helps models generalize to a wider range of tasks. This is especially helpful where real-world data is scarce or difficult to obtain, such as data for rare weather conditions, data for deep sea exploration, or data involving accidents for self-driving cars.

To increase data coverage

You can generate data with targeted characteristics to improve model performance or to get a model to express specific behaviors. For example, you can generate very short texts or very long texts. You can create conversations that contain toxic phrases for a toxic detection model. Vice versa, if real-world data is toxic, you can synthesize safe data. It's especially common to use AI to synthesize adversarial examples. It's also possible to generate data for the rare class to address the challenges of class imbalance. As described in "TrueTeacher", [Gekhman et al. \(2022\)](#), used LLMs to generate factually inconsistent summaries that they then used to train models to detect factual inconsistency.

In their paper, "Discovering Language Model Behaviors with Model-Written Evaluations" ([Perez et al., 2022](#)), Anthropic discussed various data synthesis techniques to generate specific datasets that can test 154 different AI behaviors, including personality traits,

political views, ethical stances, and social biases. They found that in head-to-head comparisons between LM (language model)-generated and human-generated datasets, “LM-written datasets approach the quality of human-written ones, sometimes even exceeding them.”

In other words, you can use synthetic data to increase data coverage: generate targeted data to cover the areas where existing data is insufficient.

To increase data quality

Even though the common perception is that synthetic data is often of lower quality than human-generated data, sometimes, the reverse can be true. *Sometimes, humans might have fundamental limitations that cause human-generated data to be of lower quality than AI-generated data.* One example is tool use data discussed earlier—humans and AI have fundamentally different modes of operations and tool preferences. Another example is in generating complex math problems—AI can generate questions that are far more complex than what an average human expert might conceive.⁹

Some teams also prefer using AI to generate preference data. While each individual human can be somewhat consistent in their preference, performance across different people tends to vary significantly, influenced not only by each person’s preference but

also by mood and motivations. AI-generated preference ratings, in contrast, can be far more consistent and reliable.

To mitigate privacy concerns

Synthetic data is often the only option for use cases where you can't use human-generated data due to privacy concerns. For instance, in healthcare, where legislation makes it hard, if not impossible, to use real patient records to train a model, you can generate synthetic patient records that do not contain any sensitive information. In insurance, you can use synthetic claims instead of using real claims that include sensitive personal and financial information.

To distill models

Sometimes, you might want to train a model to imitate the behavior of another model. The goal is often to create a cheaper and/or faster model (the distilled model) with performance comparable to that of the original model. This is done by training the distilled model using data generated by the original model.

These are just five of the many reasons why people turn to data synthesis. Because of its undeniable appeal, more models are being trained with synthetic data and more techniques are being developed to synthesize data.

Traditional Data Synthesis Techniques

Data synthesis isn't unique to AI. It has a long history in software testing, gaming, and robotics. Using algorithms to generate data is also called *procedural generation*, as opposed to *manual generation*. Procedural generation is commonly used in gaming to generate content such as levels, maps, items, and characters on the fly.¹⁰ Most data generation techniques used in these industries can be applied to AI.

Traditionally, two approaches for data synthesis and augmentation have been rule-based and simulation. A newer method made possible by advanced AI models is using AI itself to synthesize data. This section gives a quick overview of these two traditional techniques before moving on to AI-powered data synthesis in the next section.

Rule-based data synthesis

The simplest way to generate data is to use predefined rules and templates. For example, to create a credit card transaction, start with a transaction template and use a random generator like Faker to populate each field in this template:

```
An example of a transaction template.  
Transaction ID: [Unique Identifier]  
Date: [MM/DD/YYYY]  
Time: [HH:MM:SS]
```

```
Amount: [Transaction Amount]
Merchant Name: [Merchant/Store Name]
Merchant Category: [Category Code]
Location: [City, State, Country]
Payment Method: [Credit Card/Debit Card/Cash/Online]
Transaction Status: [Completed/Pending/Failed]
Description: [Transaction Description]
```

Due to the sensitivity of transaction data, many fraud detection models are first trained on synthetic transaction data generated from templates like this to prove their feasibility before being given access to real data.

It's common to use templates to generate documents that follow a specific structure, such as invoices, resumes, tax forms, bank statements, event agendas, product catalogs, contracts, configuration files, etc. Templates can also be used to generate data that follows a certain grammar and syntax, such as regular expressions and math equations. You can use templates to generate math equations for AI models to solve. DeepMind trained an Olympiad-level geometry model, AlphaGeometry, using 100 million synthetic examples ([Trinh et al., 2024](#)).

You can procedurally generate new data from existing data by applying simple transformations. For images, you can randomly rotate, crop, scale, or erase part of an image. A flipped image of a cat should still be a cat. A slightly cropped image of a soccer game should still be a soccer game.

[Krizhevsky et al. \(2012\)](#) demonstrated in their legendary AlexNet paper the usefulness of this technique by using it to augment the ImageNet dataset ([Deng et al., 2009](#)).

For texts, you can randomly replace a word with a similar word, assuming that this replacement wouldn't change the meaning or the sentiment of the sentence. For example, the original sentence "She's a *fantastic* nurse" can generate a new example: "She's a *great* nurse".

This approach can be used to mitigate potential biases in your data. If you're concerned that there's a gender bias in your data, where, for example, the word "nurse" is associated with women while the word "doctor" is associated with men, you can replace typically gendered words with their opposites, such as "she" with "he", as shown in [Table 8-2](#).

Table 8-2. Data augmentation can help mitigate certain biases in your data.

Original data	Augmented data
She's a fantastic nurse.	<i>He's</i> a fantastic nurse. She's a fantastic <i>doctor</i> .
The CEO of the firm, Mr. Alex Wang, ...	The CEO of the firm, <i>Ms. Alexa Wang</i> , ...
Today, my mom made a casserole for dinner.	Today, my <i>dad</i> made a casserole for dinner.
Emily has always loved the violin.	<i>Mohammed</i> has always loved the violin.

Similar words can be found either with a dictionary of synonymous words or by finding words whose embeddings are close to each other in a word embedding space. You can go beyond simple word replacement by asking AI to rephrase or translate an example, as we'll discuss later.

One interesting transformation is perturbation: adding noise to existing data to generate new data. Initially, researchers discovered that perturbing a data sample slightly can trick models into misclassifying it. For example, adding white noise to a picture of a ship can cause the model to misclassify it as a car. The paper “One Pixel Attack for Fooling Deep Neural Networks” ([Su et al., 2017](#)) showed that 67.97% of the natural images in the Kaggle CIFAR-

10 test dataset and 16.04% of the ImageNet test images could be misclassified by changing just one pixel. This poses a serious risk if exploited. An attacker could trick an AI model into misidentifying them as an authorized employee or make a self-driving car mistake a divider for a lane, leading to accidents.

You can train your model on perturbed data. Perturbation can both improve the model's performance and make it more robust against attacks; see [Goodfellow et al., 2013](#) and [Moosavi-Dezfooli et al., 2015](#)). In 2019, Hendrycks and Dietterich created [ImageNet-C and ImageNet-P](#) by applying 15 common visual corruptions, such as changing brightness, adding snow, changing contrast, and adding noises to ImageNet images.

Perturbation can also be used for texts. For example, to train BERT, the authors replaced 1.5% of the tokens with random words ([Devlin et al., 2018](#)). They found this perturbation led to a small performance boost.

Visual data can be augmented using more sophisticated algorithms. [Snap \(2022\)](#) has a great case study on how they augment their assets to create unrepresented corner cases and mitigate implicit biases in their data. Given a character, they synthesize similar characters but with different skin colors, body types, hairstyles, clothes, and even facial expressions. These augmented assets are then used to train AI models.

Simulation

Instead of running experiments to collect data in the real world, where it can be expensive and dangerous, you can simulate these experiments virtually. For example, to test how a self-driving car reacts when encountering a horse on the highway, it'd be dangerous to release an actual horse on the highway. Instead, you simulate this situation in a virtual environment. Examples of self-driving simulation engines include CARLA ([Dosovitskiy et al., 2017](#)), [Waymo's SimulationCity](#), and [Tesla's simulation of San Francisco](#).

Similarly, it's very common to simulate training data for robotics in a virtual environment. Let's say you want to train a robot to pour coffee, but you don't know exactly how each joint should move to make the action successful. You can simulate multiple scenarios with different joint movements and use only the scenarios where coffee is successfully poured to train the robot.

Simulations allow you to run multiple experiments with minimal costs while avoiding accidents and physical damage. A robot that works in simulations might not work in the real world, but if it fails in simulations, it'll likely fail in the real world. No matter how sophisticated your simulations are, however, they are simplifications of the real world. Sim2Real is a subfield that focuses on adapting algorithms that have been trained in simulations to the real world.

Simulations are common to generate data to teach models to use tools. As mentioned earlier, human-generated actions might not always be the most efficient for AI agents. Simulations might help uncover actions that humans overlook. Given a query, you can simulate different action sequences, execute these sequences, and validate their outcomes. The most efficient action sequence is then used as the annotated response for the query.

Simulations are particularly valuable for generating data for rare events. For example, in finance, researchers can simulate scenarios such as a company successfully going public or a significant bankruptcy to understand their market impacts. Manufacturers can simulate defects in materials or assemblies to generate data to train anomaly detection and quality control models. Similarly, by simulating the Earth's systems, climate scientists can create variations in temperature changes, precipitation patterns, and extreme weather scenarios. This synthetic data is then fed into AI models, enabling them to learn from a broader spectrum of possible futures.

Both rule-based and simulation-based techniques have been useful for many use cases, but it wasn't until AI became capable of generating realistic and high-quality data that data synthesis really took off. Let's look into those methods next.

AI-Powered Data Synthesis

Just as there are virtually infinite ways for humans to generate data, AI can also do so in many ways. The techniques discussed here are not comprehensive, but they should give you a good overview.

Powerful AI models open many new possibilities for simulations. AI can simulate the outcomes of arbitrary programs. For example, “StableToolBench” ([Guo et al., 2024](#)) demonstrates how to use AI to simulate APIs without having to evoke them. Imagine you want to train a model to interact with a set of APIs. Instead of making actual API calls—which might be costly or slow—you can use an AI model to simulate the expected outcomes of those calls.

AI can simulate humans. For example, imagine you want to train a bot to play chess. A game played by humans might take too long. Matches with AI players would be much faster. To train its Dota 2 bot, OpenAI used a simulator that enabled the bot to play approximately 180 years’ worth of games every day. The bot learned by playing against itself, an approach called *self-play*, which helped it develop and refine strategies over time ([OpenAI, 2019](#)). Similarly, DeepMind used self-play to collect data from millions of Go games to train AlphaGo ([Silver et al., 2016](#)).

Self-play is useful not just for game bots but also for general agents. You can have AIs negotiate against each other using different strategies to see

which one works better. You can have one version of the model play the role of a customer with issues and another play the customer support agent.

AI's paraphrasing and translation abilities can be used to augment existing datasets. For example, given the query “How to reset my password?”, AI can paraphrase it to create three new queries:

1. “I forgot my password.”
2. “How can I change my password?”
3. “Steps to reset passwords.”

Yu et al. (2023) rewrote the 15,000 examples in MATH and GSM-8K in different ways to create MetaMath, a new dataset of almost 400,000 examples. They showed that their models, trained on this new dataset, outperformed larger models on related math benchmarks.

It's common to use AI to translate data in high-resource languages (more available online) into low-resource languages to help train models in low-resource languages. This is useful for training a small model specializing in a low-resource language like Quechua or Lao.

You can verify the quality of translations with *back-translation*. Let's say the original English sentence is X and the translated Lao sentence is Y . You can use another model to translate the translation back into the original language, X' , then compare X' with the original sentence X . If they are very different, the translation Y is likely bad.

AI can translate not just natural languages but also programming languages. You can use AI to translate code written in one language to another. The [Llama 3 authors](#) used code translation of their SFT dataset with a wider range of programming languages. In fact, the training of Llama 3 depends heavily on synthetic data, and the authors used many creative techniques to generate useful data.

For example, they used back-translation to generate code explanations and documentation. Starting with code snippets, they used AI to generate explanations and documentation. They then again used AI to generate code snippets from the explanations and documentation. Only if the generated code is considered faithful to the original will the explanation and documentation be used to finetune the model.

AI can generate data for both pre-training and post-training, though synthetic data is intentionally included much more often in post-training than in pre-training. One possible explanation for this is that pre-training's goal is to increase the model's knowledge, and while AI can synthesize existing knowledge in different formats, it's harder to synthesize new knowledge.

However, as the internet becomes flooded with AI-generated content, models that rely on internet data are likely already pre-trained on synthetic data. There are also synthetic datasets such as [Cosmopedia](#) (Allal et al., 2024), a 25-billion-token collection of synthetic textbooks, blog posts,

stories, posts, and WikiHow articles generated by [Mixtral-8x7B-Instruct-v0.1](#) (Jiang et al., 2024).

Data synthesis for post-training is also more common because post-training data, including both instruction data and preference data, generally demands the most effort to produce. Using AI to pick the better response among several responses is more straightforward—much of it was already covered in [Chapter 3](#). The main challenge is to take into account the model’s biases, such as first-position bias, where the model is more likely to prefer the first option. To avoid this, NVIDIA researchers asked the AI judge twice, once with the response order swapped. They picked a valid (prompt, winning, losing) triplet only when the AI judge picked the same winner both times ([NVIDIA, 2024](#)).

The next section will focus on how to use AI to synthesize instruction data for supervised finetuning.

Instruction data synthesis

During instruction finetuning, each example includes an instruction and a response. AI can be used to synthesize the instructions, the responses, or both. For example, you can use AI to generate instructions and humans to write responses. You can also use humans to write instructions and AI to generate responses:

- For instruction generation, to ensure that you generate sufficient instructions to cover your use case, you can start with a list of topics, keywords, and/or the instruction types you want in your dataset. Then, for each item on this list, generate a certain number of instructions. You can also begin with a set of templates and generate a certain number of examples per template. Note that both the topic list and templates can be generated by AI.
- For response generation, you can generate one or more responses per instruction.

For instance, to create UltraChat ([Ding et al., 2023](#)), a multi-turn dialogue dataset, the authors first asked ChatGPT to generate 30 topics about various aspects of our daily lives, such as technology, food and drink, fashion, nature, education, finance, travel, etc. For each topic, they asked ChatGPT to generate 30 to 50 subtopics. The authors then used the same model to generate instructions and corresponding responses for these subtopics.

Similarly, to train Alpaca ([Taori et al., 2023](#)), Stanford researchers began with 175 (instruction, response) examples from the Self-Instruct seed dataset ([Wang et al., 2022](#)). These examples were originally written to cover a diverse and interesting range of uses. Alpaca authors then used a GPT-3 model, *text-davinci-003*, to generate 52,000 (instruction, response) pairs that mirrored these seed examples, as shown in [Figure 8-5](#).

Example seed task	Example generated task
<i>Instruction:</i> Brainstorm a list of possible New Year's resolutions. <i>Output:</i> • Lose weight • Exercise more • Eat healthier	<i>Instruction:</i> Brainstorm creative ideas for designing a conference room. <i>Output:</i> ... incorporating flexible components, such as moveable walls and furniture ...

Figure 8-5. A seed task and a generated task used to train Alpaca.

There are also many creative ways to synthesize instruction data with certain characteristics. For example, just like it's harder for humans to write longer content than shorter content, it's harder for AI to generate high-quality long responses than short instructions. The longer the response, the more chance AI has to hallucinate. What if we use human-generated responses with AI-generated instructions? Some researchers, such as [Köksal et al. \(2023\)](#), [Li et al. \(2023\)](#), and [Chen et al. \(2023\)](#), follow the *reverse instruction* approach: take existing long-form, high-quality content like stories, books, and Wikipedia articles and use AI to generate prompts that would elicit such content. This yields higher-quality instruction data, avoiding AI-generated hallucinations in the responses.

It's possible to use reverse instruction to develop increasingly powerful models without adding manually annotated data.¹¹ Li et al. (2023) shows how this works:

1. Start with a small number of seed examples to train a weak model.

2. Use this weak model to generate instructions for existing high-quality content to create high-quality instruction data.
3. Finetune the weak model with this new high-quality instruction data.
4. Repeat until desirable performance is reached.

A creative approach is to use synthetic data to finetune a model for understanding longer contexts. For example, if your current model processes a maximum of 8K tokens but you want it to handle 128K tokens, the long-context finetuning process might look like this:

- Split long documents into shorter chunks (e.g., under 8K tokens).
- For each short chunk, generate several (question, answer) pairs.
- For each (question, answer) pair, use the original long document, which may exceed 8K tokens but be shorter than your target length, as the context. This trains the model to use the extended context to answer questions.

The level of detail in the Llama 3 paper ([Dubey et al., 2024](#)) makes it an excellent case study for instruction data synthesis. I've already mentioned two ways in which Llama 3 synthesized data: code translation and code back-translation. Both of these methods generate more data from existing code snippets. However, the authors also used AI to synthesize coding instruction data from scratch, using the following workflow: